HZ BOOKS
华章科技

PEARSON

英伟达公司CUDA首席架构师Nicholas Wilt亲笔撰写，英伟达中国首批CUDA官方认证工程师翻译

全面而系统地讲解CUDA编程的各方面知识，深度解析CUDA各种优化技术，包含大量实用代码示例，是深入掌握主流异构并行计算技术的权威指南

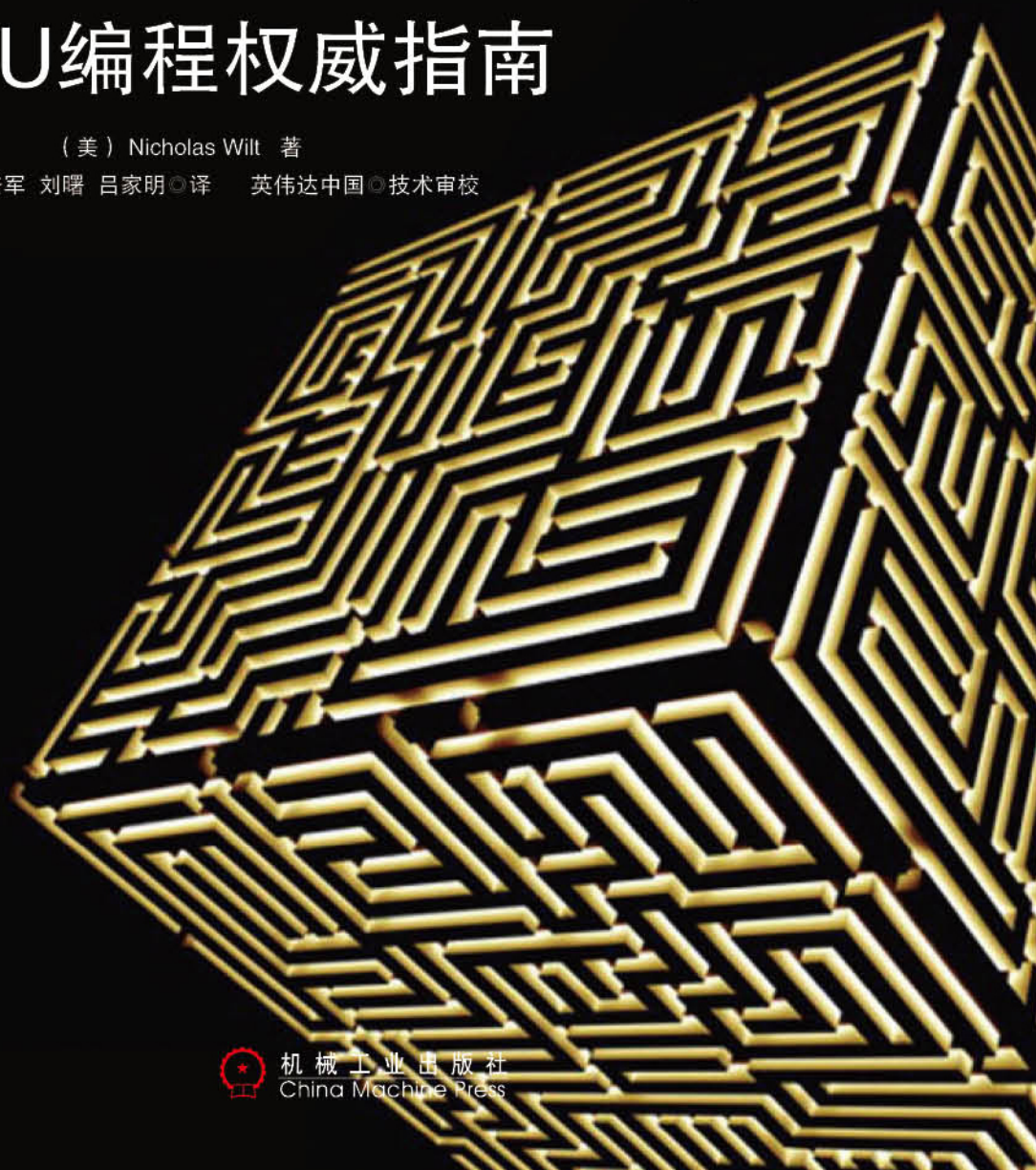
高性能计算系列丛书

The CUDA Handbook
A Comprehensive Guide to GPU Programming

CUDA专家手册
GPU编程权威指南

（美）Nicholas Wilt 著

苏统华 马培军 刘曙 吕家明 译　英伟达中国 技术审校



机械工业出版社
China Machine Press

高性能计算系列丛书

CUDA 专家手册：GPU 编程权威指南

The CUDA Handbook: A Comprehensive Guide to GPU Programming

（美） Nicholas Wilt 著

苏统华 马培军 刘曙 吕家明 译

英伟达中国 技术审校

HZ BOOKS
华章图书

机械工业出版社
China Machine Press

图书在版编目（CIP）数据

CUDA 专家手册：GPU 编程权威指南 /（美）威尔特（Wilt. N.）著；苏统华等译 . —北京：机械工业出版社，2014.8
（高性能计算系列丛书）
书名原文：The CUDA Handbook: A Comprehensive Guide to GPU Programming

ISBN 978-7-111-47265-0

I. C… II. ①威… ②苏… III. 图象处理 – 程序设计 – 手册 IV. TP391.41-62

中国版本图书馆 CIP 数据核字（2014）第 161732 号

本书版权登记号：图字：01-2013-5981

Authorized translation from the English language edition, entitled *The CUDA Handbook: A Comprehensive Guide to GPU Programming*, 9780321809469 by Nicholas Wilt, published by Pearson Education, Inc., Copyright © 2013.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

Chinese simplified language edition published by Pearson Education Asia Ltd., and China Machine Press Copyright © 2014.

本书中文简体字版由 Pearson Education（培生教育出版集团）授权机械工业出版社在中华人民共和国境内（不包括中国台湾地区和中国香港、澳门特别行政区）独家出版发行。未经出版者书面许可，不得以任何方式抄袭、复制或节录本书中的任何部分。

本书封底贴有 Pearson Education（培生教育出版集团）激光防伪标签，无标签者不得销售。

HZ BOOKS
华章图书

CUDA 专家手册：GPU 编程权威指南

[美] Nicholas Wilt 著

出版发行：机械工业出版社（北京市西城区百万庄大街 22 号 邮政编码：100037）
责任编辑：关 敏　　责任校对：董纪丽
印　　刷：　　版　　次：2014 年 8 月第 1 版第 1 次印刷
开　　本：186mm×240mm 1/16　　印　　张：23.25
书　　号：ISBN 978-7-111-47265-0　　定　　价：85.00 元

凡购本书，如有缺页、倒页、脱页，由本社发行部调换
客服热线：（010）88378991 88361066　　投稿热线：（010）88379604
购书热线：（010）68326294 88379649 68995259　　读者信箱：hzjsj@hzbook.com

版权所有 • 侵权必究
封底无防伪标均为盗版
本书法律顾问：北京大成律师事务所 韩光 / 邹晓东

Forword 中文版序

得知本书正在被翻译成瑰丽的汉字，我难抑内心的激动。众核计算对中国读者来说并不陌生。中国已经建造了多个最快的超级计算机，其中很多采用了 CUDA 技术。我们耳熟能详的天河 -1A，建成于 2010 年 10 月，配备了 7168 片特斯拉（Tesla）M2050 型号的 GPU。本书第 4 章介绍的亚马逊 cg1.4xlarge 实例，也采用了同种型号的 GPU。天河 -1A 曾雄霸世界超级计算机 500 强榜首逾半年之久，即使现在，它仍排在世界最快计算机的前 12 位。它惊人的高速度源于中国自主研发的互联技术把来自英特尔和英伟达的硬件高效地集成起来了。

当然，CUDA 并非只能应用于超级计算机。GPU 的出货量巨大，相较其他众核计算技术，它们具有价格优势。因此，支持 CUDA 的 GPU 很容易用来搭建经济的计算平台，这对于并行程序设计的教育和教学意义重大。

CUDA 技术仍在迅猛发展着。我敢断定，它将继续在并行计算的大潮中扮演重要角色。本书向大家展示了 CUDA 的内部工作机制，此中译本将推动 CUDA 在中国的普及和发展。

最后，向承担本书翻译工作的苏博士及其团队，致以诚挚谢意！感谢他们孜孜不倦地致力于 GPU 计算的推广，特别是把本书带给广大中国读者。

Nicholas Wilt

推荐序 *Forword*

自 CPU 单核性能在 2004 年左右停止提升后，功耗的限制及大量并行应用本身的特点决定了多核（CPU）加众核（GPU）的异构架构成为从最快的超级计算机到手机等移动设备上计算芯片的主流架构。所以如果想在现在和未来的任何计算设备上写出性能具有竞争力的程序，所有的开发者都需要学习异构架构下的并行计算技术。CUDA 是目前最成熟、使用者最多的异构并行计算技术。本书对开发者更深入地掌握主流异构并行计算技术会有很大帮助。

程序优化的目的是提高整体性能，所以优化时需要做的第一件事就是找出程序的哪个部分需要提高整体性能，即瓶颈分析。这是至关重要而又经常被刚接触优化的开发者忽略的一件事。如果不先进行定量的瓶颈分析，而是立刻优化自己认为是瓶颈的地方，经常会事倍功半。瓶颈分析的第一步是热点分析，即找出哪些部分占程序的大部分运行时间。热点分析可以通过在源代码里加时间测量代码或使用性能分析工具（profiler）进行。当确认整体瓶颈是 GPU 的函数或数据传输后，读者就可参阅本书对应的 GPU 优化知识进行优化。

本书第一部分共 4 章，介绍 GPU 的架构、系统软件及编程环境。性能优化本质上就是通过理解架构及相应系统软件的特点，并在程序中利用这些特点更高效地完成计算。所以优化的第一步就应该是理解目标平台的架构及系统软件基础。这部分讨论的很多内容是第一次在公开的 CUDA 书籍中出现。

第二部分共 6 章，介绍 CUDA 编程各个方面的具体知识，特别是如何利用 GPU 特点的很多优化技术。

如果性能分析告诉我们 CPU-GPU 数据传输是瓶颈，那么可以参考第 6 章使用流来计算隐藏数据传输；另一方面，如果 GPU 函数的执行是瓶颈，那么下一步就是分析内核函数的架构瓶颈。此时，定量分析也是至关重要的。如果不先做定量分析找出瓶颈在哪里，而是立刻随机地尝试各种优化技术，往往难以收到成效。这样随机尝试，可能有些优化恰好有效

果，有些优化则没有效果，会浪费大量时间在尝试各种技术上。另外，我们也无法明白为什么一种优化有效果而其他的却没有效果。除此之外，如果不做分析，我们也无从得知一个内核函数到底优化到何种程度就可以结束，从而有可能花费大量时间在一个已没有太大优化空间的内核函数上。这些都是优化技术的初学者很容易觉得优化是一件复杂而低效的工作的原因。但其实如果能采用先做定量分析再采用相应的优化技术的方法，优化完全可以变成一种高效、有据可依的技术。

对内核函数做分析，本质就是找出内核函数的性能瓶颈在架构的什么地方。一般情况下，性能瓶颈有三种可能类型。

1. 内存密集型。这种情况是内核函数的大部分时间都花在数据的读写指令上。从硬件的角度看，在这个内核函数运行期间，大部分时间都花在数据在内存总线的传输上并且内存总线的带宽已充分利用。对这种情况，如何优化可以参考第 5 章及第 10 章的内容。

2. 指令密集型。这种情况是内核函数的大部分时间都花在各种指令的执行上。从硬件的角度看，在这个内核函数运行期间，大部分时间都花在处理其指令单元的执行上。对这种情况，如何优化可以参考第 8 章的内容。

3. 延迟密集型。这种情况是内核函数的大部分时间都花在等待高延迟的指令（如内存读写）上。从硬件的角度看，在这个内核函数运行期间，大部分时间都花在数据在内存总线的传输上但内存总线的带宽没有被充分利用。对这种情况，如何优化可以参考第 7 章占有率的内容。

本书的第三部分共 5 章，给出了多个领域的具体例子，介绍如何应用各种优化技术。本部分的内容有助于读者学以致用。

虽然本书具体讨论 CUDA，但并行优化的技术及想法对几乎任何众核架构都是适用的。所以读者如果在阅读本书了解具体 CUDA 优化技术之外，也能有意识地体会蕴含其中的并行优化的基本方法，会有更大收获。

王鹏

英伟达高性能计算开发技术经理

译者序 *Words of the Translator's*

CUDA 入门相对比较容易。也许只需寥寥几个小时的学习，你就可以开始编写支持 CUDA 的程序，初窥 GPU 加速的魅力。但是要修炼成为榨取 GPU 处理能力的高级 CUDA 工程师，却并非易事。你需要了解并行计算的基本理论，需要研究 CPU/GPU 的硬件架构，需要熟悉 CUDA 的软件抽象架构和工具，需要掌握必要的领域知识，等等。目前，可以助你入门的 CUDA 书籍和教程为数不少，但在利用 CUDA 来深度榨取 GPU 潜力方面，仍几乎空白。

本书为深度优化 CUDA 程序性能提供了全方位的指导。乍看起来，本书只是一本参考手册。不错，它确实基于英伟达官方 CUDA 参考手册的很多内容。然而它又不单单是一本传统意义上的手册，它比较全面地介绍了 CUDA 的高级编程特性，是该领域难得的权威参考书。全书高屋建瓴而又简明扼要。作者从繁芜的文献资源中提炼并予以升华形成独到的总结。对很多复杂的主题，能够举重若轻；在很多关键的问题上，给大家一种豁然开朗之感。很多精辟论述是市面上任何一本 GPU 书籍所缺失的。译者在阅读本书时，亦为作者在技术和语言上的双重驾驭能力所折服。

本书作者 Wilt 从事 GPU 底层开发逾 20 年，功力深厚。在 CUDA 诞生之前，从事了 8 年 Direct3D 的开发；在英伟达供职的 8 年间，参与了 CUDA 的开发，实现了多数 CUDA 的抽象机制；目前转战亚马逊，从事 GPU 云产品的开发。作者根据自身多年的经验积淀，对 CUDA 编程模型进行了深入浅出的介绍，并结合四个典型应用场合，完美诠释了它们的具体用法和优化过程。得益于本书的启发，我们研究组基于 CUDA 的文字识别算法得到 194X 的加速，使我们能够抢在其他竞争者之前，取得 2013 年手写汉字识别竞赛的两项冠军。很多时候，速度意味着胜利！

本书的技术深度和欲覆盖的读者层面，决定了本书翻译工作的难度。本书的翻译持续

了一年之久，期间得到过很多人的帮助。除了本书列出的四位译者外，还有 5 位译者参与了本书的初稿翻译：韦振良参与了第 2 章和第 7 章，李硕参与了第 4 章和第 6 章，李松泽参与了第 10 章，孙黎参与了第 14 章，胡光龙参与了第 15 章。其中，与光龙是在参加第一届英伟达认证工程师的考试途中相遇，我们一起迷了路、一起考试过关、一起分享 CUDA 技术点滴，又一起在本书的翻译期间进行合作，谢谢光龙的友谊和无私帮助。在本书译文统稿之后，我们努力想找到合适的 CUDA 专家为我们把关。如果没有英伟达公司的侯宇涛经理，这可能会变成无法完成的任务。感谢侯经理为我们推荐了最优秀的技术审校专家，帮我们邀请了高性能计算开发技术中国区经理王鹏博士为本译作写序并选定本译作在全国推广。感谢英伟达高性能计算部门的专家王泽寰和赖俊杰，即使在春节期间，他们也在为改进本书的译稿质量而绞尽脑汁。如果没有泽寰和赖经理的审校，原书的内容恐难以尽可能原汁原味地展现。王鹏经理特意为本书作了推荐序，其中浓缩了他多年的 CUDA 性能优化经验；非常感谢，您真的做到了“读者最需要什么，您就写点什么”的宗旨。最后，还要特别感谢机械工业出版社的编辑团队，他们做了大量卓有成效的工作，是本书的幕后英雄。

本书的翻译，还得到了多项项目的资助，在此一并致谢。国家自然科学基金（资助号：61203260）、黑龙江省科研启动基金（资助号：LBH-Q13066）、哈尔滨工业大学科研创新基金（资助号：HITNSRIF2015083）对本书的翻译提供了部分资助。另外，哈尔滨工业大学创新实验课《 CUDA 高性能并行程序设计》、黑龙江省教育厅高等教育教学改革项目（资助号：JG2013010224）、哈尔滨工业大学研究生教育教学改革研究项目（资助号：JCJS-201309）也对本书的翻译提供了大力支持。

很多时候，翻译技术书籍是件吃力不讨好的事情。即使是最有资格的译者，也一定无法免受被读者指责的境遇。对读者负责，对 CUDA 教学推广负责，是我和我的团队始终铭记的准则。我们很享受翻译本书的过程，也很愿意承担因我们的水平和疏忽所招致的批评。文中可能存在的任何翻译问题，都是我们的责任。真诚地希望读者朋友不吝赐教，欢迎大家的任何意见：cudabook@gmail.com。

并行计算是计算的未来。希望本书的翻译可以帮助你让并行计算与大数据成功联姻，并享用由此带来的前所未有的新鲜体验。愿你的 CUDA 探索之旅愉快！

苏统华

哈尔滨工业大学软件学院

前　言 Preface

如果你正在读本书，意味着我无须再向你兜售 CUDA。你应该已经对 CUDA 有所了解，可能使用过英伟达的 SDK 和文档，抑或参加过并行程序设计的课程，再或者阅读过类似《 CUDA by Example 》（ Jason Sanders 和 Edward Kandrot 著，2011 年由 Addison-Wesley 出版）这样的入门书籍。

我在审校《 CUDA by Example 》时，惊诧于该书内容的浅显。它假设读者是零基础，试图描述从内存类型及其实际应用到图形互操作性，甚至到原子操作等方方面面的内容。它是很优秀的 CUDA 入门书籍，但也只能做到泛泛而谈。对于更深层次的知识，例如，平台的工作机理、GPU 的硬件结构、编译器驱动程序 nvcc 以及以前缀求和（“扫描”）为代表的基本并行算法，则鲜有涉及。

本书考虑到不同基础的读者，旨在帮助 CUDA 新手进阶中级水平，同时帮助中级程序员继续提升他们的水平到一个新高度。对于入门性质的书籍，最好从头到尾阅读，而对本书则可以根据需要选读。如果你正准备建立支持 CUDA 的新平台并在上面进行编程，建议你精读第 2 章。如果你正纳闷你的应用程序是否将受益于 CUDA 流带来的额外并发性，你应该查看第 6 章。其他的章节分别提供了软件架构、GPU 的纹理操作和流处理器簇等 GPU 子系统的详细描述，还有依据不同数据存取模式和在并行算法领域的重要程度而精心挑选的应用案例。尽管不同章之间难免存在交互引用，但每一章的内容都是相对独立的。

本书将披露包括 CUDA 5.0 在内的最新技术。最近几年，CUDA 和它的目标平台得到了长足发展。在《 CUDA by Example 》出版之际，GeForce GTX 280（GT200）才刚刚面世。迄今，CUDA 硬件已历经两代演变。因此，本书除了在如映射锁页内存（mapped pinned memory）的现有特性上不吝笔墨外，还特别关注 CUDA 支持的新特性，像费米架构的 ballot、开普勒架构的 shuffle、64 位和统一虚拟寻址特性以及动态并行（dynamic parallelism）

等。此外，本书还将讨论最近的平台进展，例如英特尔沙桥（Sandy Bridge）CPU 上集成的 PCIe 总线控制器。

尊敬的读者，你可以全篇通读本书，也可以把它放于电脑边随时查阅。但不管怎样，我真诚地希冀你能从中得到乐趣，如同我执笔分享时一样快意。

致谢

借此机会，感谢英伟达公司的朋友们。他们耐心地为我释疑、检视我的作品并反馈建设性意见。特别的谢意送给 Mark Harris、Norbert Juffa 和 Lars Nyland。

本书的审校者在成稿之前审阅了本书，他们付出了大量时间，提供的宝贵意见提高了本书的质量和清晰性，保证了技术的正确性。在此，特别感谢 Andre Brodtkorb、Scott Le Grand、Allan MacKinnon、Romelia Salomon-Ferrer 和 Patrik Tennberg 的反馈意见。

本书的写作过程历经了重重困难，如果没有编辑 Peter Gordon 的超凡耐心和支持，很难最终呈现给大家。Peter 的助手 Kim Boedigheimer 为本项目的顺利完成做了大量工作，帮助我设定了项目的各种专业标准。对她在征求、协调评审意见以及在本书成稿之后把本书上传到 Safari 网站的整个过程中付出的努力，表示特别感谢。

在本书的写作过程中，我的妻子 Robin 和我的儿子 Benjamin、Samuel 和 Gregory 一直对我支持有加，谢谢他们。

HZ BOOKS
华章图书

目　录 *Contents*

中文版序
推荐序
译者序
前　言

第一部分　基础知识

第 1 章　简介 ····· 2

1.1　方法 ····· 4
1.2　代码 ····· 4
1.2.1　验证型代码 ····· 5
1.2.2　演示型代码 ····· 5
1.2.3　探究型代码 ····· 5
1.3　资源 ····· 5
1.3.1　开源代码 ····· 5
1.3.2　CUDA 专家手册库（chLib） ····· 6
1.3.3　编码风格 ····· 6
1.3.4　CUDA SDK ····· 6
1.4　结构 ····· 6

第 2 章　硬件架构 ····· 8

2.1　CPU 配置 ····· 8

2.1.1 前端总线 ····· 9
2.1.2 对称处理器簇 ····· 9
2.1.3 非一致内存访问（NUMA） ····· 10
2.1.4 集成的 PCIe ····· 12
2.2 集成 GPU ····· 13
2.3 多 GPU ····· 14
2.4 CUDA 中的地址空间 ····· 17
2.4.1 虚拟寻址简史 ····· 17
2.4.2 不相交的地址空间 ····· 20
2.4.3 映射锁页内存 ····· 21
2.4.4 可分享锁页内存 ····· 21
2.4.5 统一寻址 ····· 23
2.4.6 点对点映射 ····· 24
2.5 CPU/GPU 交互 ····· 24
2.5.1 锁页主机内存和命令缓冲区 ····· 25
2.5.2 CPU/GPU 并发 ····· 26
2.5.3 主机接口和内部 GPU 同步 ····· 29
2.5.4 GPU 间同步 ····· 31
2.6 GPU 架构 ····· 31
2.6.1 综述 ····· 31
2.6.2 流处理器簇 ····· 34
2.7 延伸阅读 ····· 37

第 3 章 软件架构 ····· 39

3.1 软件层 ····· 39
3.1.1 CUDA 运行时和驱动程序 ····· 40
3.1.2 驱动程序模型 ····· 41
3.1.3 nvcc、PTX 和微码 ····· 43
3.2 设备与初始化 ····· 45
3.2.1 设备数量 ····· 46
3.2.2 设备属性 ····· 46

HZ BOOKS 华章图书

3.2.3 无 CUDA 支持情况 ····· 48
3.3 上下文 ····· 50
3.3.1 生命周期与作用域 ····· 51
3.3.2 资源预分配 ····· 51
3.3.3 地址空间 ····· 52
3.3.4 当前上下文栈 ····· 52
3.3.5 上下文状态 ····· 53
3.4 模块与函数 ····· 53
3.5 内核（函数） ····· 55
3.6 设备内存 ····· 56
3.7 流与事件 ····· 57
3.7.1 软件流水线 ····· 57
3.7.2 流回调 ····· 57
3.7.3 NULL 流 ····· 57
3.7.4 事件 ····· 58
3.8 主机内存 ····· 59
3.8.1 锁页主机内存 ····· 60
3.8.2 可分享的锁页内存 ····· 60
3.8.3 映射锁页内存 ····· 60
3.8.4 主机内存注册 ····· 60
3.9 CUDA 数组与纹理操作 ····· 61
3.9.1 纹理引用 ····· 61
3.9.2 表面引用 ····· 63
3.10 图形互操作性 ····· 63
3.11 CUDA 运行时与 CUDA 驱动程序 API ····· 65

第 4 章 软件环境 ····· 69

4.1 nvcc——CUDA 编译器驱动程序 ····· 69
4.2 ptxas——PTX 汇编工具 ····· 73
4.3 cuobjdump ····· 76
4.4 nvidia-smi ····· 77

HZ BOOKS 华章图书

4.5 亚马逊 Web 服务 ····· 79
4.5.1 命令行工具 ····· 79
4.5.2 EC2 和虚拟化 ····· 79
4.5.3 密钥对 ····· 80
4.5.4 可用区域（AZ）和地理区域 ····· 81
4.5.5 S3 ····· 81
4.5.6 EBS ····· 81
4.5.7 AMI ····· 82
4.5.8 EC2 上的 Linux ····· 82
4.5.9 EC2 上的 Windows ····· 83

第二部分 CUDA 编程

第 5 章 内存 ····· 88
5.1 主机内存 ····· 89
5.1.1 分配锁页内存 ····· 89
5.1.2 可共享锁页内存 ····· 90
5.1.3 映射锁页内存 ····· 90
5.1.4 写结合锁页内存 ····· 91
5.1.5 注册锁页内存 ····· 91
5.1.6 锁页内存与统一虚拟寻址 ····· 92
5.1.7 映射锁页内存用法 ····· 92
5.1.8 NUMA、线程亲和性与锁页内存 ····· 93
5.2 全局内存 ····· 95
5.2.1 指针 ····· 96
5.2.2 动态内存分配 ····· 97
5.2.3 查询全局内存数量 ····· 100
5.2.4 静态内存分配 ····· 101
5.2.5 内存初始化 API ····· 102
5.2.6 指针查询 ····· 103

HZ BOOKS 华章图书

5.2.7 点对点内存访问 ····· 104
5.2.8 读写全局内存 ····· 105
5.2.9 合并限制 ····· 105
5.2.10 验证实验：内存峰值带宽 ····· 107
5.2.11 原子操作 ····· 111
5.2.12 全局内存的纹理操作 ····· 113
5.2.13 ECC（纠错码） ····· 113
5.3 常量内存 ····· 114
5.3.1 主机与设备常量内存 ····· 114
5.3.2 访问常量内存 ····· 114
5.4 本地内存 ····· 115
5.5 纹理内存 ····· 118
5.6 共享内存 ····· 118
5.6.1 不定大小共享内存声明 ····· 119
5.6.2 束同步编码 ····· 119
5.6.3 共享内存的指针 ····· 119
5.7 内存复制 ····· 119
5.7.1 同步内存复制与异步内存复制 ····· 120
5.7.2 统一虚拟寻址 ····· 121
5.7.3 CUDA 运行时 ····· 121
5.7.4 驱动程序 API ····· 123

第 6 章 流与事件 ····· 125

6.1 CPU/GPU 的并发：隐藏驱动程序开销 ····· 126
6.2 异步的内存复制 ····· 129
6.2.1 异步的内存复制：主机端到设备端 ····· 130
6.2.2 异步内存复制：设备端到主机端 ····· 130
6.2.3 NULL 流和并发中断 ····· 131
6.3 CUDA 事件：CPU/GPU 同步 ····· 133
6.3.1 阻塞事件 ····· 135
6.3.2 查询 ····· 135

6.4 CUDA 事件：计时 ····· 135
6.5 并发复制和内核处理 ····· 136
6.5.1 concurrencyMemcpyKernel.cu ····· 137
6.5.2 性能结果 ····· 141
6.5.3 中断引擎间的并发性 ····· 142
6.6 映射锁页内存 ····· 143
6.7 并发内核处理 ····· 145
6.8 GPU/GPU 同步：cudaStreamWaitEvent() ····· 146
6.9 源代码参考 ····· 147

第 7 章 内核执行 ····· 148

7.1 概况 ····· 148
7.2 语法 ····· 149
7.2.1 局限性 ····· 150
7.2.2 高速缓存和一致性 ····· 151
7.2.3 异步与错误处理 ····· 151
7.2.4 超时 ····· 152
7.2.5 本地内存 ····· 152
7.2.6 共享内存 ····· 153
7.3 线程块、线程、线程束、束内线程 ····· 153
7.3.1 线程块网格 ····· 153
7.3.2 执行保证 ····· 156
7.3.3 线程块与线程 ID ····· 156
7.4 占用率 ····· 159
7.5 动态并行 ····· 160
7.5.1 作用域和同步 ····· 161
7.5.2 内存模型 ····· 162
7.5.3 流与事件 ····· 163
7.5.4 错误处理 ····· 163
7.5.5 编译和链接 ····· 164
7.5.6 资源管理 ····· 164

HZ BOOKS 华章图书

7.5.7 小结 …… 165

第 8 章 流处理器簇 …… 167

8.1 内存 …… 168
8.1.1 寄存器 …… 168
8.1.2 本地内存 …… 169
8.1.3 全局内存 …… 170
8.1.4 常量内存 …… 171
8.1.5 共享内存 …… 171
8.1.6 栅栏和一致性 …… 173
8.2 整型支持 …… 174
8.2.1 乘法 …… 174
8.2.2 其他操作（位操作） …… 175
8.2.3 漏斗移位（SM 3.5） …… 175
8.3 浮点支持 …… 176
8.3.1 格式 …… 176
8.3.2 单精度（32 位） …… 180
8.3.3 双精度（64 位） …… 181
8.3.4 半精度（16 位） …… 181
8.3.5 案例分析：float 到 half 的转换 …… 182
8.3.6 数学函数库 …… 185
8.3.7 延伸阅读 …… 190
8.4 条件代码 …… 191
8.4.1 断定 …… 191
8.4.2 分支与汇聚 …… 191
8.4.3 特殊情况：最小值、最大值和绝对值 …… 192
8.5 纹理与表面操作 …… 193
8.6 其他指令 …… 193
8.6.1 线程束级原语 …… 193
8.6.2 线程块级原语 …… 194
8.6.3 性能计数器 …… 195

HZ BOOKS 华章图书

8.6.4 视频指令 …… 195
8.6.5 特殊寄存器 …… 196
8.7 指令集 …… 196

第 9 章 多 GPU …… 203

9.1 概述 …… 203
9.2 点对点机制 …… 204
9.2.1 点对点内存复制 …… 204
9.2.2 点对点寻址 …… 205
9.3 UVA：从地址推断设备 …… 206
9.4 多 GPU 间同步 …… 207
9.5 单线程多 GPU 方案 …… 208
9.5.1 当前上下文栈 …… 208
9.5.2 N– 体问题 …… 210
9.6 多线程多 GPU 方案 …… 212

第 10 章 纹理操作 …… 216

10.1 简介 …… 216
10.2 纹理内存 …… 217
10.2.1 设备内存 …… 217
10.2.2 CUDA 数组与块的线性寻址 …… 218
10.2.3 设备内存与 CUDA 数组对比 …… 222
10.3 一维纹理操作 …… 223
10.4 纹理作为数据读取方式 …… 226
10.4.1 增加有效地址范围 …… 226
10.4.2 主机内存纹理操作 …… 228
10.5 使用非归一化坐标的纹理操作 …… 230
10.6 使用归一化坐标的纹理操作 …… 237
10.7 一维表面内存的读写 …… 238
10.8 二维纹理操作 …… 240
10.9 二维纹理操作：避免复制 …… 242

HZ BOOKS 华章图书

10.9.1 设备内存上的二维纹理操作 ····· 242
10.9.2 二维表面内存的读写 ····· 243
10.10 三维纹理操作 ····· 244
10.11 分层纹理 ····· 245
10.11.1 一维分层纹理 ····· 246
10.11.2 二维分层纹理 ····· 246
10.12 最优线程块大小选择以及性能 ····· 246
10.13 纹理操作快速参考 ····· 248
10.13.1 硬件能力 ····· 248
10.13.2 CUDA 运行时 ····· 249
10.13.3 驱动 API ····· 250

第三部分 实例

第 11 章 流式负载 ····· 254
11.1 设备内存 ····· 255
11.2 异步内存复制 ····· 258
11.3 流 ····· 259
11.4 映射锁页内存 ····· 260
11.5 性能评价与本章小结 ····· 261

第 12 章 归约算法 ····· 263
12.1 概述 ····· 263
12.2 两遍归约 ····· 265
12.3 单遍归约 ····· 269
12.4 使用原子操作的归约 ····· 271
12.5 任意线程块大小的归约 ····· 272
12.6 适应任意数据类型的归约 ····· 273
12.7 基于断定的归约 ····· 276
12.8 基于洗牌指令的线程束归约 ····· 277

HZ BOOKS 华章图书

第 13 章　扫描算法 ····· 278

13.1　定义与变形 ····· 278
13.2　概述 ····· 279
13.3　扫描和电路设计 ····· 281
13.4　CUDA 实现 ····· 284
13.4.1　先扫描再扇出 ····· 284
13.4.2　先归约再扫描（递归） ····· 288
13.4.3　先归约再扫描（两阶段） ····· 291
13.5　线程束扫描 ····· 294
13.5.1　零填充 ····· 295
13.5.2　带模板的版本 ····· 296
13.5.3　线程束洗牌 ····· 297
13.5.4　指令数对比 ····· 298
13.6　流压缩 ····· 300
13.7　参考文献（并行扫描算法） ····· 302
13.8　延伸阅读（并行前缀求和电路） ····· 303

第 14 章　N- 体问题 ····· 304

14.1　概述 ····· 305
14.2　简单实现 ····· 309
14.3　基于共享内存实现 ····· 312
14.4　基于常量内存实现 ····· 313
14.5　基于线程束洗牌实现 ····· 315
14.6　多 GPU 及其扩展性 ····· 316
14.7　CPU 的优化 ····· 317
14.8　小结 ····· 321
14.9　参考文献与延伸阅读 ····· 323

第 15 章　图像处理的归一化相关系数计算 ····· 324

15.1　概述 ····· 324
15.2　简单的纹理实现 ····· 326

HZ BOOKS 华章图书

15.3 常量内存中的模板 ····· 329
15.4 共享内存中的图像 ····· 331
15.5 进一步优化 ····· 334
15.5.1 基于流处理器簇的实现代码 ····· 334
15.5.2 循环展开 ····· 335
15.6 源代码 ····· 336
15.7 性能评价 ····· 337
15.8 延伸阅读 ····· 339

附录 A CUDA 专家手册库 ····· 340

术语表 ····· 347

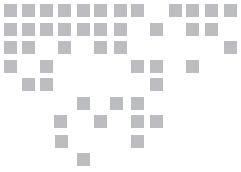
HZ BOOKS
华章图书

第一部分 *Part 1*

基础知识

HZ BOOKS
华章图书

- 第 1 章　简介
- 第 2 章　硬件架构
- 第 3 章　软件架构
- 第 4 章　软件环境



Chapter 1 第 1 章

简　介

GPU 带给计算领域以巨大变革，这已在历史上留下了浓墨重彩的一笔。我喜欢饶有兴趣地阅读这类文字，因为我很早就参与其中了。在 20 世纪 90 年代中期，我任职于微软。当时英特尔和 AMD 正在引入第一批用以加速浮点计算的多媒体指令集，我领导一个开发小组开发 Direct3D 产品。英特尔为了阻止移植他们的 CPU 计算能力到三维栅格化任务，他们尝试跟微软合作出售使用他们多媒体扩展（MMX）指令集的光栅处理器，但最终失败了。当 MMX 光栅处理器运行在尚未发布的奔腾 2 处理器上时，我们发现其速度仅是正在出售的不起眼的 S3 Virge GX 光栅处理器的一半。那时我就知道这种努力注定是要失败的。

就 Direct3D 6.0 而言，我们与 CPU 厂商一起将他们的代码集成到我们的几何流水线（geometry pipeline），使开发人员可以以透明的方式使用来自英特尔和 AMD 的新指令集，并受益于厂商优化的代码执行路径。游戏开发人员接受了新的几何流水线，但由于新的指令集被用来生成 GPU 硬件的几何流水线所需要的顶点数据，因此，它没能阻止将 CPU 计算能力输送给 GPU 的持续迁移。

在那个时期，GPU 上的晶体管数量超过了 CPU 上的数量。转折点在 1997 ~ 1998 年间，当时奔腾 2 和英伟达 RIVA TNT 的晶体管数量都约为 800 万个。随后，GeForce 256（1500 万个晶体管）、GeForce 2（2800 万个晶体管）和 GeForce 3（6300 万个晶体管）的晶体管数量均超过了当时的 CPU。此外，两种设备之间的差异越来越清楚：大部分的 CPU 芯片面积是为了支持缓存，而大部分的 GPU 芯片面积是为了逻辑。英特尔能够添加大量新的扩展指令集（MMX、SSE、SSE2 等），而增加的面积代价几乎可以忽略不计。GPU 是专为并行吞吐处理而设计的，它们的小规模缓存是为了带来更多的带宽聚合（bandwidth aggregation）而不是为了减少指令延迟。

虽然 ATI 和英伟达等公司一直在生产越来越快速和越来越强大的 GPU，CPU 厂商仍然在摩尔定律（Moore's Law）㊀ 的指引下追求更高的时钟速率。第一款奔腾处理器（1993 年）的时钟速率为 60MHz，而启用 MMX 的奔腾处理器（1997 年）的时钟速率为 200MHz。到 20 世纪末，时钟速率已超过 1000MHz。但此后不久，计算史上发生了一桩重大事件：摩尔定律碰了钉子。晶体管将继续变小，但时钟速率可能不会再继续增加。

这一事件并非意料之外。英特尔的 Pat Gelsinger 在 2001 年美国电气和电子工程师协会（IEEE）举办的固态电路会议上发表主题演讲，曾表示：如果继续目前的设计思路，芯片会在 2010 年跟核反应堆一样热，而到 2015 年将达到太阳表面的热度。在未来，性能将来自“同步多线程”（simultaneous multithreading，SMT）技术，可能要通过把多个 CPU 核心集成到一个芯片上来达到目的。事实上，CPU 厂商已经这么做了。今天，很难找到一个只有单个 CPU 核心的桌面 PC。几十年来，摩尔定律带给软件开发人员搭便车（free ride）的机会：提高 CPU 时钟频率，几乎根本不用软件开发人员操心，就允许程序运行更快。但这种好事将成为过去。多核心 CPU 需要多线程应用程序。只有当应用程序可以并行化时，众多 CPU 核心才能带来预期的高性能。

GPU 的定位很好地利用了摩尔定律的新趋势。虽然那些未曾考虑并行化的 CPU 应用程序将需要大量的重构（如果它们有并行化的可能），然而在图形应用程序上已经采用了可以利用各独立像素之间固有的并行模式的思路。对于 GPU 而言，通过增加执行核心的数量来提高性能是一个很自然的手段。事实上，GPU 设计者往往倾向于添加更多的核心而不是更强大的核心。他们摒弃了以最大限度地提高时钟频率（GPU 的发展路线从来不是、现在依然不是靠制造接近晶体管电路极限的时钟频率。）、预测执行（speculative execution）、分支预测（branch prediction）和存储转发（store forwarding）等为代表的 CPU 厂商认为理所当然的策略。为了防止更加强大的处理器受限于 I/O 速度（I/O bound），GPU 设计者结合内存控制器，并与内存厂商一起研制可以远远超过 CPU 可用带宽很多的 GPU 内存带宽。

但是，GPU 的强劲计算能力对于非图形开发者来说很难利用。一些充满探索精神的程序员借助如 Direct3D 和 OpenGL 的图形 API，通过迷惑图形硬件来执行非图形的计算任务。这一方法被冠以专门的术语：通用 GPU 编程（general-purpose GPU programming，GPGPU）。但 GPU 的大部分计算潜力仍一直处于未发掘状态，直到 CUDA 的出现改变了这一切。Ian Buck 当时在斯坦福大学开展一项名为 Brook 的项目，意在简化 GPGPU 应用程序的开发过程。Buck 后来进入英伟达并领导开发了一套新的开发工具，这使得 GPU 上非图形应用程序的开发更容易。这个英伟达的专用工具套装就是 CUDA。CUDA 允许 C 语言程序员使用一些简单易用的语言扩展编写 GPU 并行代码。

自 2007 年推出以来，CUDA 一直深受好评。数以万计使用该技术的研究论文已被发表。

㊀ 摩尔定律主要有三种不同说法：集成电路芯片上所集成的电路的数目，每隔 18 个月就翻一倍；微处理器的性能每隔 18 个月提高一倍，或价格下降一半；用一美元所能买到的计算机性能，每隔 18 个月翻一倍。——译者注

它已被用于多个商业软件包，跨度从 Adobe 的 CS5 到 Manifold 的地理信息系统（geographic information system，GIS）。对于适合的工作任务，运行在支持 CUDA 的 GPU 上，相比同年代 CPU，可以获得 5 ~ 400 倍不等的加速。这些加速的来源各不相同。有的加速是因为 GPU 有更多的核心；有的是因为有更高的内存带宽；还有一些是因为应用程序可以利用在 CPU 中不曾有的专门 GPU 硬件，例如可以更快计算超越函数的纹理硬件或特殊函数单元（SFU）。但并非所有的应用程序都可以采用 CUDA 实现。更确切地说，并非所有的并行应用程序都可以采用 CUDA 实现。但 CUDA 已使用在各种应用中，比任何其他的 GPU 计算技术的应用更广泛。我希望本书可以帮助有一定 CUDA 基础的开发者更有效地使用 CUDA。

1.1 方法

写一本以 CUDA 为主题的书籍是件困难的事情。并行程序设计需要涉及的因素很多，即使暂不考虑操作系统因素（Windows、Linux 和 MacOS）、平台因素（特斯拉与费米，集成显卡与独立显卡，多 GPU）、CPU/GPU 并发因素以及 CUDA 的特定因素（例如，决定是使用 CUDA 运行时还是驱动程序 API 编写代码），也是相当复杂的。当你还需要考虑如何最好地组织 CUDA 内核程序时，其复杂程度可能让人窒息。

本书为了应对这一复杂性，在介绍多数主题时从不同角度不止一次地进行解释。“纹理映射硬件做哪些事情?”这一问题跟“怎样写一个进行纹理映射操作的内核程序?”是不同的。本书分别在不同章节回答这两个问题。异步内存复制操作可以在几处不同的主题下来解释：软件的抽象（例如，参与的主机内存必须分配为锁页内存）、不同的硬件实现、支持该功能的 API 以及优化策略之间的相互作用。读者有时不妨查阅索引，然后选取相应主题下的多处文本来阅读。

性能优化指南通常组织得像咨询栏目。很多时候，它们给出的指导缺乏足够的实际应用场景，而且往往前后矛盾。上述观点不是为了斥责它们，它们只是问题复杂性的一个症状而已。优化 CPU 的金律，是经历了至少 20 年的持续探索，而 GPU 是更难进行编写程序的，所以期望 CUDA 的优化建议很简单就有点不切实际了。

此外，GPU 计算是新生事物，即使是 GPU 架构师仍在学习最佳的编程方式，更不用说开发人员了。对 CUDA 开发人员而言，终极决定因素是性能，而性能通常采用系统时间来衡量！网格和线程块大小、何时以及如何使用共享内存、每个线程应该计算多少次运算以及占用率的数值，都可能影响性能。只有通过实现多种方法并测量每个方法的实际表现，才能给出切实的推荐。

1.2 代码

开发人员希望 CUDA 代码能说明问题而不仅仅是玩具，有实用性而无须具备冷门主题

上的技术储备以及有高性能且不能掩盖实现从初始移植到做出最终版本之间的尝试过程。为此，本书介绍了 3 种类型的代码示例，旨在分别迎合上述需求：验证型（microbenchmark）、演示型（microdemo）和探究型（optimization journey）。

1.2.1 验证型代码

验证型代码旨在说明一个非常具体的 CUDA 问题对性能的影响，例如，非合并的内存事务如何降低设备的内存带宽或执行内核转换（kernel thunk）会需要 WDDM 驱动程序多长时间才能完成。它们被设计成可以独立编译的程序并跟许多 CUDA 程序员自己实现的验证代码很相似。从某种意义上说，我编写了这一组验证型代码，是为了节省大家的时间。

1.2.2 演示型代码

演示型代码是小型应用程序，旨在揭示具体硬件或软件的行为问题。跟验证型代码一样，它们都很小、自成一体，但以强调功能性为主而不是强调性能。例如，纹理操作一章包括的演示型代码分别用以说明如何从 1D 设备内存得到纹理、如何进行浮点型到整型的转换、多种纹理寻址模式是如何工作的以及纹理的线性插值是如何被 9 位的权重所影响的。

跟验证型代码一样，这些演示型代码是考虑到开发者可能会编写它们或者需要它们。本书提供之后，大家就不必再去编写了。

1.2.3 探究型代码

基于 CUDA 的许多论文仅呈现所用方法的实验结果。也许会附带解释一句：文中方法是在调查多种方法并进行权衡后才采用的。作者往往受到篇幅和截止日期的限制，无法给出他们工作的更多细节。

基于 CUDA 进行数据并行编程比较核心的主题，本书给出探究型代码。逐步加大所考察问题的复杂性，不断优化程序性能。这是 Mark Harris 在《基于 CUDA 优化并行归约》[㊀]一文中的处理思路。本书精选的几个案例均采用探究式分析，包括归约、并行前缀和（即“扫描”）和 N- 体问题。

1.3 资源

1.3.1 开源代码

本书的源代码可以在 www.cudahandbook.com 网站上下载。它们是开源的，著作权遵照 2 句版 BSD 许可证[㊁]。

㊀ http://bit.ly/Z2q37x。

㊁ www.opensource.org/licenses/bsd-license.php。

1.3.2 CUDA 专家手册库（chLib）

CUDA 专家手册库，可以在源代码的 chLib/ 目录下找到。它包含一个可以移植的库文件，支持计时、线程化操作、驱动程序 API 辅助工具等内容。在附录 A 中有针对它的更详细描述。

1.3.3 编码风格

不考虑括号里的参数，本书代码的主要特征，是基于 goto 方式的错误处理机制。这可能会招致大家的批评。执行多个资源分配（或者其他可能出现失败的操作，并且失败会传递给调用者）的函数采用类似 Linux 内核代码的常用方式，按照初始化 / 错误检查 / 资源清理三步法模式来组织。

如果出现失败，所有的资源清理工作在函数末尾执行同一代码来完成。很重要的一点，记得在函数的开始部分初始化资源为特定的无效值，这样清理代码知道哪些资源必须被释放。如果资源分配或其他功能失败，代码执行一次 goto（清理代码）。A.6 节的 chError.h 文件中描述了使用 CUDA 运行时和驱动程序 API 实现此代码模式的错误处理宏。

1.3.4 CUDA SDK

SDK 为所有 CUDA 开发者提供一个共享的经验，所以我们假设你已经了安装 CUDA SDK，并且可以使用它来生成 CUDA 程序。该 SDK 还包括 GLUT（GL 实用库），便于从同一批程序代码中得到面向不同操作系统的 OpenGL 应用程序。GLUT 旨在生成演示级别的而不是产品级别的应用程序，但它符合我们的需求。

1.4 结构

本书按照逻辑结构分成三部分。第一部分包括第 1 ~ 4 章，概述 CUDA 硬件和软件的架构。

第 2 章详细介绍 CUDA 的硬件平台和 GPU 本身。

第 3 章介绍 CUDA 的软件架构。

第 4 章介绍 CUDA 软件环境，包括 CUDA 软件工具描述和亚马逊的 ECI 环境。

第二部分包括第 5 ~ 10 章，深入介绍 CUDA 编程模型的方方面面。

第 5 章涵盖内存，包括设备内存、常量内存、共享内存和纹理内存。

第 6 章介绍流和事件，这一机制用于 CPU 和 GPU 之间、GPU 不同硬件单元（例如，复制引擎和流处理器簇）之间或者多个独立 GPU 之间的“粗 – 细粒度”结合的并行处理。

第 7 章涵盖内核执行，包括动态并行这一 CUDA 5.0 和 SM 3.5 中的新特性。

第 8 章涵盖流处理器簇的各个方面。

第 9 章涵盖多 GPU 应用程序，包括以 N- 体为例的点对点操作和高难度的并行操作。

第 10 章涵盖 CUDA 纹理的各个方面。

最后，在 11 ~ 15 章的第三部分讨论几个有针对性的 CUDA 应用程序。

第 11 章介绍带宽限制、以向量乘法为例的流式负载等内容。

第 12 和 13 章描述归约和并行前缀求和（也称为“扫描”）算法，两者都是并行编程的重要基石。

第 14 章介绍 N- 体应用，这是一族可以大大受益于 GPU 计算的重要的高计算密集型应用。

第 15 章深入考查称为归一化相关系数特征提取的图像处理操作。第 15 章提供的代码是本书中唯一同时使用纹理和共享内存以提供最佳性能的地方。

HZ BOOKS
华章图书

Chapter 2 第 2 章

硬件架构

本章分别从 GPU 的系统层面到功能单元对 CUDA 平台进行更为具体的描述。前三节讨论多种构建 CUDA 系统的不同方法；第四节主要讨论了地址空间以及 CUDA 的内存模型是如何在软硬件上实现的；第五节主要讨论 CPU/GPU 交互，并且特别关注指令如何提交到 GPU 以及 CPU/GPU 是如何进行同步的；最后，针对复制引擎和流处理器簇等功能单元进行了宏观描述，并特意给出了支持 CUDA 的三代硬件上流处理器簇的框图。

2.1 CPU 配置

本节及随后的两节介绍各种 CPU/GPU 架构，并对 CUDA 开发者如何编程方面给出了一些意见和建议。我们研究了 CPU 配置、集成 GPU 和多 GPU 配置。现在，让我们从图 2-1 开始谈起。

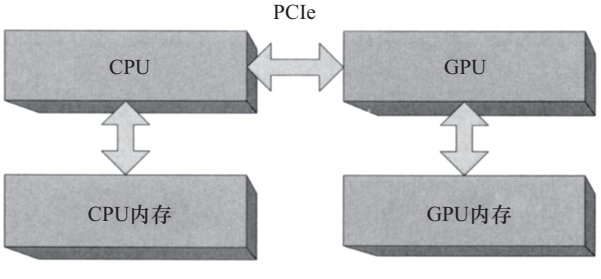


图 2-1 CPU/GPU 架构简略图

图 2-1 中省略了一个重要元素，那就是连接 CPU 与外界的“芯片组”(“核心逻辑”)。系统每一比特的输入与输出，包括磁盘、网络控制器、键盘、鼠标、USB 设备以及 GPU 的输入输出，都要通过芯片组。直到最近，芯片组被一分为二：一个是连接大多外围设备和系统的 “南桥” ㊀，另一个是包含图形总线（加速图形

㊀ 为简单起见，本节的所有图中均略去南桥。

端口，后被 PCIe 接口取代）和内存控制器（通过前端总线与内存相连）的“北桥”。

每一个 PCIe（peripheral communications interconnect express）2.0 的“通道”（lane）理论上可以提供 500MB/s 的带宽。对于一个给定的外设，其通道数可以是 1、4、8 或 16 个。GPU 需要平台上所有外设的最大带宽，所以它们一般被插入由 16 通道构成的 PCIe 插槽。考虑到数据包的附加消耗，该连接的 8G/s 带宽实际上能达到 6G/s。[㊀]

2.1.1　前端总线

图 2-2 是将北桥和内存控制器添加到图 2-1 后的简化图。为了完整起见，图 2-2 还显示了 GPU 的集成内存控制器，它是在一个与 CPU 内存控制器完全不同的约束集下设计的。GPU 必须调解所谓的同步客户端，例如视频显示，其带宽要求是固定的。GPU 内存控制器在设计时还考虑了 GPU 对延迟的容忍度以及大量内存带宽方面的内在需求。在写作本书时，高端 GPU 通常所提供的本地显存带宽大大超过 100G/s。GPU 内存控制器总是和 GPU 集成，因此本章其余部分的图示中省略了它们。

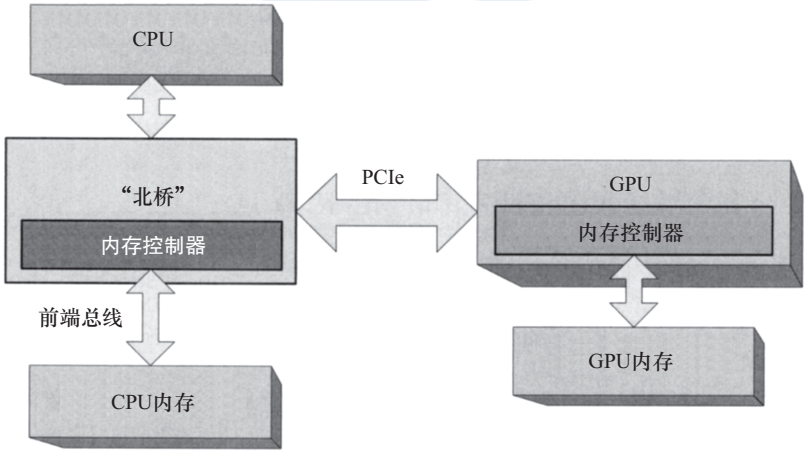


图 2-2　CPU/GPU 架构（带北桥）

2.1.2　对称处理器簇

图 2-3 显示的是一个传统北桥配置上的多 CPU 系统。[㊁]多核处理器之前，应用程序必须使用多线程以便充分利用多 CPU 的额外运算能力。即使每个 CPU 和北桥本身都包含有缓存，北桥也必须确保每个 CPU 看到相同的、一致的内存视图。由于这些所谓的“对称处理器簇”（SMP）系统共享同一个通往 CPU 内存的路径，不同 CPU 的内存访问性能

㊀　PCI 3.0 所能提供的带宽相当于 PCIe 2.0 的两倍。

㊁　我们之所以提供图 2-3，不只是因为该配置是支持 CUDA 的计算机，更是作为历史资料参照。

相对一致。

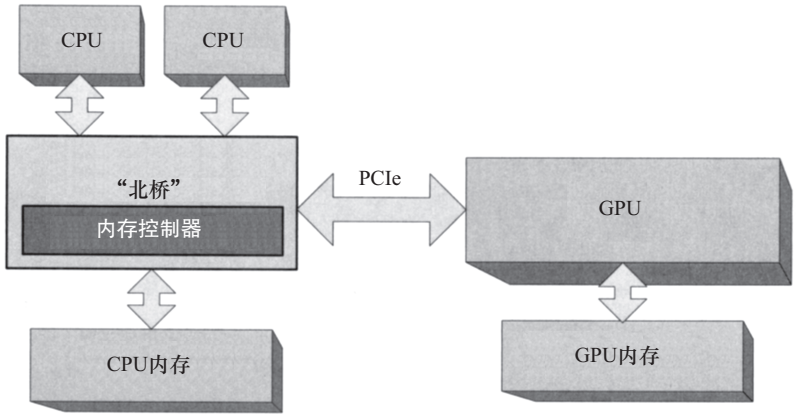


图 2-3 多 CPU（SMP 配置下）

2.1.3 非一致内存访问（NUMA）

从 AMD 的 Opteron 处理器和 Intel 的 Nehalem（酷睿 i7）处理器谈起，北桥的内存控制器直接集成到 CPU 中，如图 2-4 所示。这种结构的变化，提高了 CPU 的内存性能。

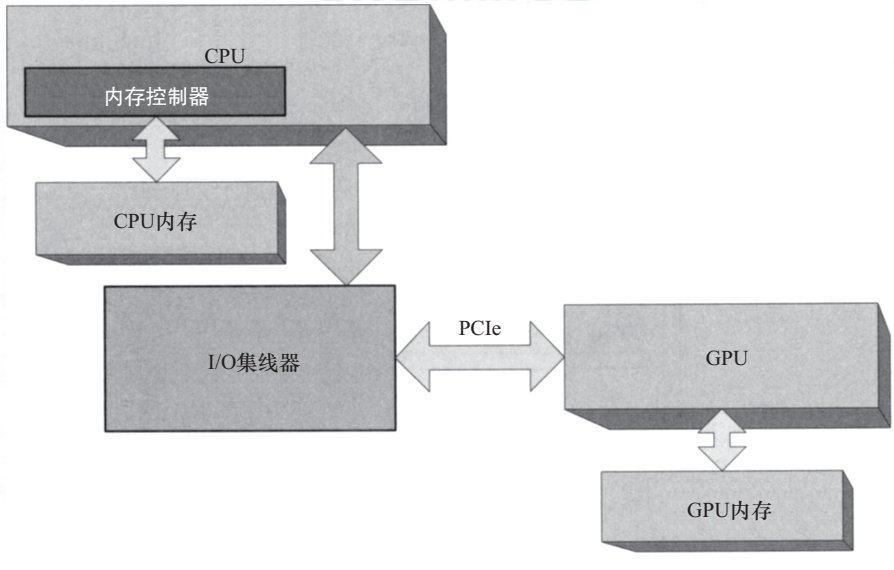


图 2-4 带有集成内存控制器的 CPU

对于开发人员，图 2-4 中的系统和我们讨论过的只是略有不同。而对于包含多个 CPU 的系统而言，如图 2-5 所示，那就变得很不一样了。

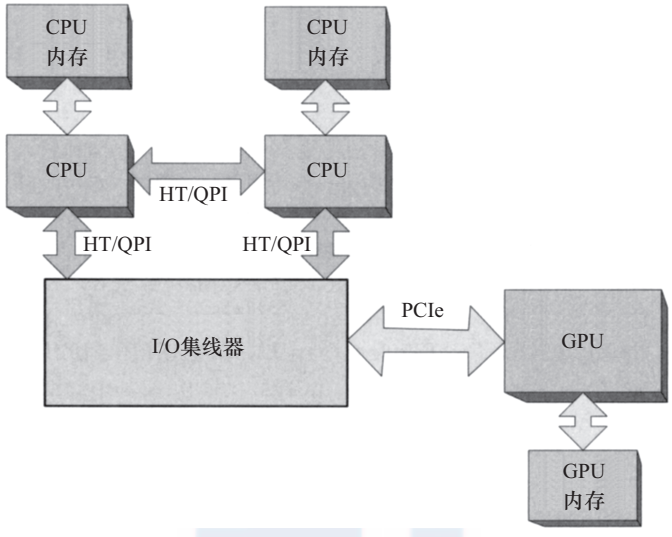


图 2-5　多 GPU（NUMA）

对于配置有多个 CPU 的机器[㊀]，这种架构意味着，每个 CPU 都有自己的内存带宽池。与此同时，由于多线程操作系统和应用程序依赖于前文中 CPU 和北桥配置下缓存的一致性，Opteron 和 Nehalem 架构分别推出了超传输（HT）和快速通道互联（Quick Path Interconnect，QPI）。

HT 和 QPI 是 CPU 之间、CPU 与 I/O 集线器的点对点连接。在采用 HT/QPI 的系统中，每个 CPU 都可以访问任何内存存储单元。但是对内存的物理地址直接附属于 CPU 的本地内存单元的访问会更快。“非本地访问”的执行有赖于 HT/QPI 检查其他 CPU 的缓存，清除所请求数据的缓存副本，然后传递数据到发出请求的 CPU。通常，这些 CPU 的大型片上缓存降低了非本地内存访问的成本。而且，发出请求的 CPU 可以在自身缓存层次上保留该数据，直到另一个 CPU 向内存提出申请为止。

为了帮助开发者解决这些性能陷阱，Windows 和 Linux 系统引入 API。这样应用程序能够使用特定 CPU 以及设置“线程亲和的”（thread affinity）CPU。因此，当操作系统把线程调度到 CPU 时，可以保证大部分甚至是全部的内存都是本地访问的。

一个爱钻研的程序员可以使用这些 API 来精心设计代码以暴露 NUMA 的性能缺陷。但更常见（和隐性）的问题是，由于运行在不同 CPU 上的两个线程的“伪共享”，导致过多的 HT/QPI 事务在同一缓存行上访问内存存储单元。因此，NUMA API 必须慎用。它们虽然给程序员提供了提高性能的工具，但也容易因开发者误用而带来性能问题。

㊀ 在这样的系统中，CPU 也可以被称为“节点”或“CPU 插槽”。

减轻访问非本地内存对性能影响的方法之一，是以缓存行边界为准，在 CPU 间均匀划分物理内存，也就是所谓的内存重叠[⊖]。对于 CUDA 而言，这种方法在如图 2-5 所示的系统中非常有效（其为一个 NUMA 模式配置，多 CPU 和 GPU 通过一个共享的 I/O 集线器连接）。由于 PCIe 带宽往往是整体应用性能的瓶颈，许多系统将使用独立的 I/O 集线器去服务更多的 PCIe 总线，如图 2-6 所示。

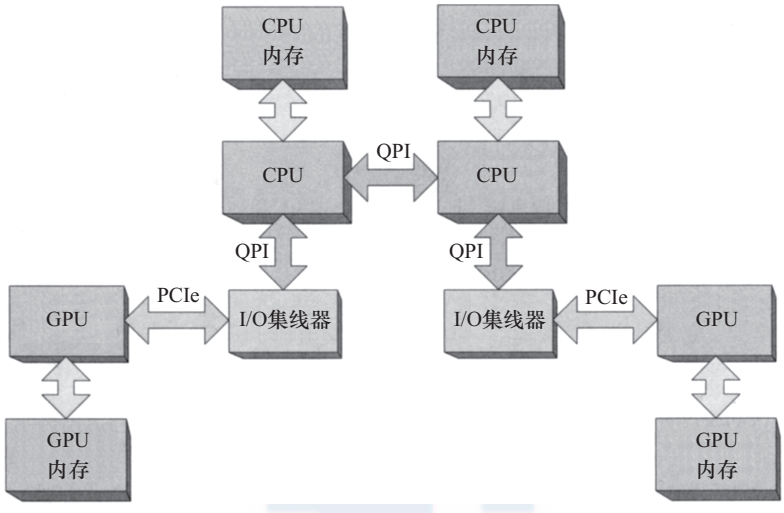


图 2-6 带多个总线的多 CPU（NUMA 配置）

为了很好地运行在这样一个“亲和”系统上，CUDA 应用程序必须注意 NUMA API 的使用，以便为连接到给定 GPU 的 PCIe 总线执行本地的内存分配和线程亲和的调度。否则，由 GPU 发起的内存复制将是非本地的，内存事务将在 HT/QPI 互连结构中需要额外的“跳跃”。由于 GPU 需要巨大的带宽，这些 DMA 操作会降低 HT/QPI 为其主要对象服务的能力。和伪共享相比，对于 CUDA 应用程序而言，GPU 的非本地内存复制操作对性能的影响有可能更为要命。

2.1.4 集成的 PCIe

如图 2-7 所示，通过将 I/O 集线器集成到 CPU 中，英特尔的沙桥（Sandy Bridge）系列处理器是迈向全面的系统集成的又一步。单个沙桥 CPU 的 PCIe 接口有多达 40 个通道（请记住，一个 GPU 最多可使用 16 个通道，所以 40 个通道足够支持两个完整的 GPU）。

对于 CUDA 开发者，集成的 PCIe 有利有弊。弊端是，PCIe 通路始终是亲和的。设计师不能建立单一 I/O 集线器服务于多个 CPU 的系统（即图 2-5 所示的系统）；所有的多 CPU 系统都类似于图 2-6。其导致的结果是，不同 CPU 上的 GPU 之间无法执行点对点操作。而优

⊖ 也有相反的说法，说这使得所有的内存访问“同样糟糕”。

点是，CPU 缓存可以直接参与 PCIe 总线通信：DMA 读请求可以直接读取缓存，并且 GPU 写入的数据会放入缓存中。

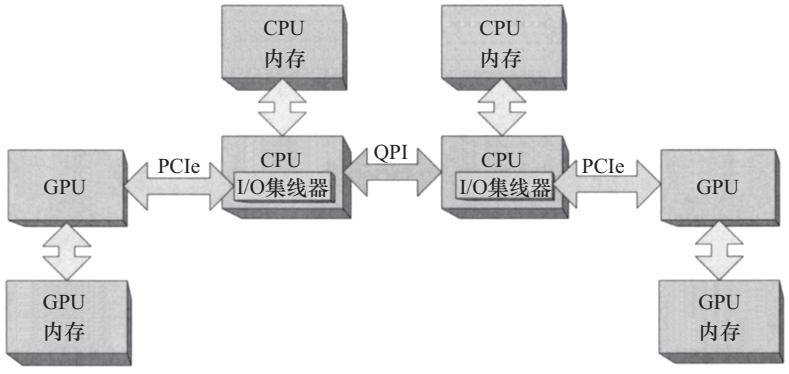


图 2-7　带集成 PCIe 的多 CPU

2.2　集成 GPU

在这里，术语“集成”的意思是“集成到芯片组”。如图 2-8 所示，先前只属于 CPU 的内存池现在可以被集成到芯片组的 CPU 和 GPU 所共享。例如，英伟达芯片组中支持 CUDA 的 GPU，包括 MCP79（适用于笔记本电脑和上网本）和 MCP89 等。MCP89 将是制造的最新、最好的可支持 CUDA 的 x86 芯片组，除了集成了三级缓存，它拥有 3 倍数量于 MCP7x 芯片组的流处理器簇。

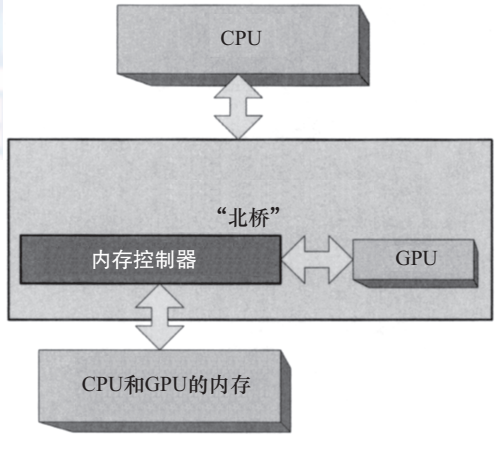


图 2-8　集成 GPU

CUDA 中用于映射锁页内存（mapped pinned memory）的 API，在集成 GPU 上具有特殊的意义。这些 API 把分配的主机内存映射到 CUDA 内核的地址空间，使它们能够直接被访问。这也被称为“零复制”，因为内存是共享的，复制操作不需要通过总线。事实上，在传输受限型工作量上，一个集成的 GPU 可以超过一个更大的独立 GPU。

“写结合”（write-combined，WC）内存的分配在集成 GPU 上也具有重要意义。CPU 的缓存侦测行为在访问这种内存时是被禁止的，这样可以提高访问内存过程中 GPU 的性能。当然，如果 CPU 从“写结合”内存中读取数据，通常的 WC 内存应用的性能会有损失。

集成 GPU 与独立 GPU 不是相互排斥的。MCP7x 和 MCP89 芯片组提供了 PCIe 连接接

口（图 2-9）。在这样的系统中，CUDA 更倾向于在独立 GPU 上运行，因为大多数的 CUDA 应用程序将在独立 GPU 上部署。例如，在单个 GPU 上运行时，一个 CUDA 应用程序会自动选择在独立 GPU 上运行。

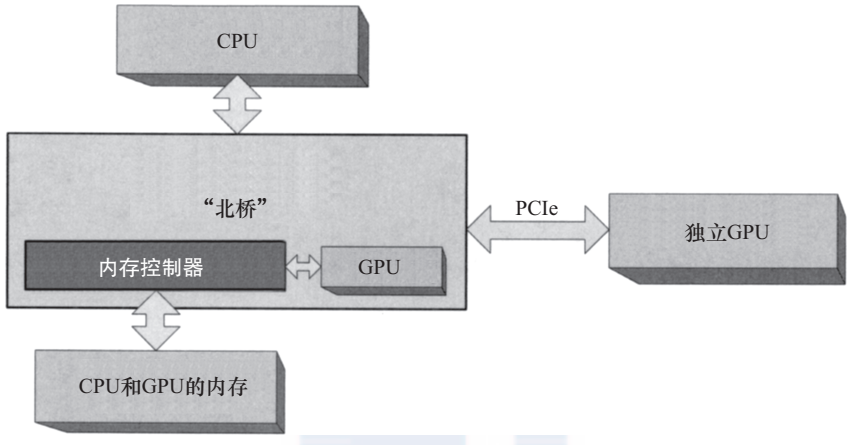


图 2-9 带独立 GPU 的集成 GPU

CUDA 应用程序可以通过检查 cudaDeviceProp.integrated 的值或者通过传递 CU_DEVICE_ATTRIBUTE_INTEGRATED 到 cuDeviceGetAttribute() 函数来查询一个 GPU 是否是集成 GPU。

对于 CUDA 而言，使用集成 GPU 并不稀奇，上百万的计算机在主板上集成了支持 CUDA 的 GPU。但它们会引起大家的好奇心。并且在短短几年内，它们将会变得落伍，因为英伟达已经退出了 x86 芯片组市场。据传言，英伟达已经宣布他们的出货重点放在集成了支持 CUDA 的 GPU 与 ARM CPU 的片上系统（systems on a chip，SOC）上。可以判定，零复制优化将在那些系统上工作得很好。

2.3 多 GPU

本节将探讨把多 GPU 安装在一个系统的不同方法及其对 CUDA 开发者的影响。为了讨论方便，我们会从下面的图中略去 GPU 内存。图中，每个 GPU 默认连接到相应专用内存。

大约在 2004 年，英伟达推出了“速力”（Scalable Link Interface，SLI）技术，可以让多 GPU 并行工作，以提供更高的图形性能。使用可以容纳多个 GPU 的主板，用户可以通过在他们的系统上安装两个 GPU 来提升近一倍的图形性能（图 2-10）。默认情况下，英伟达驱动程序配置了这些 GPU 卡，使它们表现得好像是单个速度更快的卡，用以加速如 Direct3D 和 OpenGL GPU 的图形 API。打算使用 CUDA 的最终用户，必须明确在 Windows 的显示控制面板上启用它。

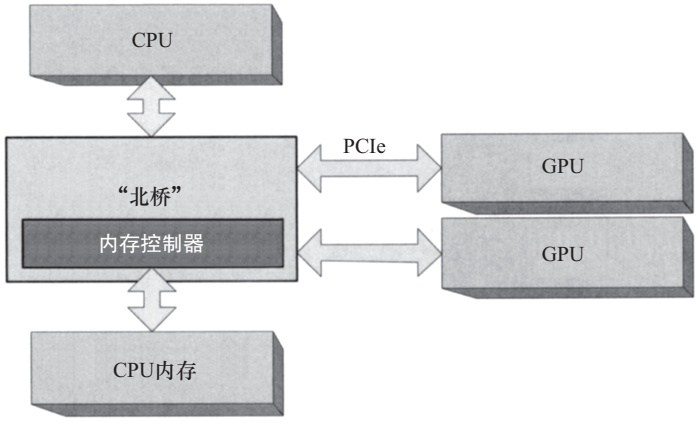


图 2-10　多插槽 GPU

还可以构造有多个 GPU 的 GPU 板（图 2-11）。这种板的例子有 GeForce 9800GX2（双 G92）、GeForce GTX295（双 GT200）、GeForce GTX590（双 GF110）和 GeForce GTX690（双 GK104）。这类 GPU 板上的 GPU 之间仅有桥接芯片是共享的，该芯片使一对 GPU 芯片可以通过 PCIe 交换信息。但它们并不共享内存资源，每个 GPU 都拥有一个集成内存控制器，使连接到 GPU 的内存具有全带宽性能。板上的 GPU 可以通过点对点的内存复制来交换信息，只需使用桥接芯片成功绕过主要的 PCIe 通道。此外，如果它们是费米架构或更高级别的 GPU，那么每个 GPU 都可以映射属于其他 GPU 的内存到其全局地址空间里。

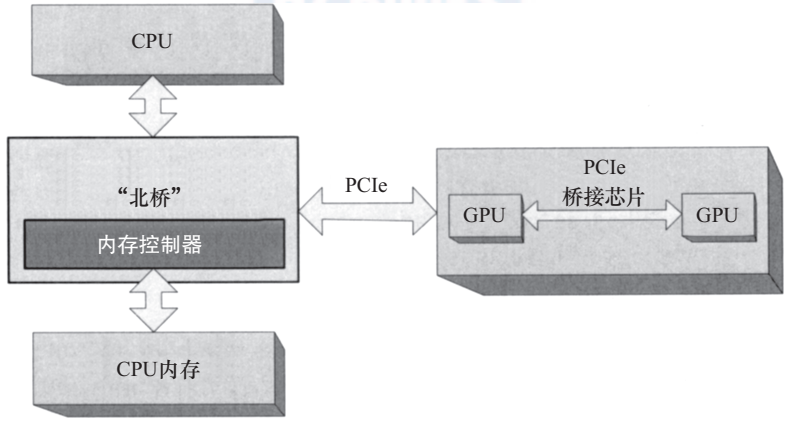


图 2-11　多 GPU 板

SLI 可以使用户能够像使用单个 GPU 一样使用多个 GPU，只不过性能更快（通常多个 GPU 在同一块电路板上，如图 2-11 所示）。当图形应用程序下载纹理或者其他数据时，英伟达的显卡驱动程序会将数据和大部分渲染命令以广播的方式传送到每个 GPU。而渲染命令可能会有一些小的变化，以使每个 GPU 控制的那部分输出缓冲区得以渲染。由于 SLI 导致多

GPU 表现为单个 GPU，并且 CUDA 应用程序不能像图形应用程序那样透明地得到加速，所以 CUDA 开发人员一般不使用 SLI。

这种双 GPU 板设计超支了供 GPU 使用的 PCIe 带宽。由于只一个 PCIe 插槽对应的带宽供板上的一对 GPU 使用，传输受限型工作任务的性能将遭受负面影响。如果有多个 PCIe 插槽可供选择，最终用户就可以安装多个双 GPU 板。如图 2-12 就显示了装载了 4 个 GPU 的机器。

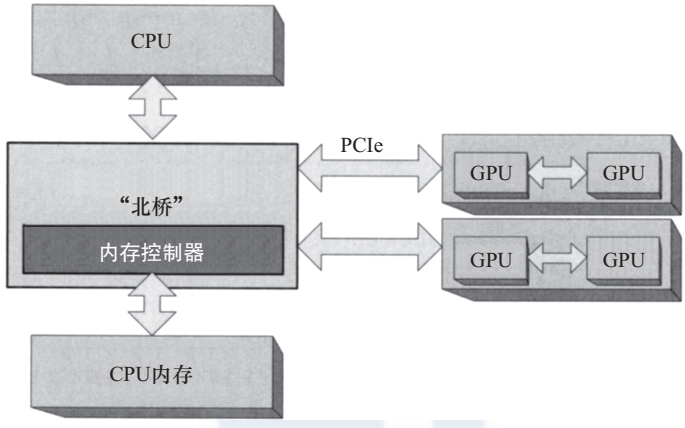


图 2-12 多插槽式多 GPU 板

如果有多个如图 2-6 中系统的 PCIe I/O 集线器，NUMA 系统考虑的布局和线程亲和将适用于插入该配置的单 GPU 板。

如果芯片组、主板、操作系统和驱动程序支持，甚至可以把更多的 GPU 装载进系统中。安特卫普大学（University of Antwerp）的研究人员，通过将 4 个 GeForce 9800GX2 整合到一个单一桌面电脑里，构建了一个叫做 FASTRA 的 8-GPU 系统，引起一时轰动。一个基于双 PCIe 芯片组构建的类似系统看起来如图 2-13 所示。

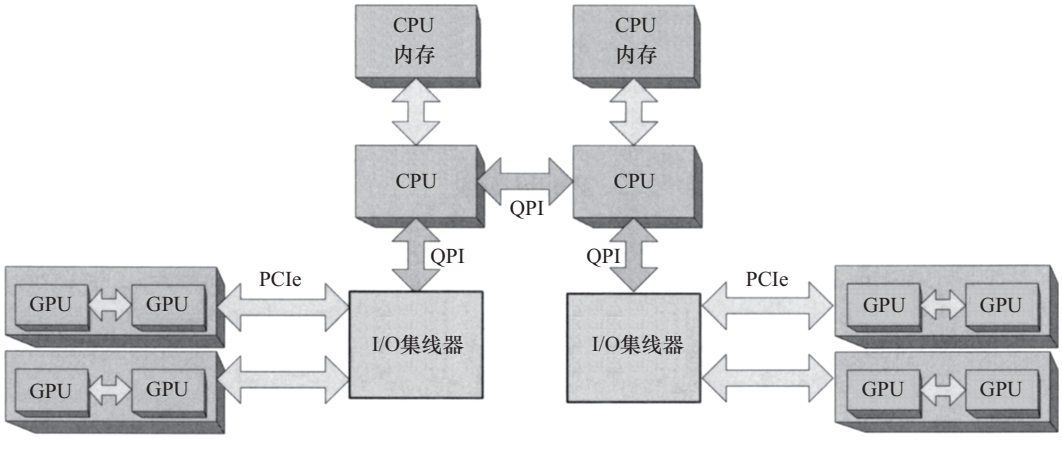


图 2-13 多 GPU 板与多 I/O 集线器连接图

作为边注，点对点内存访问（其他 GPU 设备内存的映射，而不是内存复制）不能跨越 I/O 集线器工作，也不适用于集成了 PCIe 插槽的沙桥 CPU。

2.4　CUDA 中的地址空间

每个 CUDA 初学者都知道，CPU 和 GPU 的地址空间是分开的。CPU 不能读取或写入 GPU 的设备内存，反过来，GPU 也无法读取或写入 CPU 的内存。其结果是，应用程序必须明确地从 GPU 的内存中传入 / 传出数据，以对其进行相应处理。

实际情况比较复杂，CUDA 已经增加了许多新的功能，如映射锁页内存和点对点访问。本节将以详细说明地址空间在 CUDA 中如何工作为第一要义。

2.4.1　虚拟寻址简史

虚拟地址空间是如此普遍和成功的抽象物，大多数程序员每天都使用它们，并从中受益，却不知道它们的存在。早期的人们发现，给计算机中的内存位置分配连续编号是非常有用的。虚拟地址空间是这一发现的延伸。其标准度量单位是字节，例如，64KB 内存中的计算机的内存位置由 0 到 65535。指定内存位置的 16 位值称为地址，地址的计算和相应内存位置上的操作过程则统称为寻址（addressing）。

早期的计算机进行物理寻址。它们会计算内存位置，然后读取或写入相应存储单元，如图 2-14 所示。随着软件越来越复杂，以及计算机能承载多个用户或越来越普遍地运行多个作业，每个程序却可以读取或写入任何物理内存位置。很明显，这是不会被接受的！否则，机器上运行的软件很有可能会因为写入错误的内存位置而对其他软件造成致命破坏。除了健壮性问题，其还有安全方面的瑕疵：软件可通过读取属于其他人的内存位置来窥探其他软件。

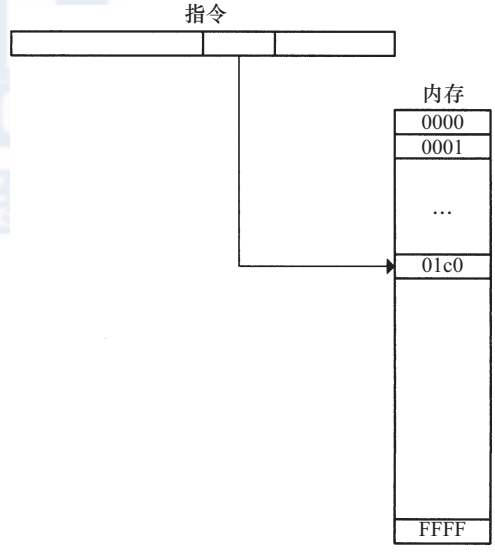


图 2-14　16 位地址空间简图

因此，现代的计算机推行虚拟地址空间。每个程序（操作系统设计者称其为一个进程）都会得到类似于图 2-14 中的内存视图，但每个进程都有自己的地址空间。未经操作系统特别许可，它们不能读取或写入属于其他进程的内存。机器指令使用虚拟地址而不是物理地址，随后由操作系统执行一系列查找表的操作，将其翻译成一个物理地址。

在大多数系统中，虚拟地址空间被划分成许多页，它们是寻址的单位，其大小至少 4096

字节。通常，硬件查找指定页内存的页表项（PTE）来得到所在物理地址，而不直接引用物理内存的地址。

我们可以很清楚地从图 2-15 看出，虚拟寻址能使一个连续的虚拟地址空间映射到物理内存并不连续的一些页。此外，当应用程序试图读取或写入页面未被映射到物理内存的内存位置时，硬件会发送必须由操作系统处理的错误信号。

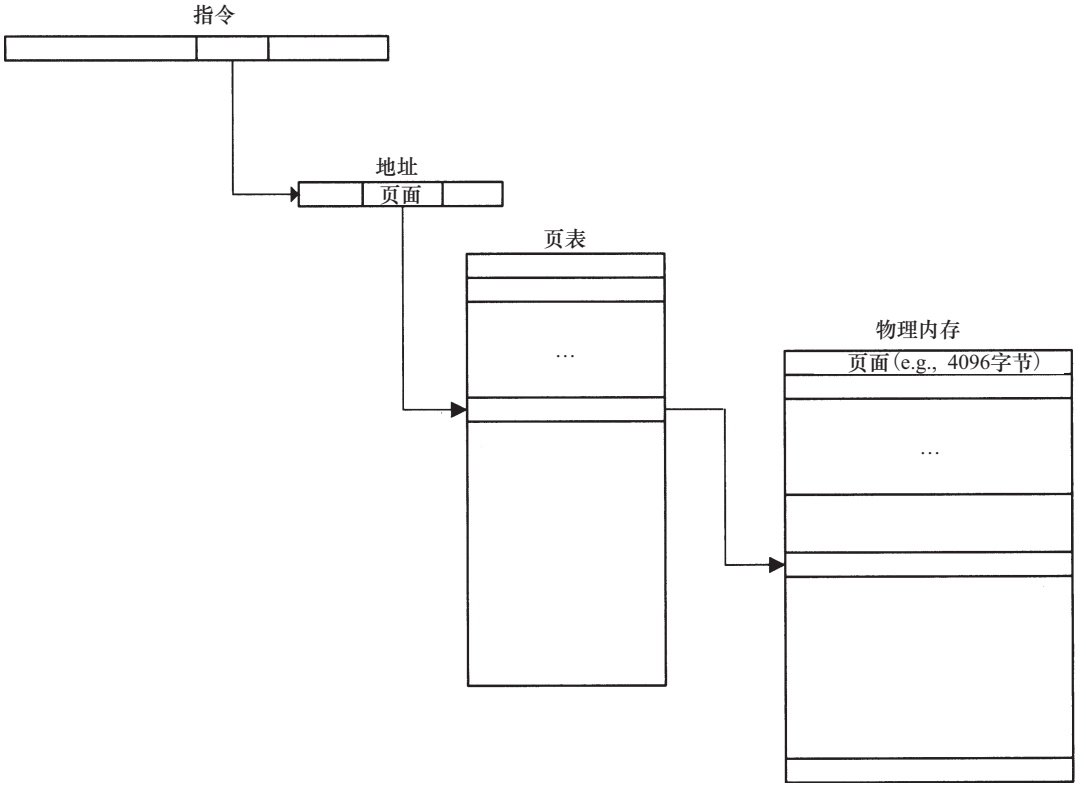


图 2-15 虚拟地址空间

另外做个边注：实际上，所有硬件都不会实现如图 2-15 所示的单级页表。最起码，地址会被分成至少两个索引：第一个索引指向页表的“页目录”，第二个索引代表第一个索引约束下选择的页表。这种分层设计减少了页表所需的内存量，把不活跃的页表标记为非常驻状态并交换到磁盘上，就像无效页面一样。

除了物理内存位置，PTE 还包含了在做地址转换时硬件可以验证的权限位。例如，操作系统可以使内存页只读。此时，如果应用程序试图写入该页面，硬件将会产生一个错误信号。

操作系统使用虚拟内存硬件来实现很多功能：

- 缓式分配（lazy allocation）：通过设置 PTE 允许分配无物理内存支持的页，从而可以分配大容量的内存。如果请求内存的应用程序碰巧访问这些页面，操作系统会立刻找

到一个有物理内存的页面并解决这个故障。

- 请求式调页（demand paging）：内存可以被复制到磁盘中并且页面被标记为“非常驻”。如果这样的内存再次被引用，硬件会产生“页面故障”信号，而且操作系统会将数据复制到一个物理页中并修正 PTE 指向该页，然后继续执行。如此，故障得以解决。
- 写时复制（copy-on-write）：通过创建第二组映射到相同的物理页面的 PTE，并将两组 PTE 标记为只读，虚拟内存得以“复制”。如果硬件捕获到一个试图写入这些页面的操作，操作系统将会复制该组页面到另一组物理页面，并再次标志这两组 PTE 为可写，然后恢复执行。如果应用程序只写入一个很小比例的“复制页面”，写时复制也就在性能上具有明显优势。
- 映射文件 I/O（mapped file I/O）：文件可以被映射到地址空间，并且通过访问文件可以解决页面故障。对进行随机访问相关文件的应用程序，通过委托操作系统中高度优化的 VMM 代码进行内存管理是非常有用的，特别是因为它是紧密耦合的大容量存储驱动程序。

CPU 所执行的每一次内存访问都会有地址转换，明白这点是很重要的。为了使这个操作快捷，CPU 包含了一些特殊的硬件：被称为转换旁置缓冲器（translation lookaside buffer，TLB）的缓存以及“页面查询器”（page walker）。前者的作用是保留最近转换的地址区间；后者的作用是通过读取页表来处理在 TLB 中未命中的缓存。[㊀] 现代的 CPU 还包括了支持“统一的地址空间”的硬件，可以让多 CPU 通过 AMD 的 HT（超传输）和英特尔的快速通道互连（QPI），有效地访问另一个内存。由于这些硬件设施可以使 CPU 使用统一的地址空间来访问系统中的任何内存位置，所以本节前文在提到“CPU”和“CPU 地址空间”时未提及系统中 CPU 的数量。

附注：内核模式和用户模式

关于 CPU 的内存管理的最后一点是操作系统的代码必须使用内存保护，以防止应用程序破坏操作系统自身的数据结构（例如，控制地址转换的页表）。为了协助内存保护，操作系统拥有一个执行任务的“特权”模式，可以在实现关键的系统功能时使用。为了管理页表等低层次硬件资源，抑或是对磁盘、网络控制器、CUDA GPU 等外设上的硬件寄存器进行编程，CPU 必须在内核模式下运行。应用程序代码所使用的非特权执行模式称为用户模式。[㊁] 除了操作系统提供商编写的代码，控制硬件外设的低级别驱动代码也在内核模式下运行。由于内核模式代码的错误可能会导致系统稳定性或安全性问题，内核模式代码会秉承一个更高的质量标准。此外，许多操作系统服务，如映射文件 I/O 或上面列出的设施外的其他内存管理，都不能在内核模式中使用。

为了保证系统的稳定性和安全性，用户模式和内核模式之间的接口是经过精心设计的。

㊀ 编写程序（对于 CPU 和 CUDA）来显示 TLB 的规模和结构以及页面查询器的内存开销，这也是有可能的（如果页面查询器在足够短的时间内通过足够的内存）。

㊁ 内核模式和用户模式分别对应 x86 下的“Ring 0”和“Ring 3”术语。

用户模式代码必须在内存中设立一个数据结构，并执行一个特殊的系统调用来验证内存和发出的请求的有效性。这种从用户模式到内核模式的过渡称为一个内核转换。内核转换代价不小，会增加 CUDA 开发者的成本。

每次用户模式驱动程序下实现的 CUDA 硬件交互都要由内核模式代码进行仲裁。这通常意味着它代其分配内存资源，以及硬件寄存器等硬件资源。例如，用户模式驱动程序下的硬件寄存器可以把任务提交给硬件。

大部分 CUDA 的驱动程序是在用户模式下运行。例如，为了分配系统锁页内存（例如，使用 cudaHostAlloc() 函数），CUDA 的应用程序会调用用户模式下的 CUDA 驱动程序，然后组成一个内核模式下的 CUDA 驱动程序请求，并执行内核转换。内核模式下的 CUDA 驱动程序，混合使用低级别的操作系统服务（例如，它可能调用系统服务来映射 GPU 硬件寄存器）和具有硬件特性的代码（例如，对 GPU 内存管理硬件进行编程）来满足上述请求。

2.4.2 不相交的地址空间

虽然 GPU 硬件不如 CPU 支持那么丰富的功能集，CUDA 也使用虚拟地址空间。GPU 确实执行内存保护，所以 CUDA 程序不可以随机地读取或破坏其他 CUDA 程序的内存，也不可以访问还没有被内核模式驱动程序映射的内存。但是，GPU 并不支持请求式调页，所以被 CUDA 分配的每一字节虚拟内存都必须对应一个字节的物理内存。此外，请求式调页功能是操作系统为实现前述特性而使用的基础硬件机制。

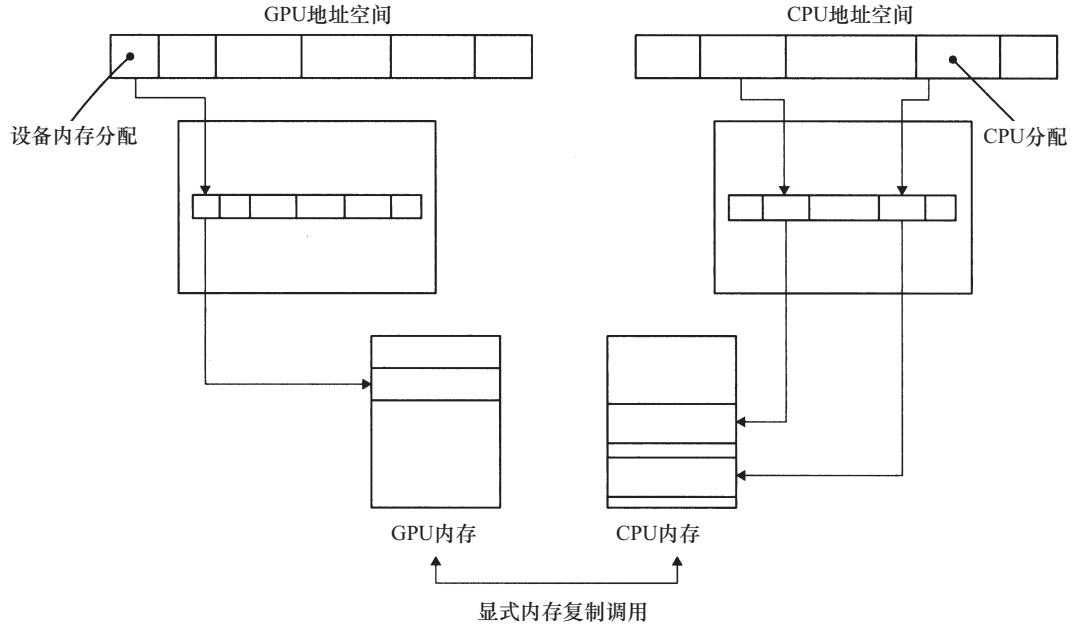


图 2-16 不相交的地址空间

由于每个 GPU 有它自己的内存和地址转换硬件，GPU 的地址空间和 CUDA 应用程序中的 CPU 地址空间是相互分开的。图 2-16 显示的是在使用映射锁页内存之前 CUDA 1.0 版的地址空间架构。CPU 和 GPU 都有自己的地址空间，用来映射各自设备自身的页表。两者的设备都要通过显式的内存复制命令来交换数据。GPU 可以分配锁页内存，该内存是 GPU 为 DMA 映射的页面锁定的内存。它只能使 DMA 的速度更快，却不能让 CUDA 内核程序访问主机内存。㊀

CUDA 驱动程序跟踪锁页内存的起止范围，并自动加速引用它们的内存复制操作。异步内存复制调用需要锁页内存起止信息，以确保在内存复制完成之前操作系统不会取消映射或移动物理内存。

并非所有的 CUDA 应用程序都可以分配到让 CUDA 使用的主机内存。例如，一个大型应用程序中的 CUDA 插件可能会在非 CUDA 代码分配的主机内存上操作。为了适应这种情况，CUDA 4.0 增加了注册现有的主机内存地址起止范围（它可以锁定一个虚拟地址区间，并将其映射到 GPU）以及跟踪数据结构的地址范围的能力，以便于让 CUDA 明白它是锁页内存。然后，内存就可以被传递给异步内存复制调用，否则，它将被看作是由 CUDA 分配的。

2.4.3 映射锁页内存

如图 2-17 所示，CUDA 2.2 增加了一个叫做映射锁页内存的特性。映射锁页内存是被映射到 CUDA 地址空间的锁页主存，在 CUDA 内核程序里可以直接对其读取或写入。CPU 和 GPU 的页表更新了，以便 CPU 和 GPU 中拥有指向相同主机内存缓冲区的地址区间。由于地址空间不同，GPU 指向该缓冲区的指针必须使用 cuMemHostGetDevicePointre() 或 cudaHostGetDevicePointer() 函数来查询。㊁

2.4.4 可分享锁页内存

如图 2-18 所示，CUDA 2.2 还增加了一个叫做可分享锁页内存的特性。设置锁页内存“可分享”，会导致 CUDA 驱动程序把该内存映射给系统中的所有 GPU，而不仅仅是当前上下文的 GPU。此特性为系统中的 CPU 和所有 GPU 创建一组独立的页表条目，使相应的设备将虚拟地址转换成实际物理内存。主机内存的起止范围也被添加到了每个活跃的 CUDA 上下文的跟踪机制中，因此每个 GPU 都能识别出作为可分享方式分配的锁页内存。

图 2-18 可能表示了开发者对多地址空间的容忍限度。在这里的每个分配，双 GPU 系统有 3 个地址，而 4GPU 系统则有 5 个地址。虽然 CUDA 通过 API 能够快速查找一个给定 CPU 的地址空间范围，并传回相应的 GPU 地址空间范围，但在 *N* 个 GPU 的系统中将拥有 *N*+1 个地址且全部指向同一物理分配，这样是不方便的。

㊀ 在 32 位操作系统上，为了执行内存复制，支持 CUDA 的 GPU 可以映射锁页内存到一个 40 位地址空间里，而该空间已经超过了 CUDA 内核程序的控制范围。

㊁ 对于多 GPU 配置，CUDA 2.2 还增加了一个叫做“可分享锁页内存”的特性，导致上述分配会被映射到每个 GPU 的地址空间。但这并不能保证 cu(da)HostGetDevicePointer() 函数在不同 GPU 上会返回相同的值。

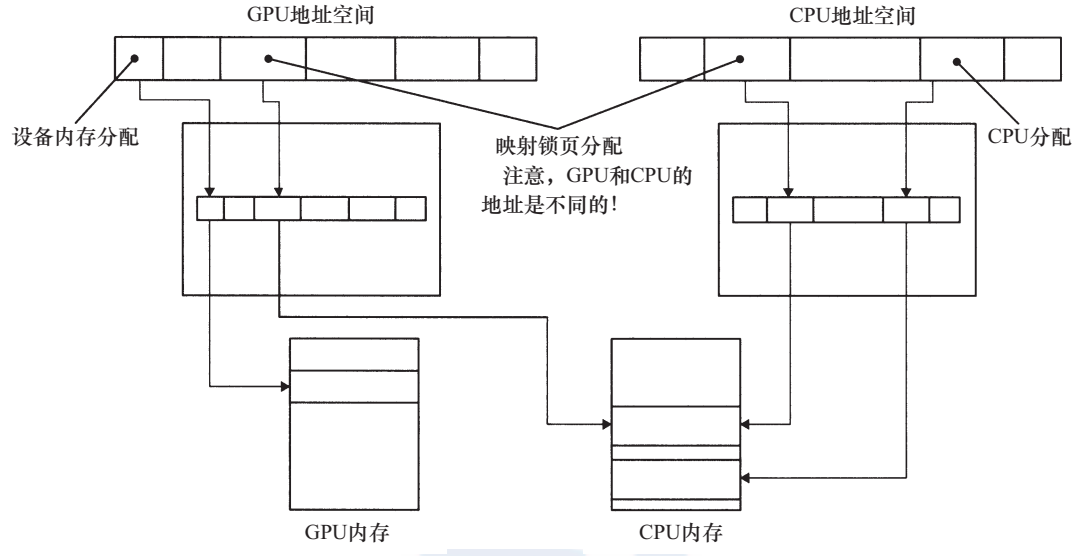


图 2-17 映射锁页内存

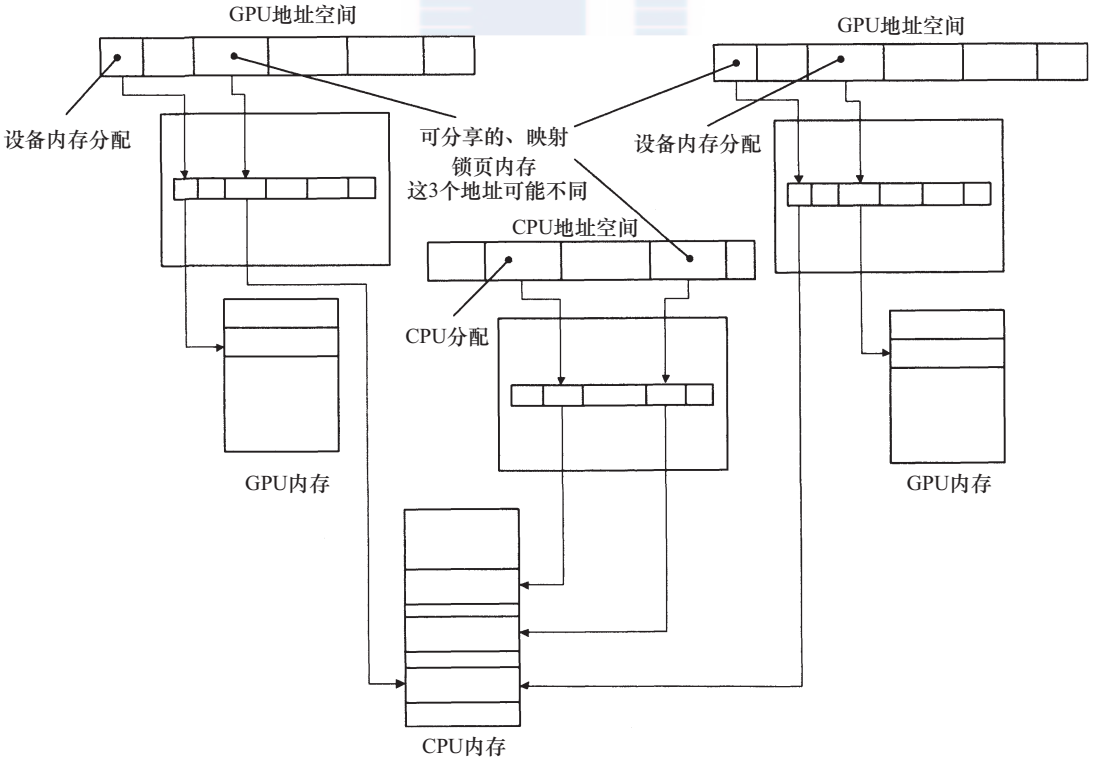


图 2-18 可分享、映射锁页内存

2.4.5　统一寻址

32 位 CUDA GPU 需要多个地址空间，它最多只能映射 2^{32}=4GB 的地址空间。由于一些高端的 GPU 有多达 4GB 的设备内存，所以它们很难在寻找所有设备内存地址的同时映射锁页内存，更不用说像 CPU 那样使用统一的地址空间了。但是，对于在费米架构或更高级 GPU 下的 64 位操作系统而言，如下的一个更简单抽象是可行的。

如图 2-19 所示，CUDA 4.0 增加了一个叫做统一虚拟寻址（unified virtual addressing，UVA）的特性。当 UVA 生效时，CUDA 会从相同的虚拟地址空间为 CPU 和 GPU 分配内存。CUDA 驱动程序通过以下两步来完成上述任务：第一，初始化程序执行基于 CPU 地址空间的大型虚拟分配，该分配过程中可能会碰到无物理内存支持的情况；第二，将 GPU 分配的内存映射到上述地址空间。由于 64 位 CPU 支持 48 位虚拟地址空间[㊀]，而 CUDA GPU 只支持 40 位，应用程序使用 UVA 时应确保 CUDA 被提前初始化，以保证 CUDA 所需要的虚拟地址先于 CPU 代码中的分配请求而被满足。

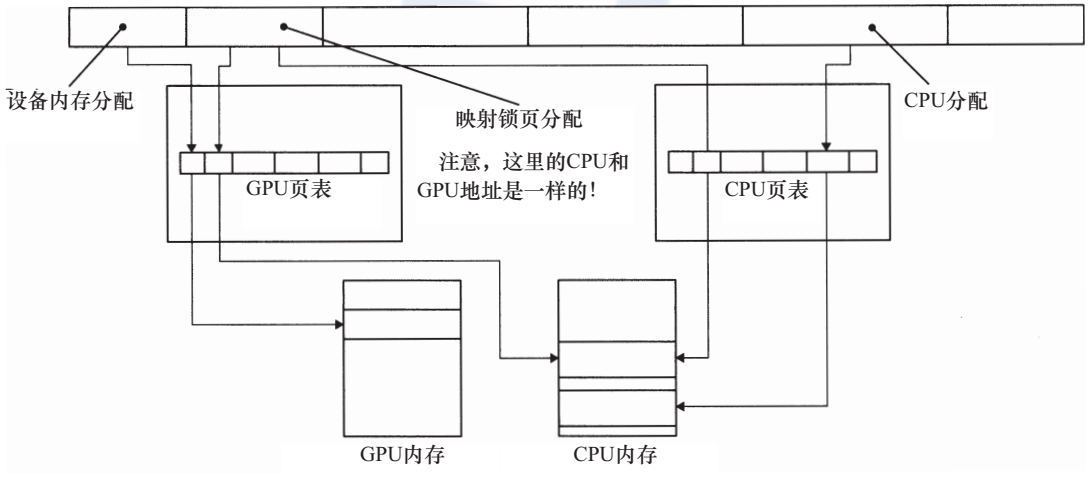


图 2-19　统一虚拟寻址（UVA）

对于映射锁页分配，GPU 和 CPU 的指针是相同的。对于其他类型的分配，CUDA 可以通过地址推断出既定分配存在于哪个设备。其结果是，线性内存复制函数族（需要指定方向的 cudaMemcpy() 和 cuMemcpyHtoD()、cuMemcpyDtoH() 等）被更简单的 cuMemcpy() 和 cudaMemcpy() 函数所替代，而后两个函数并不需要指定内存方向。

支持 UVA 的系统会自动启用 UVA。在本书撰写过程中，使用 TCC 驱动程序的 64 位 Linux、64 位 MacOS 和 64 位 Windows 都支持 UVA。然而，WDDM 驱动程序暂时还不支持 UVA。当 UVA 生效时，CUDA 所执行的所有锁页分配都是被映射的和可分享的。对已被使用 cuMemRegisterHost() 函数分配为锁页的系统内存，需要注意设备指针仍然需要使用 cu(da)

㊀ 48 位虚拟地址空间 =256TB。未来的 64 位 CPU 将支持更大的地址空间。

HostGetDevicePointer() 函数查询。即使在 UVA 生效时，CPU 也不能访问设备内存。另外，默认情况下，GPU 不能互相访问彼此的内存。

2.4.6 点对点映射

在介绍 CUDA 的虚拟内存抽象机制旅程的最后阶段，我们讨论一下点对点的设备内存映射如图 2-20 所示。点对点可以使费米架构 GPU 读写另一个费米架构 GPU 的内存。点对点映射仅支持启用 UVA 的平台，并且只对连接到相同 I/O 集线器上的 GPU 有效。由于使用点对点映射时 UVA 始终是有效的，不同设备的地址空间范围不重叠，并且驱动程序（和运行时）可以从指针值推断出所驻留的设备。

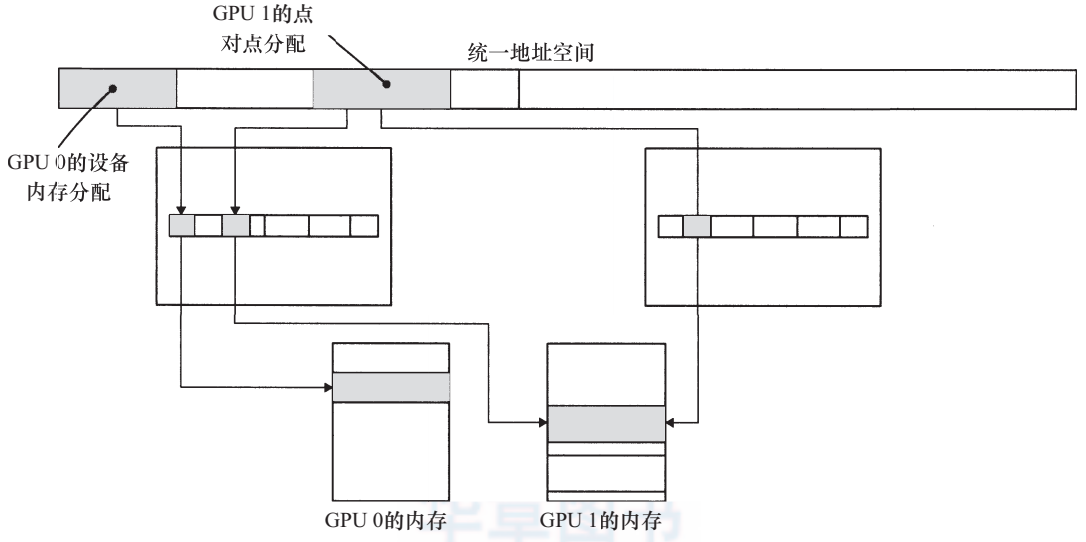


图 2-20 点对点映射

点对点内存寻址是非对称的。例如，图 2-20 为非对称的映射，在该图中，对于 1 号 GPU 的内存分配，0 号 GPU 是可见的，反之则不行。为了让 GPU 之间能够看到对方的内存，每个 GPU 必须显式地映射其他 GPU 的内存。而对于用来管理点对点映射的 API 的函数，我们将在 9.2 节进行相应讨论。

2.5 CPU/GPU 交互

本节描述 CPU/GPU 交互的关键知识点如下。

- 锁页主机内存：GPU 可以直接访问的 CPU 内存；
- 命令缓冲区：由 CUDA 驱动程序写入命令，GPU 从此缓冲区读取命令并控制其执行；
- CPU/GPU 同步：指的是 CPU 如何跟踪 GPU 的进度。

本节在硬件层次上介绍这些设施，仅为帮助读者了解它们如何适用到 CUDA 开发的需要时才使用 API。为了简单起见，本节使用图 2-1 中的 CPU/GPU 模型，省略了多 CPU 或者多 GPU 编程的复杂性。

2.5.1　锁页主机内存和命令缓冲区

出于某些显而易见的原因，CPU 和 GPU 最擅长访问它们自己的内存，但是 GPU 可以通过直接内存访问（direct memory access，DMA）方式来访问 CPU 中的锁页内存。锁页是操作系统常用的操作，可以使硬件外设直接访问 CPU 内存，从而避免过多的复制操作。“被锁定”的页面已被操作系统标记为不可被操作系统换出的，所以设备驱动程序给这些外设编程时，可以使用页面的物理地址直接访问内存。而 CPU 仍然可以访问上述锁页内存，但是此内存是不能移动或换页到磁盘上的。

由于 GPU 是不同于 CPU 的设备，DMA 还可以使 GPU 读取和写入 CPU 内存的操作与 CPU 的执行操作相互独立且并行执行。我们必须注意 CPU 和 GPU 之间的同步，以避免竞争。而对于可以在 GPU 活动时有效利用 CPU 时钟周期的应用程序而言，并发执行具有显著的性能优势。

图 2-21 描绘了一个为直接访问而被 GPU 映射的“锁页”缓冲区[⊖]。CUDA 程序员都很熟悉锁页缓冲区，因为 CUDA 一直允许他们通过 cudaMallocHost() 等 API 来分配锁页内存。在表面下的内在机制中，对应这种缓冲区的主要应用程序之一是将命令提交到 GPU。CPU 将命令写入一个供 GPU 消耗的“命令缓冲区”，与此同时，GPU 读取并执行先前写入的命令。图 2-22 显示的是 CPU 和 GPU 如何共享这个缓冲区。此图是被简化了的，因为这类命令可能是几百字节大小，而该缓冲区大到足够容纳几千个这样的命令。缓冲区的“前缘”正由 CPU 构建且尚未准备好供 GPU 读取，“后缘”正在被 GPU 读取。介于中间的命令全部准备就绪，只要 GPU 进程准备好了就可以进行传输。

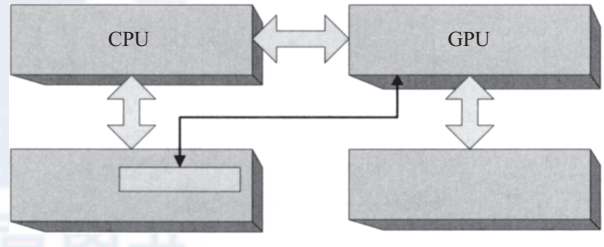


图 2-21　锁页缓冲区

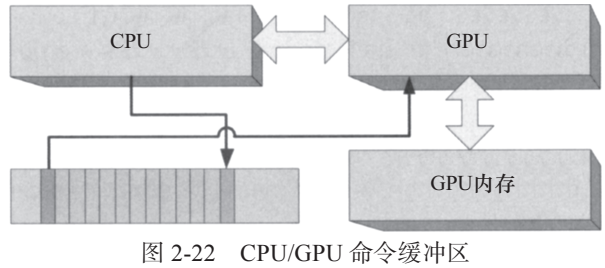


图 2-22　CPU/GPU 命令缓冲区

通常情况下，CUDA 驱动程序可以重用命令缓冲区的内存，因为一旦 GPU 处理完一条

⊖　重要提示：在此背景下，GPU 的“映射操作”涉及设置指向 CPU 内存的物理地址的硬件表。该内存也有可能无法被映射到能被 CUDA 内核程序访问的地址空间中。

命令后，该命令占用的内存可以被 CPU 重新写入。图 2-23 显示了 CPU 是如何“循环处理”命令缓冲区的。

考虑到启动一个 CUDA 内核程序需要几千个 CPU 时钟周期，此时，一个 CPU/GPU 并发的重要方法是在 GPU 进行处理时准备更多的 GPU 命令。应用程序并不能均衡地保持 CPU 和 GPU 忙碌，有可能会出现“CPU 受限”或“GPU 受限”的情况，分别如图 2-24 和图 2-25 所示。在一个 CPU 受限型应用程序里，GPU 随时就绪，只要下一个命令准备好就马上进行处理；而在一个 GPU 受限型应用程序里，CPU 的命令缓冲区已经完全被填满，在写入下一 GPU 命令之前必须等待 GPU 处理完上一命令。有些应用程序本质上是 CPU 或 GPU 受限型的，所以 CPU 和 GPU 受限并不一定说明应用程序的结构有什么本质的问题。然而，知道应用程序是 CPU 受限型的还是 GPU 受限型的有助于找出性能优化机会。

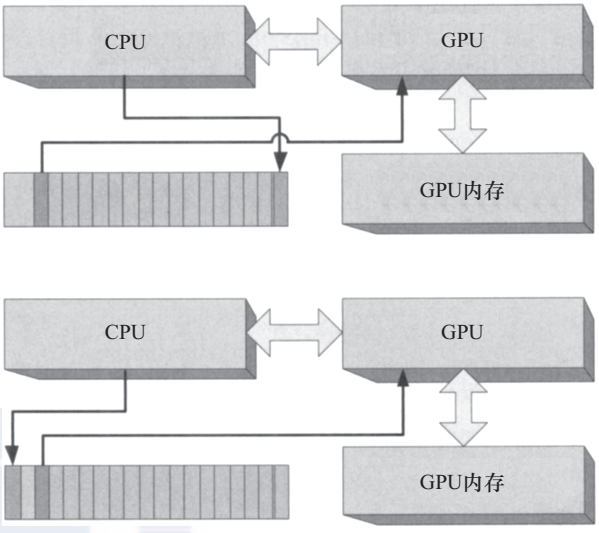


图 2-23 命令缓冲区的循环处理

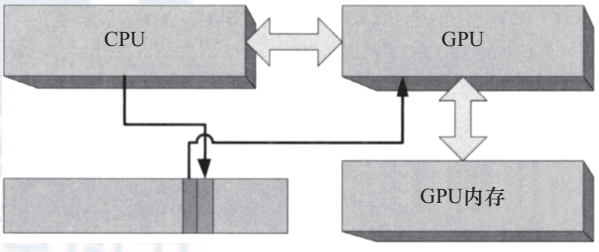


图 2-24 GPU 受限型应用程序

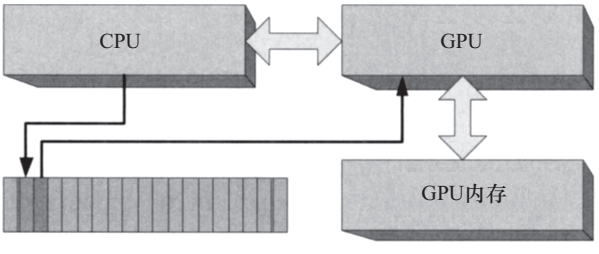


图 2-25 CPU 受限型应用程序

2.5.2 CPU/GPU 并发

前一节介绍在 CUDA 系统可用的最粗粒度并行：CPU/GPU 并发。CUDA 内核程序的所有启动都是异步的：CPU 通过将命令写入命令缓冲区来请求启动内核，然后直接返回，而不检查 GPU 进度。内存复制也可以选择异步方式，这使 CPU/GPU 并发以及可能使内存复制与内核处理并发执行。

1. 阿姆达尔法则（Amdahl's Law）

当 CUDA 程序编写正确时，CPU 和 GPU 可以完全地进行并行操作，这有可能使性能翻倍。然而 CPU 或 GPU 受限型程序不会从 CPU/GPU 并发得到太大好处，因为即使其他装

置也是并行运行的，CPU 或 GPU 的某一方也会限制整体性能。这种含糊不清的观察可以使用阿姆达尔法则正确描述，该定律于 1967 年在一篇论文中被吉恩 · 阿姆达尔（Gene Amdahl）首次阐明。[⊖]阿姆达尔法则通常如下表示：

$$加速比 = 1/(r_s + r_p/N)$$

其中，$r_s+r_p=1$ 且 r_s 代表的是串行部分比率。在研究 CPU/GPU 并发等小规模的性能场合时，这一公式形式似乎不太方便。所以，将其变形成如下公式：

$$加速比 = N/(N(1-r_p)+r_p)$$

这清楚地表明，如果 $r_p=1$，则加速为 N 倍。如果 CPU 和 GPU 各有一个（$N=2$），那来自全并发的最大加速为 2 倍。对平衡工作负载这基本是可实现的，例如视频转码中，CPU 可以与 GPU 同时执行，其中 GPU 执行并行操作（如像素处理），而 CPU 执行串行操作（如可变长解码）。但对于更多的 CPU 或 GPU 受限型应用程序，这种类型的并发只能提供有限的优势。

对那些认为并行是解决一切性能问题的万能药的人来说，阿姆达尔的论文无异于一个警钟。我们将其应用于这本书中讨论内部 GPU 并发、多 GPU 并发，以及移植 CUDA 内核所得加速比等其他地方。它可以用来让我们知道哪些并发形式不能赋予一个给定的应用程序任何优势，因此开发人员可以把时间花在探索其他提高性能的途径上。

2. 错误处理

CPU/GPU 并发也会对错误处理造成影响。当 CPU 开始启动一批内核而其中一个导致内存故障时，CPU 并不能发现这一故障，直到它执行了 CPU/GPU 同步才行（在下一节描述）。开发人员可以手动执行 CPU/GPU 同步作为辅助，具体通过调用 cudaThreadSynchronize()、cuCtxSynchronize() 等函数来完成。在诸如 cudaFree() 或 cuMemFree() 的函数调用中也会导致 CPU/GPU 的同步。《CUDA C 语言编程指南》中通过使用可能会导致 CPU/GPU 同步的函数，提及了这一行为：“注意，这个函数也可能会从之前的异步启动返回错误代码。”

由于目前 CUDA 是并发执行的，如果真的发生了故障，我们并没有办法知道是哪个内核程序造成的故障。至于调试代码，如果难以通过同步操作来隔离故障，开发人员可以设置 CUDA_LAUNCH_BLOCKING 的环境变量，以迫使所有启动的内核同步。

3. CPU/GPU 同步

CUDA 的大多数 GPU 命令涉及内存复制或内核程序启动的执行，而也有一个重要的命令子集帮助 CUDA 驱动程序跟踪 GPU 处理命令缓冲区时的进展。由于应用程序不知道一个给定的 CUDA 内核程序会运行多久，所以 GPU 本身必须给 CPU 汇报工作进度。图 2-26 显示的是命令缓冲区和“同步位置”（也位于锁页主机内存上），它们被 CUDA 驱动程序和 GPU 使用，以跟踪相应进程。一个单调递增的整数值（“进度值”）是由驱动程序维护，每一个主

⊖ http://bit.ly/13UqBm0。

要的 GPU 操作后都跟着一个将新进度值写入共享同步位置的命令。如图 2-26 中的例子，进度值将一直为 3，直到 GPU 完成当前命令的执行并将 4 写入同步位置中为止。

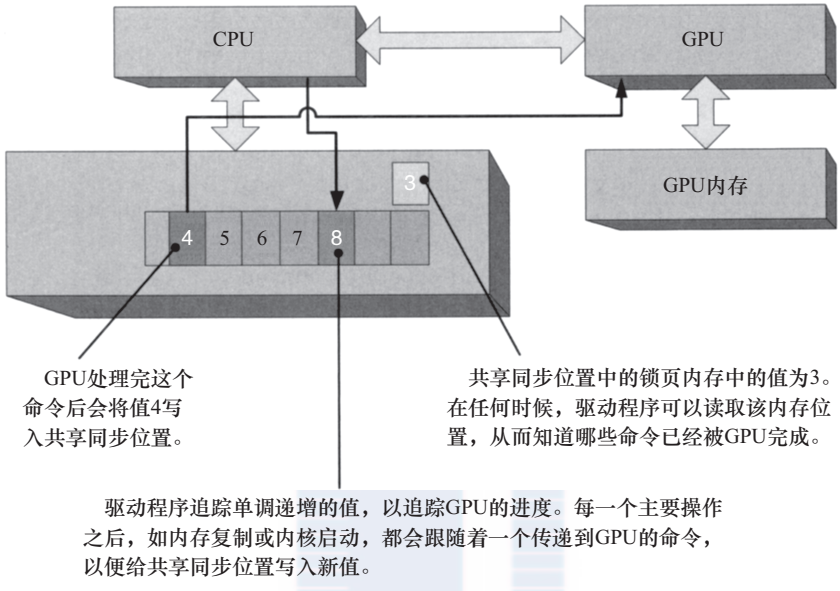


图 2-26 共享同步值（条件满足前）

CUDA 既隐式又显式地暴露了这些硬件功能。上下文范围的同步通过简单调用 cuCtxSynchronize()、cudaThreadSynchronize() 等函数来检查 GPU 请求的最近同步值，并且一直等待，直到同步位置获得该值。例如，在图 2-27 中，如果由 CPU 写入的命令 8 之后紧接着 cuCtxSynchronize() 或 cudaThreadSynchronize() 函数，则驱动程序将一直等待，直到共享同步值大于或等于 8 为止。

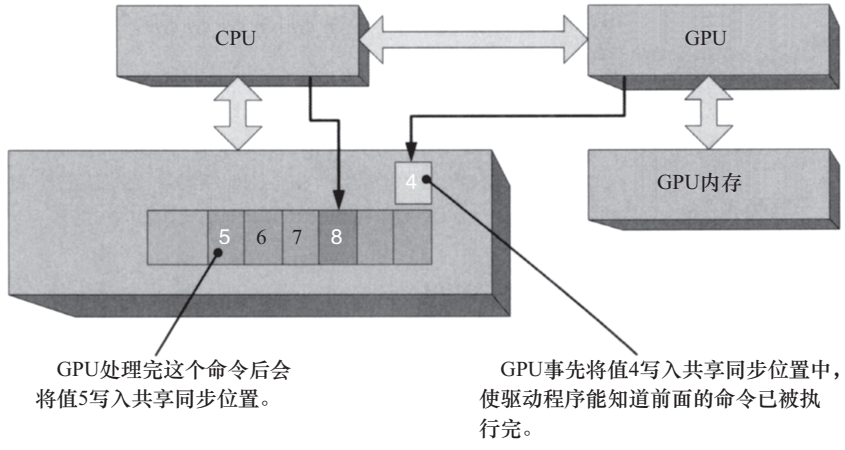


图 2-27 共享同步值（条件满足后）

CUDA 事件则更明确地暴露了这些硬件能力。cuEventRecord() 函数的作用是将一个命令加入队列使得一个新的同步值写入共享同步位置中，cuEventQuery() 和 cuEventSynchronize() 则分别用于检查和等待这个事件的同步值。

早期版本的 CUDA 只是简单地轮询共享的同步位置，反复地读内存，直到等待准则满足为止。但是这种方法代价很大，且只有当应用程序不必等待太久时才有用（即同步位置不一定要被读取很多遍，就可以因等待标准已经得到满足而退出）。对大多数应用程序来说，基于中断的方案（CUDA 公开称为“阻塞同步”）更好，因为它们使 CPU 等待线程挂起，直到 GPU 发出中断信号为止。驱动程序将 GPU 中断映射到一个特定的线程同步原语平台，如 Win32 事件或 Linux 的信号。当应用程序开始等待时，若等待条件不成立，这样做可以挂起 CPU 的线程。

通过指定 CU_CTX_BLOCKING_SYNC 到 cuCtxCreate() 或指定 cudaDeviceBlockingSync 到 cudaSetDeviceFlags()，应用程序可以强制使用上下文范围的同步进入阻塞状态。然而，使用阻塞的 CUDA 事件（指定 CU_EVENT_BLOCKING_SYNC 到 cuEventCreate() 或指定 cudaEventBlockingSync 到 cudaEventCreate()）更可取，因为它们粒度更细且可以与任何类型的 CUDA 上下文进行无缝互操作。

敏锐的读者可能会关注 CPU 和 GPU 在不使用原子操作或其他同步原语的情况下读取和写入这种共享的内存位置。但由于 CPU 只读取共享位置，竞争条件并不是关注点。这样，最坏的情况是 CPU 读取了一个“过时的”值，从而导致其等待时间要比实际的长。

4. 事件和时间戳

主机接口有一个机载高分辨率计时器，它可以在写入一个 32 位的同步值时同时写一个时间戳。CUDA 使用这个硬件设施实现 CUDA 事件中的异步计时功能。

2.5.3　主机接口和内部 GPU 同步

GPU 可能包含多个引擎，以使内核执行和内存复制并发进行。在这种情况下，GPU 的驱动程序将写入一些命令，这些命令被分发到同时运行的不同引擎中。每个引擎都有自己的命令缓冲区和共享同步值，引擎进度的跟踪如图 2-26 和图 2-27 所述。图 2-28 显示的是两个复制引擎和一个计算引擎并行工作时的情形。其中，主机接口负责读取命令并将其调度到相应引擎。在图 2-28 中，一个主机到设备的内存复制操作和两个相关操作（一个内核启动和一个设备到主机的内存复制）已被提交给硬件。依据 CUDA 编程抽象模型，这些操作都是在同一个流上进行的。这个流就像是一个 CPU 线程，在内存复制之后接受内核启动的提交。因此，CUDA 驱动程序为了实现内部 GPU 同步必须将命令插入主机接口的命令流中。

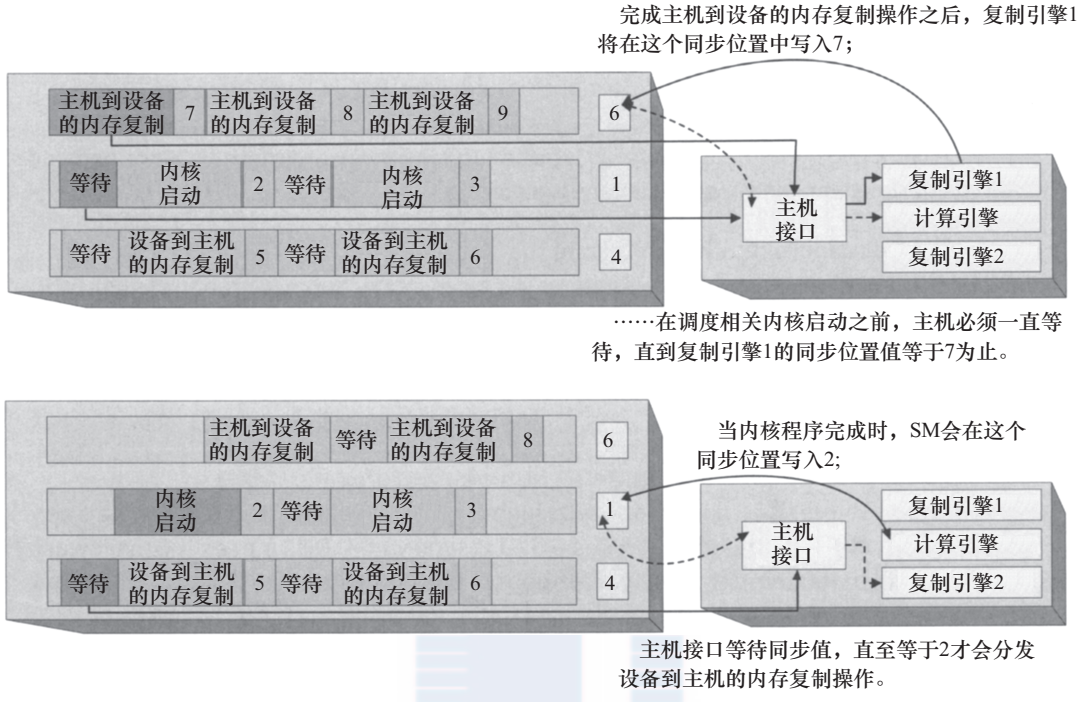


图 2-28 内部 GPU 同步

如图 2-28 所示，主机接口在协调流同步需求上起着核心作用。例如，在完成所需的内存复制之前，内核是启动不了的。此时，DMA 单元可以停止给既定引擎传递命令，直到共享的同步位置获得一个特定值为止。此操作与 CPU/GPU 同步类似，但 GPU 是对它内部的不同引擎进行同步的。

在此硬件机制之上的软件抽象模型是一个 CUDA 流。它在该操作中就像 CPU 线程一样，每个流都是串行排队，为了并发执行需要多个流。由于引擎间共享命令缓冲区，应用程序必须在不同的流里以软件的方式流水线化它们的请求。因此，它们必须完成

```
foreach stream
  Memcpy device←host
  Launch kernel
  Memcpy host←device
```

而不是

```
foreach stream
       Memcpy device←host
foreach stream
       Launch kernel
foreach stream
       Memcpy host←device
```

没有软件方式的流水线，DMA 引擎将会通过引擎的同步以维护每个流的串行执行模式，

这将打破并发性。

开普勒架构上的多 DMA 引擎

英伟达最新开普勒架构的硬件实现了一个引擎对应一个 DMA 单元，从而避免了应用程序需要以软件的方式对它们的流操作进行流水线化。

2.5.4　GPU 间同步

由于图 2-26 ~ 图 2-28 中的同步位置都是在主机内存上，所以它们可以被系统中的任何一个 GPU 访问。其结果是，在 CUDA 4.0 中，英伟达能够在 cudaStreamWaitEvent() 和 cuStreamWaitEvent() 函数的形式中添加 GPU 之间的同步。这些 API 调用导致驱动程序为主机接口将等待命令插入当前 GPU 的命令缓冲区中，使得 GPU 一直等待，直到事件的给定同步值被写入为止。从 CUDA 4.0 开始，事件不一定会被等待中的同一个 GPU 用信号唤醒。流原先只能在单个 GPU 的硬件单元之间同步执行，现在已提升到可以在 GPU 之间同步执行了。

2.6　GPU 架构

以下三种不同的 GPU 架构可以运行 CUDA：

- 特斯拉架构（Tesla）硬件，如在 2006 年推出的 GeForce 8800 GTX（G80）中。
- 费米架构（Fermi）硬件，如在 2010 年推出的 GeForce GTX 480（GF100）中。
- 开普勒架构（Kepler）硬件，如在 2012 年推出的 GeForce GTX 680（GK104）中。

GF100/GK104 命名来源于实现 GPU 的 ASIC 集成电路。它们之中的“F”和“K”分别代表费米架构和开普勒架构。

特斯拉和费米系列继承了英伟达的传统，他们会首先出货大型高端旗舰级芯片来赢取基准测试。这些芯片是昂贵的，因为英伟达的制造成本与制造 ASIC 时所需要的晶体管数量（进而影响芯片面积大小）密切相关。通过大型芯片的基准测试之后，会出货更小的芯片：中端的尺寸为大型的$\frac{1}{2}$，低端的为$\frac{1}{4}$，诸如此类。

与这一传统不同的是，英伟达的第一个开普勒级芯片定位于中端层次。上述大型芯片出货几个月后，第一个开普勒级芯片成功完成。GK104 拥有 35 亿个晶体管，而 GK110 则拥有 71 亿晶体管。

2.6.1　综述

从 CUDA 看来，GPU 可以作如下简化：

- 一个连接 GPU 与 PCIe 总线的主机接口；
- 0 ~ 2 个复制引擎；
- 一个连接 GPU 与其设备内存的 DRAM 接口；
- 一定数目的 TPC 或 GPC（纹理处理集群或图形处理集群），每个包含一定的缓存和一些流处理器簇（SMs）。

在本章结尾引用的有关架构的论文将对支持 CUDA 的 GPU 的功能进行更为完整的叙述，其中包括支持抗锯齿渲染等特定图形功能。

1. 主机接口

主机接口实现了上一节中已经描述的功能。读取 GPU 命令（如内存复制和内核启动命令），并将其分派给相应的硬件单元，而且还具备了 CPU 和 GPU 之间、GPU 上不同引擎之间和不同 GPU 之间同步的设施。在 CUDA 中，通过流和事件 API（见第 6 章），主机接口的基本功能得以体现。

2. 复制引擎

复制引擎可以在流处理器簇做计算时执行主机与设备之间的内存传输。最早的 CUDA 硬件并没有任何复制引擎，后来版本的硬件则包括了一个复制引擎，可以传输线性设备内存（CUDA 数组除外），而最新的 CUDA 硬件则包括了两个复制引擎，可以使 PCIe 总线饱和并可以在 CUDA 数组和线性内存之间转换。[⊖]

3. DRAM 接口

GPU 的 DRAM 接口包含了用于合并内存请求的硬件，可以支持超过 100GB/s 的带宽。越是新的 CUDA 硬件所含的 DRAM 接口越高级。最早的硬件（SM 1.x）有复杂的合并要求，其要求地址要连续且 64、128 或 256 字节对齐（根据操作数大小而定）。从 SM 1.2（GT200 或 GeForce GTX 280）开始，无地址对齐要求，地址可以依据数据局部性被合并起来。费米架构硬件（SM 2.0 和更高版本）有一个直写模式的二级缓存，可以提供和 SM 1.2 合并硬件一样的好处，并在数据重用时带来额外性能提升。

4. TPC 和 GPC

TPC 和 GPC 是介于整个 GPU 和流处理器簇之间的硬件单元，用于执行 CUDA 计算。特斯拉架构硬件将 SM 组合成 TPC（纹理处理集群），其中，TPC 包含有纹理硬件支持（特别包含一个纹理缓存）和 2 个或 3 个 SM，后面会有详细描述。费米架构硬件组则将 SM 组合为 GPC（图形处理器集群），其中，每个 GPU 包含有一个光栅单元和 4 个 SM。

⊖ 两个以上的复制引擎并没有多大的意义，因为每个引擎都能使 PCIe 的两个方向之一饱和。

在大多数情况下，CUDA 开发人员并不需要关心 TPC 或 GPC，因为对于计算硬件来说，SM 才是计算硬件的最重要的抽象单元。

5. 特斯拉架构和费米架构对比

第一代支持 CUDA 的 GPU 的代号为特斯拉，而第二代为费米。在开发过程中，它们属于机密代码，但英伟达决定将它们作为外部产品的名称来描述前两代支持 CUDA 的 GPU。为了制造悬念，英伟达选择使用“特斯拉”这一名称来形容使用 CUDA 机器构建计算集群的服务器级别的 GPU 板。[㊀]为了区分昂贵的服务器级别特斯拉板和架构族，本书指的架构族是“特斯拉架构硬件”、“费米架构硬件”和“开普勒架构硬件。”

特斯拉架构硬件与费米架构硬件之间的差异也适用于开普勒架构。

在执行非合并的内存事务的代码时，早期的特斯拉架构硬件存在严重的性能损失（高达 6 倍）。后来随着特斯拉架构硬件的实现，从 GeForce GTX 280 开始，将非合并事务的损失减少了约 2 倍。特斯拉架构硬件也有性能计数器，使开发人员能够测量有多少内存事务是非合并的。

特斯拉架构硬件只包括了一个 24 位整数乘法器，因此开发人员必须使用 __mul24() 等内建函数，以获得最佳性能。完整的 32 位乘法（即 CUDA 中原生的操作符 *）通过小型指令序列进行模拟。

特斯拉架构硬件对共享内存初始化为零，而费米架构硬件并不进行初始化。对于使用驱动程序 API 的应用程序而言，这种变化的一个细微副作用是：在费米架构下使用 cuParamSeti() 来传递指针参数到 64 位平台上的应用程序，并不能正常工作。由于参数是在特斯拉架构硬件的共享内存中传递的，参数未初始化的上半部分将成为 64 位指针中最重要的 32 位。

SM 1.3 中的 GT200 引入双精度支持。而 GT200 属于特斯拉架构第二代“赢取基准测试的”芯片。[㊁] 在当时，该特性被认为是有风险的，因此它以节省面积的方式实现，可以按照英伟达期望的双精与单精比率来增删硬件（如对于 GT200，这一比率为 1∶8）。费米架构更彻底的集成双精度支持且性能更好。[㊂] 最后，对于图形应用程序，特斯拉架构硬件是第一款支持 DirectX 10 的硬件。

费米架构硬件的功能比特斯拉架构硬件更强大。它支持 64 位寻址，增加了一级和二

㊀ 当然，特斯拉品牌刚出现时，费米架构硬件还没出现。营销部门的工程师告诉我们，体系架构代号和品牌名称都是“特斯拉”只是一个巧合。

㊁ 事实上，SM 1.2 与 SM 1.3 唯一的不同是 SM 1.3 支持双精度。

㊂ 在 SM 3.x 中，英伟达不再把双精度浮点性能当做影响 SM 版本的因素，因此，GK104 的双精度浮点运算性能较差，而 GK110 的较好。

级缓存，加入了一个完整的 32 位整数乘法指令和一些专门用于扫描算法的新指令，它增加了表面加载 / 存储操作，以便 CUDA 内核可以在不使用纹理硬件的情况下就能读取和写入 CUDA 数组，它是第一个以多复制引擎为特色的 GPU 系列，改善了虚函数等 C++ 代码支持。

费米架构硬件并不包括跟踪非合并内存传输所需的性能计数器。此外，因为它不包括有 24 位乘法器，费米架构硬件在运行要使用 24 位内建乘法函数时可能会出现一些小的性能损失。在费米架构中，使用乘法运算符 * 是最快速的方式。

对于图形应用程序，费米架构硬件可以运行 DirectX 11。表 2-1 总结了特斯拉架构和费米架构硬件之间的差异。

表 2-1 特斯拉架构和费米架构硬件之间的差异

特　　性	特斯拉架构	费米架构
非合并内存事务造成的性能损失	高达 8 倍①	高达 10%②
24 位乘法器	√	
32 位乘法器		√
共享内存 RAM	√	
一级缓存		√
二级缓存		√
并发内核		√
表面加载 / 存储		√

①：对于 Tesla 硬件，高达 2 倍。

②：启用 ECC 时，高达 2 倍。

6. 纹理操作的细微差别

特斯拉架构和费米架构硬件之间的一个细微差异是这样的：在特斯拉架构的硬件中，执行纹理操作的指令，其输出会覆盖输入寄存器向量。在费米架构硬件中，输入和输出寄存器向量可以是不同的。其结果是，特斯拉架构硬件可能要使用额外的指令来移动纹理坐标到输入寄存器，以恢复原始输入向量。

特斯拉架构和费米架构硬件的另一个微小的区别是，当对一维 CUDA 数组进行纹理操作时，费米架构硬件通过使用二维纹理并设置 0.0 为第二个坐标来模拟相应功能。由于这个模拟的成本只需一个额外的寄存器和少数额外指令，所以应用程序很少会注意到这一差异。

2.6.2 流处理器簇

GPU 的力量之源是流处理器簇（也称为 SM）。就像上一节所提到的，每个 SM 1.X 硬件中的 TPC 包含 2 个或 3 个 SM，而每个 SM 2.X 硬件中的 GPC 则包含 4 个 SM。最先支持 CUDA 的 GPU——G80 或 GeForce 8800 GTX，包含 8 个 TPC，而每一个 TPC 中有 2 个 SM，所以一共有 16 个 SM。接下来的一个支持 CUDA 的大 GPU 是 GT200 或 GeForce GTX 280，它增加了 TPC 中的 SM 数量，包含 10 个 TPC 并且每个 TPC 含有 3 个 SM，总共 30 个 SM。

每一个支持 CUDA 的 GPU 中 SM 的数量可能从二到几十个，并且每一个 SM 包含：

- 执行单位，用以执行 32 位整数和单、双精度浮点数运算；
- 特殊函数单元（SFU），用以计算 log/exp, sin/cos, rcp/rsqrt 的单精度近似值；
- 一个线程束调度器，用以协调把指令分发到执行单元；
- 一个常量缓存，用于广播式传送数据给 SM；
- 共享内存，用于线程之间的数据交换；
- 纹理映射的专用硬件。

图 2-29 展现了特斯拉架构流处理器簇（SM 1.X）。它包含了 8 个流处理器，支持 32 位整数和单精度浮点数运算。第一个 CUDA 硬件根本不能支持双精度浮点数运算，但是自从 GT200 开始，SM 就包含了双精度浮点数运算的单元。[㊀]

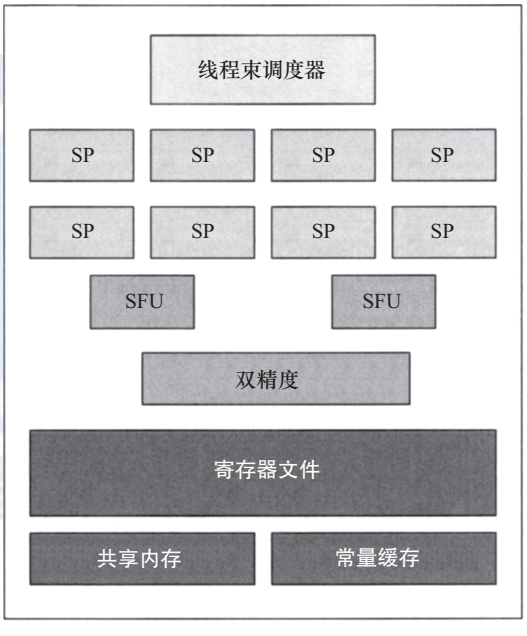


图 2-29　SM 1.x

图 2-30 展示了费米架构流处理器簇（SM 2.0）。不同于特斯拉架构的分离方式实现的双精度浮点数支持，费米架构的每一个 SM 2.0 拥有完整的双精度支持。虽然双精度执行速度比单精度慢，但由于 SM 2.0 的单 – 双精比率比特斯拉架构的 8 : 1 小得多，所以整体双精度性能要高很多。[㊁]

图 2-31 显示的是更新过后的费米架构流处理器簇（SM 2.1），可以在诸如 GF104 芯片上找到。为了获得更高的性能，英伟达选择增加流处理器的数量到 48 个。SFU 同 SM 的比率从 1 : 8 增加到 1 : 6。

图 2-32 显示了最近的（截止到本文写作时）流处理器簇的设计，它们为我们展现了英伟达最新开普勒架构硬件的特色。这种设计与前几代的设计完全不同，被英伟达称为 SMX（下一代 SM）。其中，核心的数量增加了 6 倍，到了 192 个，并且每个 SMX 都远大于以往 GPU 中类似的 SM。最大的费米架构 GPU——GF110 有大约 30 亿晶体管，共 16 个 SM；而 GK104 有 35 亿个晶体管和更高的性能，却只有 8 个 SMX。为了节省面积和提高能耗效率，英伟达为每一个 SM 增加了许多共享内存 / 一级缓存资源。就像费米架构的 SM 一样，每个开普勒架构 SMX 拥有 64KB 缓存，它可以分割成 48KB 一级缓存与 16KB 共享内

㊀ GT200 添加了一些指令集和双精度支持（如共享内存的原子操作），因此 GT200 的指令集中没有双精度的属于 SM 1.2，有双精度的属于 SM 1.3。

㊁ 对于开普勒架构的硬件，英伟达可以根据 GPU 的目标市场不同将浮点性能进行调整。

存，或者是 48KB 共享内存与 16KB 一级缓存。对于 CUDA 开发者来说，开普勒架构相比之前的架构，更鼓励将数据存储在寄存器中（而不是一级缓存，或者共享内存）。

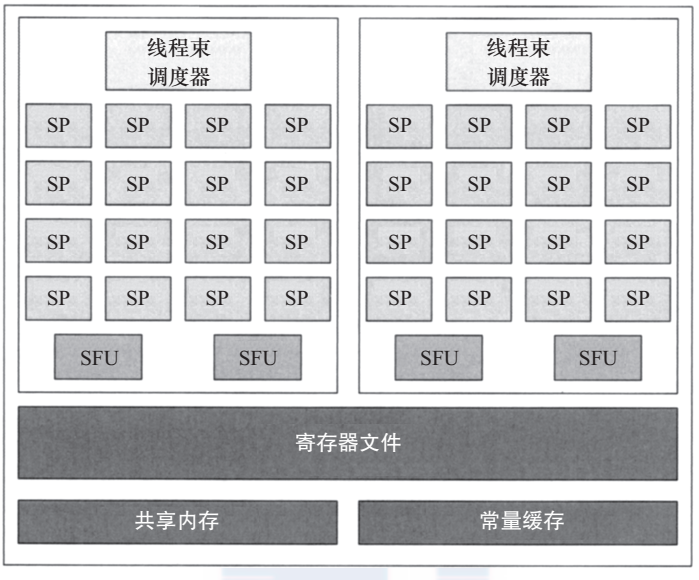


图 2-30 SM 2.0（费米架构）

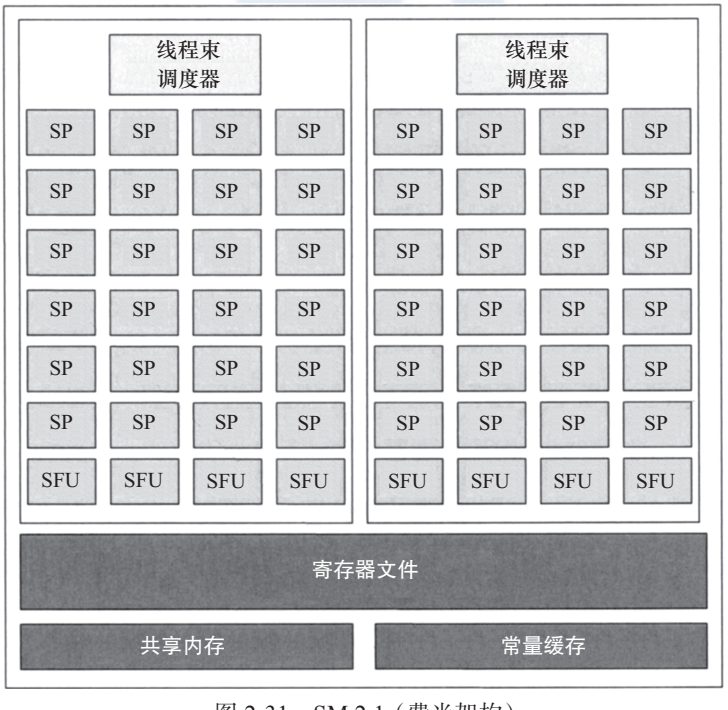


图 2-31 SM 2.1（费米架构）

图 2-32　SM 3.0（SMX）

2.7　延伸阅读

英伟达已经在其网站上的白皮书中详细描述了费米架构和开普勒架构。下面的白皮书介

绍了费米架构。

NVIDIA GeForce GPU 的下一代：www.nvidia.com/object/GTX_400_architecture.html。

以下的白皮书介绍了开普勒架构及其在英伟达 GeForce GTX 680（GK104）的具体实现：www.geforce.com/Active/en_US/en_US/pdf/GeForce-GTX-680-Whitepaper-FINAL.pdf。

英伟达的工程师们也发表了多篇架构方面的论文，提供了各种支持 CUDA 的 GPU 的更详细的描述：

Lindholm, E., J. Nickolls, S. Oberman, and J. Montrym. NVIDIA Tesla: A unified graphics and computing architecture. *IEEE Micro* 28 (2), March–April 2008, pp. 39–55.

Wittenbrink, C., E. Kilgariff, and A. Prabhu. Fermi GF100 GPU architecture. *IEEE Micro* 31 (2), March–April 2011, pp. 50–59.

Wong 等人使用 CUDA 开发了验证性实验，并阐明了特斯拉架构硬件方面的知识点：

Wong, H., M. Papadopoulou, M. Sadooghi-Alvandi, and A. Moshovos. Demystifying GPU microarchitecture through microbenchmarking. 2010 IEEE International Symposium on Performance Analysis of Systems and Software (IPSASS), March 28–30, 2010, pp. 235–246.

HZ BOOKS
华章图书

第 3 章 Chapter 3

软件架构

本章将会介绍 CUDA 的软件架构。在第 2 章中，我们介绍了 CUDA 的硬件平台和同 CUDA 交互的方法，我们会在本章的开始部分向大家介绍 CUDA 的软件平台和 CUDA 所支持的操作环境。然后，会对 CUDA 中的每个软件抽象从设备与上下文（context）到模块（module）与内核到内存作简要的介绍。这一部分在描述特定的软件抽象如何被硬件所支持时会回顾第 2 章中的内容。最后，我们会花一些时间对比 CUDA 运行时（CUDA runtime）和驱动程序 API（driver API），并考察 CUDA 源代码是怎样编译为在 GPU 上执行的微码（microcode）的。当然本章仅仅是对 CUDA 软件架构的一个概览，更多主题以及更详细的内容会在后面的章节中展开。

3.1 软件层

图 3-1 展示了 CUDA 应用程序中的不同层次，从应用程序本身到 CUDA 硬件上执行的 CUDA 驱动程序。除了内核模式驱动程序以外的所有软件都执行在目标操作系统的非特权用户模式下。在现代多任务操作系统的安全模型中，用户模式是“不可信任的”，硬件与操作系统软件必须采取措施严格分割一个与另一个应用程序。对 CUDA 来说，这意味着一个 CUDA 应用程序使用的主机和设备内存无法被另一个 CUDA 程序使用，唯一的例外是这些程序申请内存共享，这只有在内核模式下才可做到。

CUDA 库，例如 cuBLAS，建立在 CUDA 运行时或驱动程序接口层之上。CUDA 运行时是支持 CUDA 的集成 C++/GPU 工具的库。当 nvcc 编译器分割 .cu 文件分别发送至主机和设备，主机的部分会自动生成对 CUDA 运行时的调用，以方便像 nvcc 中用特别的 3 对尖括号 <<<>>> 启动内核这样的操作。

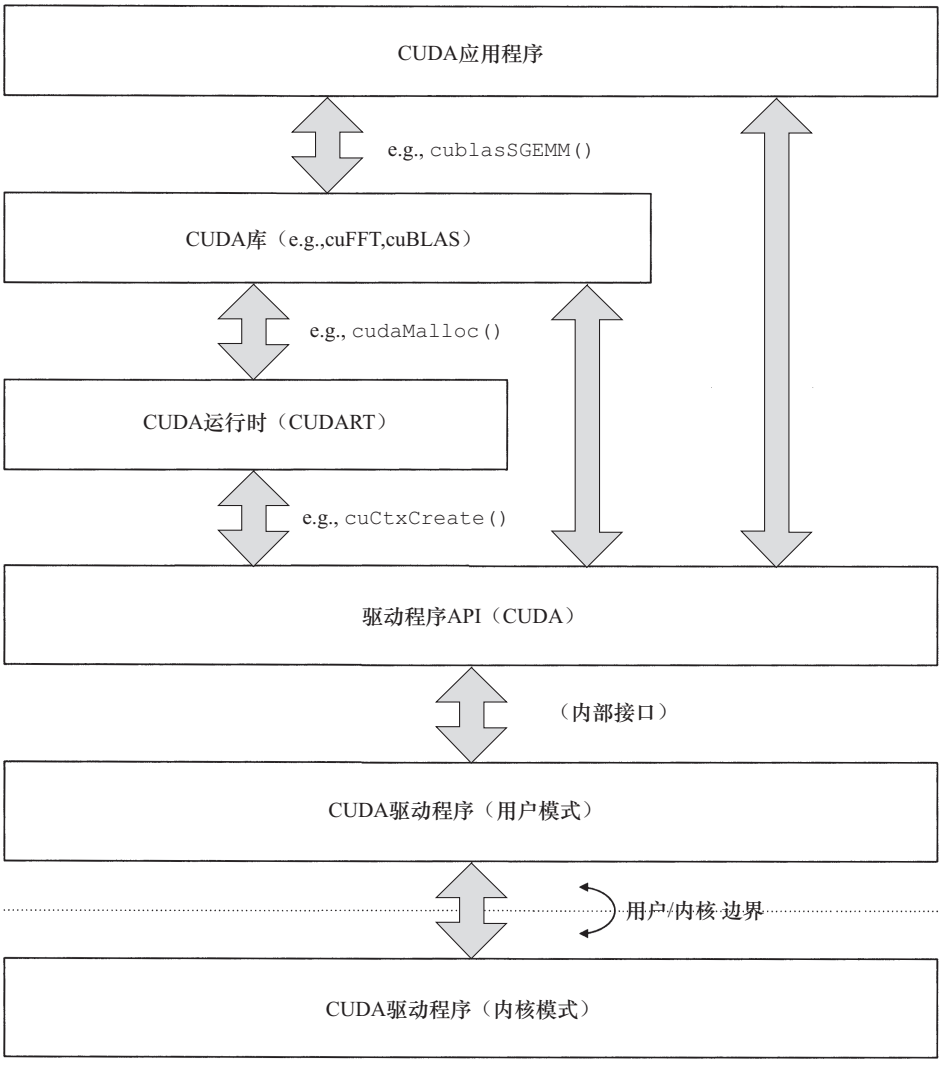


图 3-1 CUDA 软件层次结构

CUDA 驱动程序 API，由 CUDA 用户模式驱动程序直接导出，是 CUDA 应用程序可用的最底层 API。驱动程序 API 直接调用用户模式驱动程序，之后可能会继续调用内核模式做分配内存等操作。驱动程序 API 和 CUDA 运行时中的函数名称分别以 cu*() 和 cuda*() 打头，许多函数，像 cudaEventElapsedTime()，在本质上是相同的，只是在前缀上有一些区别。

3.1.1 CUDA 运行时和驱动程序

CUDA 运行时（经常简写为 CUDART）是一个由 CUDA 语言集成环境使用的库。每一个版本的 CUDA 工具集（CUDA toolchain）都有其特定的 CUDA 运行时版本，应用程序会自

动链接和运行时兼容的工具集，程序只有当匹配版本的 CUDART 在路径中可用时才能正确运行。

CUDA 驱动程序是后向兼容的，支持所有同版本 CUDA 编写的程序和旧版本程序。它提供一个相对低层的“驱动程序 API”（在 cuda.h 中），使开发者能够更紧凑地整理资源和为初始化定时。驱动程序的版本可由 cuDriverGetVersion() 函数获取。

```
CUresult CUDAAPI cuDriverGetVersion(int *driverVersion);
```

函数返回一个十进制数，给出由驱动程序支持的 CUDA 版本，例如，3010 代表 CUDA3.1，5000 代表版本 5.0。

表 3-1 总结了 CUDA 每个版本中的特性。在 CUDA 运行时应用程序中，版本信息由 cudaDeviceProp 结构体中的 major 和 minor 成员提供。我们会在 3.3.2 小节中详细审视 cudaDeviceProp。

表 3-1　CUDA 驱动程序每版本新特性

CUDA 版本	引入的驱动程序新特性
1.0	CUDA
1.1	流与事件，并发 1D 内存复制和内核执行
2.0	3D 纹理操作
2.1	提升 OpenGL 互用性
2.2	可分享、映射和写结合锁定内存；等步长内存纹理操作
3.0	费米架构，多引擎复制和内核并行执行
3.1	GPUDirect
3.2	64 位寻址；CUDA 内核中的 malloc()/free()
4.0	统一虚拟寻址；提升的线程支持；主机内存注册；GPUDirect 2.0（点对点内存复制和映射）；分层纹理
4.1	立体纹理；进程间点对点映射
4.2	开普勒架构（Kepler）
5.0	动态并行；GPUDirect RDMA

CUDA 运行时需要机器中安装的驱动程序版本高于或等于由运行时支持的 CUDA 版本。如果驱动程序的版本低于运行时版本，CUDA 应用程序会初始化失败，弹出错误 cudaErrorInsufficientDriver(35)。CUDA 5.0 引入了设备运行时（device runtime）概念，这是 CUDA 运行时的一个子集，可被 CUDA 内核直接调用。有关设备运行时的更详细描述在第 7 章给出。

3.1.2　驱动程序模型

除了 Windows Vista 和其后续版本，所有 CUDA 运行的操作系统（Linux、MacOS 和 Windows XP）通过用户模式客户端驱动程序（user mode client driver）访问硬件。这些驱动程序都不约而同地回避了一个需求，这一情况在所有现代操作系统中都普遍存在，即硬件资源

可被内核代码操纵。现代的硬件，例如 GPU 可以巧妙地解决这个需求——通过分配特定的寄存器（例如给硬件提交工作的硬件寄存器）进入用户模式。因为用户模式不被操作系统信任，所以对用户模式下的硬件寄存器，硬件必须包含对抗流氓写入的保护程序。这样做的目的是防止用户模式代码通过直接内存存取（DMA）修改不应该修改的代码（例如操作系统内核代码）。

硬件在用户模式和命令流之间使用一个中间层来防止内存损坏，这样 DMA 技术只能在之前已经被验证且已被内核代码映射的内存上使用。驱动程序开发者必须小心验证编写的内核代码，以确保代码只会访问可以使用的内存。这样的好处是程序在向硬件提交工作时不会招致多余的内核模式转换花销，我们的驱动程序可以以峰值的效率执行。

许多操作，例如内存分配，仍然需要内核模式转换，因为编辑 GPU 页表只能在内核模式下执行。因此，用户模式驱动程序采取了一系列步骤来减少内核模式转换的次数，例如，CUDA 内存分配器尽量从内存池取出内存满足内存分配。

1. 统一虚拟寻址

在 2.4.5 小节中详细描述了统一虚拟寻址（unified virtual addressing，UVA），在 64 位 Linux，64 位 XPDDM 和 MacOS 操作系统上均可使用。在这些平台上，统一虚拟寻址是内置可用的。但在本书编写时，UVA 还不能够在 WDDM 上运行。

2. Windows 显示驱动模型

在 Windows Vista 操作系统中，微软公司采用了新的桌面演示模型，在这个模型中，屏幕的输出由后台缓冲区与页面翻转协作完成，正如同电子游戏一样。这个新的“ Windows 桌面管理器”（WDM）有能力比以往 Windows 版本更广泛地使用 GPU，所以微软公司决定修改 GPU 驱动模型以适应现行的桌面演示模型。新的 GPU 驱动模型——Windows 显示驱动模型（Windows display driver model，WDDM）已经是 Windows Vista 及其后续版本中的默认驱动模型。而 XPDDM 则继续使用在 Windows 的以往版本中[㊀]。

就 CUDA 而言，WDDM 有两个主要的改变：

1）WDDM 不再允许硬件寄存器被分配进入用户模式。硬件命令——甚至是启动 DMA 操作的指令——必须通过内核模式来调用。用户→内核转换成本对机器来说是一个十分昂贵的行为，机器中可以使用用户模式驱动程序缓冲区来暂存指令，延迟提交。

2）WDDM 的设计初衷是为了允许多应用程序同时使用同一 GPU，但 GPU 并不支持请求式调页，WDDM 中包括了在内存对象基础上模拟换页的工具。对图形应用程序，目标包括内存对象、Z 缓冲区或纹理内存；对 CUDA，内存对象包括全局内存和 CUDA 数组。由于驱动需要在内核调用之前建立好对 CUDA 数组的访问，所以 CUDA 数组可以被 WDDM 交

㊀ 特斯拉板（Tesla boards）（没有任何显示输出的能运行 CUDA 的电路板）可以在 Windows 上使用 XPDDM，称为特斯拉计算集群（Tesla compute cluster，TCC）驱动程序，可被 nvidia-smi 工具触发。

换。全局内存是驻留在一段连续的地址空间（可以存储指针的地方）里的，为了让内核成功启动，给定 CUDA 上下文中每一个内存对象都会驻留在这段地址空间上。

上述第一点的最主要影响：对 CUDA 请求的工作，例如，内核启动或异步内存复制操作，一般来说不会立即提交给硬件去操作。

提交被挂起的工作的一个惯例操作是查询默认流（NULL stream）：cudaStreamQuery(0) 或 cuStreamQuery(NULL)。如果没有挂起的工作，这一调用会迅速返回；如果有工作正在挂起，这项工作将会被提交，因为这个调用是异步的，执行可能在硬件处理结束之前返回给调用者。而在非 WDDM 平台上，查询 NULL 流总是非常快的。

上述第二点的主要作用是使 CUDA 内存分配控制更加灵活。在用户模式客户端驱动程序中，成功的内存分配意味着：一旦内存空间被分配，那么它无法为操作系统中的其他客户端程序（例如游戏或正在运行的 CUDA 应用程序）再次占用。在 WDDM 下，如果有应用程序在相同的 GPU 中竞争使用时间，Windows 能够交换出内存对象使每一个应用程序顺利运行。Windows 操作系统会尽其所能提高这项工作的效率。当然，对于所有的可能有换页操作的程序，尽量不让换页操作发生是最好的选择。

3. 超时检测与恢复

因为 Windows 使用 GPU 同用户交互，所以计算程序不能占用 GPU 过多的时间。在 WDDM 中，Windows 设定了强制延时界限（默认为 2 秒），这意味着，如果出现了持续的时间超过设定延时的情况，系统会弹出带有“显示驱动程序停止响应并已恢复”（display driver stopped responding and has recovered）的对话框，随后显示驱动程序重新启动。如果这种情况发生，在所有 CUDA 上下文所做的工作都会丢失。查看 http://bit.ly/16mG0dX 获取详细内容。

4. 特斯拉计算集群驱动程序

因为计算程序不需要 WDDM，英伟达提供**特斯拉计算集群**（TCC）驱动程序，只在特斯拉级板上可用（Tesla-class board）。特斯拉计算集群驱动程序（Tesla compute cluster driver）是一个用户模式下的客户端驱动程序，所以这并不需要**内核转换**（kernel thunk）去给硬件提交工作，我们可以使用 nvidia-smi 工具来启用或禁用它。

3.1.3　nvcc、PTX 和微码

nvcc 是 CUDA 开发者使用的编译器驱动程序，它可以使 CUDA 源代码转成一个可执行的 CUDA 应用程序。nvcc 可以做很多事，如编译、链接和用一条指令执行一个示例程序（本书中很多程序鼓励使用的一种用法），编译一个只在 GPU 上执行的 .cu 文件。

图 3-2 展示了使用 nvcc 的推荐工作流程，包括 CUDA 运行时和驱动程序 API 应用程序。对于规模很大的应用程序，nvcc 最好严格地编译 CUDA 代码并封装 CUDA 功能到可以被其他工具调用的代码，这是由于 nvcc 存在如下限制：

- nvcc 只在特定的编译器上工作。许多 CUDA 开发者不曾注意到这一限制，因为他们只使用支持 CUDA 的编译器。但是在软件生产环境中，CUDA 的代码量与其他代码相比简直是微不足道，所以编译器是否支持 CUDA 未必是决定编译器使用的因素。
- nvcc 所改变的编译环境可能与其他大量应用程序的生成环境不兼容。
- nvcc 会使用一些非标准内建类型（如 int2）和固有名称（如 __popc()）“破坏”命名空间的纯洁性。只有在最近的 CUDA 版本中，固有的符号才变成可选项，且可以通过引入适当的 sm_*_intrinsics.h 头文件来使用。

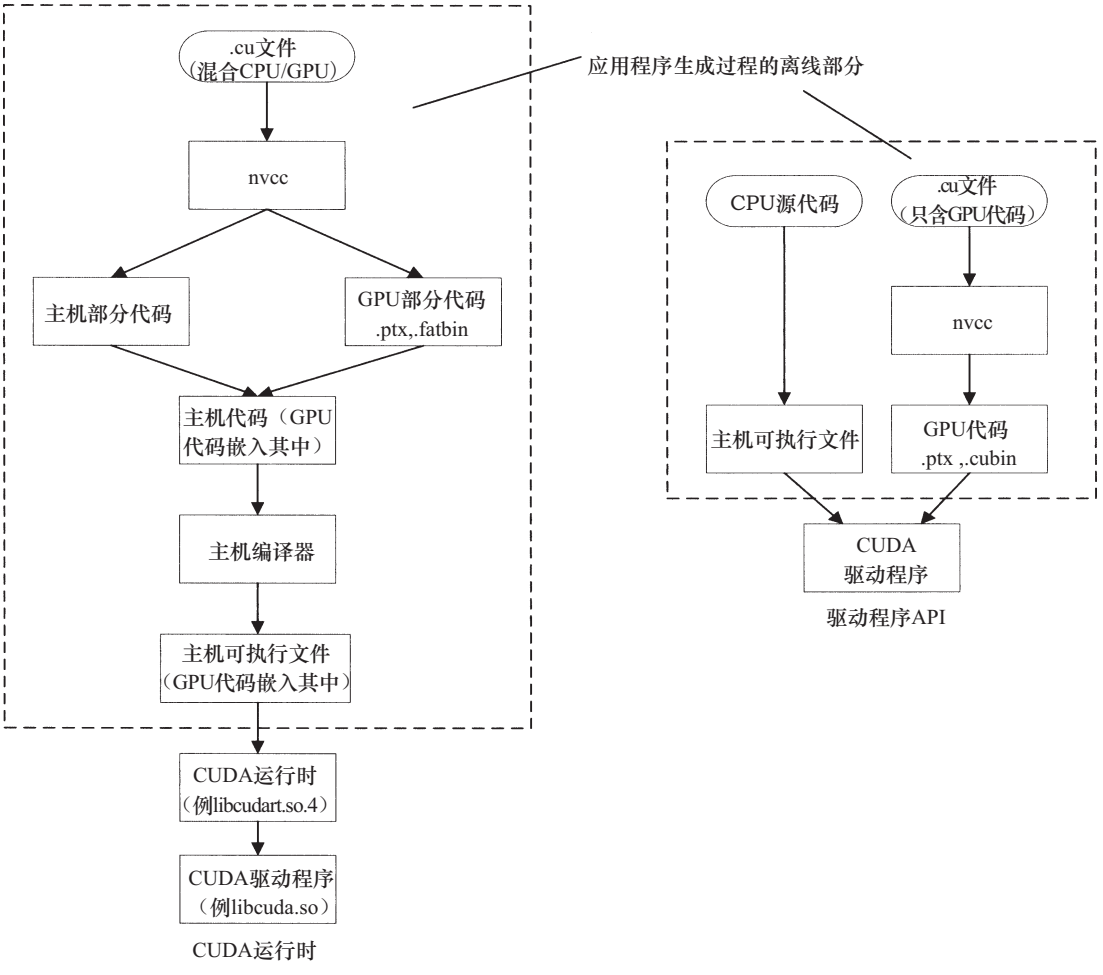


图 3-2 nvcc 工作流程图

对于 CUDA 运行时应用程序，nvcc 会把 GPU 代码转化为字符串字面值，嵌入可执行文件。如果选用 --fatbin 选项，可执行文件会自动加载目标 GPU 的微码，如果没有微码可用，会让驱动程序自动编译 PTX 为微码。

nvcc 与 PTX

并行线程执行（Parallel Thread eXecution，PTX）代码是编译后的 GPU 代码的一种中间形式，它可以再次编译为原生的 GPU 微码。这是英伟达公司为 CUDA 程序面向未来指令集所建立的一个创新机制，这意味着只要给定的 CUDA 内核的 PTX 是可用的，CUDA 驱动程序便可将其编译为在 GPU 上执行的程序的微码（即使在编写代码时 GPU 对其并不可用）。

PTX 可以离线或在线方式编译为 GPU 微码。离线编译指的是生成未来可在计算机上执行的程序。在图 3-2 中，我们圈出了 CUDA 编译过程中的离线编译部分。对于在线编译方式，我们可以理解为“即时”（just-in-time，JIT）编译，指的是程序可以在运行时将 PTX 代码在线编译为 GPU 微码。

nvcc 可以通过调用 PTX 编译器 ptxas 离线编译 PTX，这将会把 PTX 编译为特定 GPU 版本的原生微码。生成的微码存储在 CUDA 二进制形式的‘cubin’（发音类似“Cuban”）文件中，cubin 文件可以使用 cuobjdump --dump-sass 命令进行反汇编，这将根据特定 GPU 的微码生成 SASS 代码[㊀]。

PTX 同样可以通过 CUDA 驱动程序在线编译（JITted）。当我们运行使用 --fatbin 选项编译的（这是默认选项）CUDART 程序时，在线编译是自动启动的。每个内核的 .cubin 和 PTX 都包含在可执行文件里，如果硬件不支持可执行文件里的 .cubin，驱动程序便会编译 PTX。驱动程序将其缓存在磁盘上，因为编译 PTX 可能花费大量时间。

最后，PTX 可以在运行时通过显式调用 cuModuleLoadEx() 编译。驱动程序 API 不会自动嵌入或加载 GPU 微码。.cubin 和 .ptx 均作为参数传给 cuModuleLoadEx() 函数，如果 .cubin 不兼容机器的 GPU 架构，会返回一个错误。对驱动程序 API 开发者来说，一个合理的处理策略是编译并嵌入 PTX，并且应该总调用 cuModuleLoadEx() 以 JIT 编译到 GPU，依赖驱动程序来缓存编译后的微码。

3.2　设备与初始化

设备就是指真实的 GPU 实体。当 CUDA 初始化完成（显式调用驱动程序接口 cuInit() 或隐式调用 CUDA 运行时函数），CUDA 驱动程序将枚举可用的设备，并创建一个全局数据结构体，这个结构体包括设备名称和一些不可变的参数，例如设备内存数量和最大时钟速率等。

在一些平台上，英伟达包含设置特定设备的工具。nvidia-smi 工具可设置 GPU 的策略，例如，它可以启用或禁用 ECC（纠错），它也可以用来控制设备创建的 CUDA 上下文数量。下面是一些可能的模式：

㊀　审视 SASS 代码是程序优化的主要策略之一。

- 默认：多个 CUDA 上下文可被设备创建。
- 独占模式：只有一个 CUDA 上下文可被设备创建。
- 禁止模式：禁止设备创建 CUDA 上下文。

如果一个设备已被列出，但无法在其上创建上下文，那么设备可能是处于禁止模式或者独占模式中，对于后者，说明已有一份 CUDA 上下文被创建。

3.2.1 设备数量

应用程序可以通过调用 cuDeviceGetCount() 或 cudaGetDeviceCount() 函数查询有多少设备可用。之后，设备可以通过一个来自范围 [0..DeviceCount–1] 的索引来引用它。驱动程序 API 需要应用程序调用 cuDeviceGet() 来映射设备的索引到一个设备句柄（CUdevice）上。

3.2.2 设备属性

驱动程序 API 应用程序可以通过调用 cuDeviceGetName() 来查询设备名称，调用 cuDeviceTotalMem() 查询全局内存。设备的主要与次要计算能力（即 SM 版本，例如第一代费米架构 GPU 的计算能力为 2.0）可以使用 cuDeviceComputeCapability() 函数查询。

CUDA 运行时应用程序可以调用 cudaGetDeviceProperties()，返回包括设备名称与属性信息的结构体 cudaDeviceProp，表 3-2 给出了结构体中成员的描述。

CUDA 驱动程序 API 接口函数 cuDeviceGetAttribute() 可以查询设备属性，根据 CUdevice_attribute 参数一次返回一个属性。CUDA 5.0 中，CUDA 运行时添加了相同的功能函数：cudaDeviceGetAttribute()，想必因为基于结构体的接口在设备上运行时很笨重。

表 3-2 cudaDeviceProp 成员

cudaDeviceProp 成员	描　述
`char name[256];`	ASCII 字符串设备标识
`size_t totalGlobalMem;`	设备可用全局内存（字节）
`size_t sharedMemPerBlock;`	每线程块可用共享内存（字节）
`int regsPerBlock;`	每线程块可用 32 位寄存器
`int warpSize;`	线程束中线程容量
`size_t memPitch;`	最大内存复制步长
`int maxThreadsPerBlock;`	每线程块最大线程数
`int maxThreadsDim[3];`	线程块维度的最大值
`int maxGridSize[3];`	网格维度最大值
`int clockRate;`	时钟频率（千赫）
`size_t totalConstMem;`	设备可用常量内存（字节）
`int major;`	主计算能力

（续）

cudaDeviceProp 成员	描　　述
int minor;	次计算能力
size_t textureAlignment;	纹理对齐要求
size_t texturePitchAlignment;	要求绑定到等步长内存的纹理索引满足等步长对齐约束
int deviceOverlap;	设备可以并发进行内存复制和内核程序执行。已弃用，使用 asyncEngineCount 代替
int multiProcessorCount;	设备中的处理器簇数量
int kernelExecTimeoutEnabled;	指明内核程序是否有运行时限制
int integrated;	设备是否是集成 GPU（而不是独立 GPU）
int canMapHostMemory;	设备可以使用 cudaHostAlloc/cudaHostGetDevicePointer 对主机内存进行映射
int computeMode;	计算模式（参见 ::cudaComputeMode)
int maxTexture1D;	最大 1D 纹理大小
int maxTexture1DMipmap;	最大 1D 细化纹理大小
int maxTexture1DLinear;	线性内存相关的 1D 纹理大小
int maxTexture2D[2];	最大 2D 维度
int maxTexture2DMipmap[2];	最大 2D 细化纹理维度
int maxTexture2DLinear[3];	最大维度（宽）
int maxTexture2DGather[2];	执行纹理聚集操作时的最大纹理维度
int maxTexture3D[3];	最大 3D 纹理维度
int maxTextureCubemap;	最大立方图纹理维度
int maxTexture1DLayered[2];	最大 1D 分层纹理维度
int maxTexture2DLayered[3];	最大 2D 分层纹理维度
int maxTextureCubemapLayered[2];	最大立方图分层纹理维度
int maxSurface1D;	最大 1D 表面大小
int maxSurface2D[2];	最大 2D 表面维度
int maxSurface3D[3];	最大 3D 表面维度
int maxSurface1DLayered[2];	最大 1D 分层表面维度
int maxSurface2DLayered[3];	最大 2D 分层表面维度
int maxSurfaceCubemap;	最大立方图表面维度
int maxSurfaceCubemapLayered[2];	最大立方图分层表面维度
size_t surfaceAlignment;	表面对齐要求
int concurrentKernels;	设备可能执行多个内核并发
int ECCEnabled;	设备是否打开 ECC
int pciBusID;	设备 PCI 总线 ID

（续）

cudaDeviceProp 成员	描　述
int pciDeviceID;	PCI 设备 ID
int pciDomainID;	PCI 域 ID
int tccDriver;	如果设备是一个启用 TCC 驱动程序的特斯拉设备，返回 1
int asyncEngineCount;	异步引擎数量
int unifiedAddressing;	设备与主机共享统一的地址空间
int memoryClockRate;	内存峰值时钟频率（千赫）
int memoryBusWidth;	全局内存总线宽度（位）
int l2CacheSize;	二级缓存大小（字节）
int maxThreadsPerMultiProcessor;	每个处理器簇最大驻留线程数

3.2.3 无 CUDA 支持情况

CUDA 运行时程序可以运行在不能运行 CUDA 的机器或未安装 CUDA 的机器上。如果 cudaGetDeviceCount() 函数返回 cudaSuccess 和一个非 0 的设备数，CUDA 就是可用的。

与 CUDA 运行时相反，当使用驱动程序 API 时，除非驱动程序二进制文件是可用的，否则直接链接 nvcuda.dll（Windows）或 libcuda.so（Linux）的可执行文件便不会成功加载。正因为驱动程序 API 需要 CUDA，所以当 CUDA 不存在时，直接启动程序会引发图 3-3 中的错误。

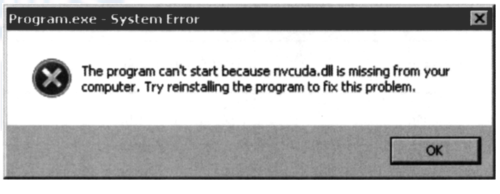


图 3-3　当 CUDA 不存在时的错误对话框（Windows）

对于 CUDA 的应用程序，CUDA SDK 提供了一组头文件和一个 C 源文件，包装了驱动程序 API，这样，应用程序便可以检查 CUDA 而避免操作系统发出异常信号。这些文件，存放在子目录 <SDKRoot>/C/common/inc/dynlink 下，可以代替 CUDA 的核心文件。它们会插入一组中间函数，可以在 CUDA 可用时延迟加载 CUDA 库。

作为例子，让我们比较两个小程序。它们使用驱动程序 API 初始化 CUDA 并输出系统中的设备名字。代码清单 3-1 给出了 init_hardcoded.cpp 文件的代码，通过下面的命令可以使用 CUDA SDK 编译这个文件：

```
nvcc -oinit_hardcoded -I ../chLib init_hardcoded.cpp -lcuda
```

使用 nvcc 编译 C++ 文件，不包含任何的 GPU 代码，是一种很方便的获取 CUDA 头文件的方式。在这行命令的开始，-oinit_hardcode 确定了输出的可执行文件的根名称。命令最后的 -lcuda 使 nvcc 链接驱动程序 API 库，缺少它，会引发一个链接错误。这个程序硬链接（hard-link）CUDA 驱动程序 API，所以在未安装 CUDA 的机器上会编译失败。

代码清单 3-1　初始化（硬编码）

```
/*
 * init_hardcoded.cpp
 *
 */
#include <stdio.h>

#include <cuda.h>
#include <chError.h>

int
main()
{
    CUresult status;
    int numDevices;

    CUDA_CHECK( cuInit( 0 ) );
    CUDA_CHECK( cuDeviceGetCount( &numDevices ) );

    printf( "%d devices detected:\n", numDevices );
    for ( int i = 0; i < numDevices; i++ ) {
        char szName[256];
        CUdevice device;
        CUDA_CHECK( cuDeviceGet( &device, i ) );
        CUDA_CHECK( cuDeviceGetName( szName, 255, device ) );
        printf( "\t%s\n", szName );
    }

    return 0;
Error:
    fprintf( stderr, "CUDA failure code: 0x%x\n", status );
    return 1;

}
```

代码清单 3-2 给出的程序不需要系统中装有 CUDA。可以看到，源代码几乎是相同的，仅有几行差别。其中，

```
#include <cuda.h>
```

被替换为：

```
#include "cuda_drvapi_dynlink.c"
#include "dynlink/cuda_drvapi_dynlink.h"
```

而 cnInit() 函数调用改变为指定一个 CUDA 版本。即，

```
CUDA_CHECK( cuInit(0) );
```

被替换为：

```
CUDA_CHECK( cuInit( 0, 4010 ) );
```

这里我们为 CUDA_CHECK 传入了第二个参数 4010，代表 CUDA 4.1 版本，如果系统不包括 4.1 版本的函数库，程序会编译失败。

注意你可以单独编译然后链接 cuda_drvapi_dynlink.c 到你的应用程序里，代替直接‘include’这个文件到源代码中。头文件和 C 文件一同工作，在驱动程序 API 中插入一组包装好的函数。头文件使用预处理器重新命名驱动程序 API 函数以包装在 cuda_drvapi_dynlink.h

头文件中声明的函数（例如调用 cuCtxCreate() 会实际调用 tcuCtxCreate()）。在支持 CUDA 的系统中，驱动程序动态链接库是动态加载的，包装好的函数会调用在初始化时从驱动程序动态链接库获取的函数指针；在没有 CUDA 的系统中，或者驱动程序不匹配请求的 CUDA 版本，初始化函数会返回一个错误。

代码清单 3-2 初始化（dynlink）

```
/*
 * init_dynlink.cpp
 *
 */

#include <stdio.h>

#include "dynlink/cuda_drvapi_dynlink.h"
#include <chError.h>

int
main()
{
    CUresult status;
    int numDevices;

    CUDA_CHECK( cuInit( 0, 4010 ) );
    CUDA_CHECK( cuDeviceGetCount( &numDevices ) );

    printf( "%d devices detected:\n", numDevices );
    for ( int i = 0; i < numDevices; i++ ) {
        char szName[256];
        CUdevice device;
        CUDA_CHECK( cuDeviceGet( &device, i ) );
        CUDA_CHECK( cuDeviceGetName( szName, 255, device ) );
        printf( "\t%s\n", szName );
    }

    return 0;
Error:
    fprintf( stderr, "CUDA failure code: 0x%x\n", status );
    return 1;

}
```

CUDA 专用动态链接库（CUDA-Only DLLs）

对 Windows 开发者，另一个在不支持 CUDA 的平台上创建 CUDA 应用程序的方式如下：

1）把 CUDA 代码转移到动态链接库。

2）调用 LoadLibrary() 显式加载动态链接库。

3）在 __try/__except 语句中调用 LoadLibrary() 以捕捉 CUDA 不存在情况下的异常。

3.3 上下文

上下文类似于 CPU 中的进程。除了少数例外，它是管理 CUDA 程序中所有对象生命周期的容器，包括如下部分：

- 所有内存分配（包括线性设备内存、主机内存、CUDA 数组）
- 模块
- CUDA 流
- CUDA 事件
- 纹理与表面引用
- 使用本地内存的内核的设备内存
- 进行调试、分析、同步操作时，所使用的内部资源
- 换页内存复制所使用的锁定中转缓冲区

CUDA 运行时不提供对 CUDA 上下文的直接访问，它通过延迟初始化（deferred initialization）来创建上下文。每一个 CUDART 库调用和内核调用都会检查上下文是否是当前使用的，如果必要的话，创建 CUDA 上下文（使用之前用 cudaSetDevice(), cudaSetDeviceFlags(), cudaGLSetGLDevice() 等函数设置的状态）。

许多应用程序倾向于控制初始化延迟时间。为了让 CUDART 进行没有任何负面效果的初始化，调用

```
cudaFree(0);
```

CUDA 运行时应用程序可以通过驱动程序 API 访问当前上下文栈（后文描述）。

对驱动程序 API 中的每个指定上下文状态的函数，CUDA 运行时把上下文和设备同等对待。对于驱动程序 API 函数 cuCtxSynchronize()，CUDA 运行时用 cudaDeviceSynchronize() 取而代之；对驱动程序 API 函数 cuCtxSetCacheConfig()，CUDA 运行时中则有 cudaDeviceSetCacheConfig() 对应。

当前上下文

为了代替当前上下文栈，CUDA 运行时提供 cudaSetDevice() 函数，为调用的线程设置当前上下文。一个设备可以是多个 CPU 线程的当前上下文[㊀]。

3.3.1 生命周期与作用域

所有与 CUDA 上下文相关的分配资源都在上下文被销毁的同时被销毁，除了少数例外，给定 CUDA 上下文创建的资源可能不能被其他的 CUDA 上下文使用，这一限制不仅适用于内存，也同样适用于 CUDA 流与 CUDA 事件等对象。

3.3.2 资源预分配

CUDA 尽力去避免“懒惰分配”（lazy allocation）。懒惰分配只分配需要的资源以避免因缺少资源导致的失败操作。例如，换页内存复制不会因内存不足而失败，因为机器通过分配

㊀ 早期版本的 CUDA 禁止上下文同时成为多个线程的当前状态，因为驱动程序并不是线程安全的。现在驱动程序实现了所需的同步——即使在应用程序调用同步函数，例如 cudaDeviceSynchronize() 的情况下。

锁页中转缓冲区以执行换页内存复制，而这个操作发生在上下文创建时。如果 CUDA 不能分配这些缓冲区，上下文创建就失败了。

在少量情况下，CUDA 不会预分配一个给定操作所需的全部资源。内核启动所需的本地内存数量可能被限制，因此 CUDA 不会预分配最大理论数量的内存。所以，当它需要比默认分配给 CUDA 上下文的内存值更多的本地内存时，内核启动可能失败。

3.3.3 地址空间

除了在上下文销毁时被自动销毁的（清理）对象外，另一个上下文中的重要抽象是地址空间：一组私有的虚拟内存地址，它可以分配线性设备内存或用以映射锁页主机内存。这些地址每一个上下文都不相同。同一地址对不同上下文可能有效也可能无效，当然也不会解析到相同的地址空间除非有特殊规定。CUDA 上下文的地址空间是与 CUDA 主机代码使用的 CPU 地址空间独立的。事实上，不同于共享内存的多 CPU，多 GPU 中的 CUDA 上下文并不共享一个内存空间。当系统使用统一虚拟地址时，CPU 和 GPU 共享相同的地址空间，其中的每个分配都有进程中唯一的地址。但在特殊情况下，CPU 和 GPU 只能读写各自的内存区，像映射的锁页内存（参见 5.1.3 小节）或点对点内存（参见 9.2.2 小节）。

3.3.4 当前上下文栈

大多数 CUDA 入口点并不含有上下文参数。取而代之的，它们在 CPU 线程的“当前上下文”上进行操作，它存储在线程的本地存储句柄里。在驱动程序 API 中，每一个 CPU 线程有一个当前上下文栈，创建一个上下文的同时将这个新上下文压入栈。

当前上下文栈有 3 个主要功能：

- 单线程应用程序可以驱动多个 GPU 上下文。
- 库可以创建并管理它们自己上下文，而不需要干涉调用者的上下文。
- 库不知晓调用它的 CPU 线程信息。

为 CUDA 建立当前上下文栈的最初动机是使单线程 CUDA 应用程序可以驱动多个 CUDA 上下文。在创建并初始化每个 CUDA 上下文后，程序可以将其中的某个上下文从栈弹出，使其变成一个飘浮上下文。由于对一个 CPU 线程，只有一个当前上下文，所以一个单线程 CUDA 应用程序可以在栈中依次压入和弹出上下文，使其在任何时间点上，只有一个飘浮上下文，实现对多个上下文的驱动。

在多数的驱动程序架构中，压入和弹出上下文的成本足够廉价，所以一个单线程应用程序可以保持多个 GPU 同时忙碌。对只在 Windows Vista 和其后续版本中的 WDDM 驱动程序，弹出当前上下文操作只在没有 GPU 命令挂起的时候才能快速运行。当有命令挂起，驱动程序会引发内核转换，以保证在弹出 CUDA 上下文之前提交挂起的命令[㊀]。

㊀ 这一花销不仅仅局限在驱动程序 API 或当前上下文栈上，当有命令挂起时，调用 cudaSetDevice() 来切换设备同样会引发一次 WDDM 上的内核转换。

当前上下文栈的另一个优势是驱动来自多个 CPU 线程的 CUDA 上下文的能力。使用驱动程序 API 的应用程序可以传递 CUDA 上下文到其他 CPU 线程中：使用 cuCtxPopCurrent() 函数弹出上下文，再使用 cuCtxPushCurrent() 从另一个线程中压入上下文。库可以使用这个功能创建 CUDA 上下文，而不需要了解或干涉它们的调用者。例如，一个 CUDA 插件库可以在初始化时创建它自己的 CUDA 上下文，然后弹出并保持其“漂浮”，除非被主程序调用才会中断。漂浮上下文使库完全对哪一个 CPU 线程调用不知情。当然这种使用方式也是有利有弊：一方面，漂浮上下文内存不会被第三方 CUDA 内核的恶意写入污染；另一方面，库只能够在分配的 CUDA 资源上进行操作。

附加和分离上下文

在 CUDA 4.0 之前，每一个 CUDA 上下文都有一个“使用计数”，上下文被创建时设置为 1。cuCtxAttach() 和 cuCtxDetach() 函数分别为使用计数加 1 或减 1 [㈠]。采用“使用记数”的目的是为了使应用程序创建的上下文与链接到的库联系起来，这样程序就可以通过自己创建的 CUDA 上下文与库相互操作了[㈡]。27

当 CUDART 第一次调用时，如果一个上下文已经处于当前状态，程序会把上下文附加到 CUDART 上，而不是新建一个上下文。CUDA 运行时无法访问上下文的“使用记数”。而在 CUDA 4.0 之后，使用计数已被弃用，使用 cuCtxAttach()/cuCtxDetach() 函数没有任何作用。

3.3.5　上下文状态

类似于 CPU 中的 malloc() 和 printf() 函数，cuCtxSetLimit() 和 cuCtxGetLimit() 函数可以设置和获取内核函数中对应函数的 GPU 空间上限。cuCtxSetCacheConfig() 函数在启动内核时指定所期望的缓存配置（是否分别分配 16kb 和 48kb 给共享内存和一级缓存）。这暗示我们，任何需要 16Kb 以上共享内存空间的内核需要分配 48Kb 的共享内存。除此之外，上下文状态可以被内核特定的状态重写（cuFuncSetCacheConfig()）。这些状态存在于整个上下文范围（换句话讲，它们并不是特定于每一次内核启动的），它们改动的代价十分昂贵。

3.4　模块与函数

模块是代码与一同加载的数据的集合，类似于 Windows 中的动态链接库（DLL）和 Linux 中的动态共享对象（DSO）。CUDA 运行时和 CUDA 上下文不显式支持模块，模块只在 CUDA 驱动程序 API 中可用[㈢]。

㈠ 直到 CUDA 2.2 中添加了 cuCtxDestroy()，在此之前上下文通过调用 cuCtxDetch() 分离。

㈡ 回头来看，英伟达公司把引用记数放在比驱动程序 API 更高的软件层会是更明智的选择。

㈢ 如果 CUDA 在即时编译中添加来自源代码的多次请求能力（像 OpenCL 能做到的一样），英伟达公司可能会发现在 CUDA 运行时中暴露模块是合适的。

CUDA 中没有类似于对象文件的此类可被整合到模块中的中间结构。取而代之的是，nvcc 直接生成可以被加载为 CUDA 模块的文件：

- 面向特定 SM 版本的 .cubin 文件
- 通过驱动程序可以编译为硬件执行代码的 .ptx

数据不需要以这些形式发送给最终的用户。CUDA 提供加载模块的 API，模块以能够嵌入在可执行文件或其他地方的 NULL 结尾字符串的形式表示[㊀]。

一旦 CUDA 模块被加载，应用程序便可以查询包含在其中的资源。资源类型分如下几种：

- 全局变量
- 函数（内核）
- 纹理引用

一点值得注意：所有的资源在模块加载时就被创建，所以查询函数不会因为缺少资源而失败。

同上下文一样，CUDA 运行时隐藏了模块的管理细节。所有的模块在 CUDART 初始化时加载。对于拥有大量 GPU 代码的应用程序，选择 CUDA 驱动程序 API 而不选择 CUDA 运行时的一个首要原因是：前者可以显式的管理加载和卸载模块。

模块通过调用 nvcc 创建。nvcc 可以创建哪种类型的模块，取决于命令行的参数，参看表 3-3。由于 cubin 已被编译至特定的 GPU 架构，它们不需要即时编译，而且可以非常快地加载。但是它们既不是向后兼容的（例如被编译到 SM 2.x 的 cubin 不能在 SM 1.x 架构上运行）也不是向前兼容的（例如编译到 SM 2.x 架构上的 cubin 不能在 SM 3.x 架构上运行）。所以只有预先准确知道目标 GPU 架构（以及相应 cubin 版本），才可以用没有嵌入相同版本 PTX 代码模块的 cubin 文件作为备用。

表 3-3 nvcc 模块类型

Nvcc 参数	描　述
-cubin	编译到特定的 GPU 架构
-ptx	驱动程序即时编译可使用的中间资源
-fatbin	Cubin 和 PTX 的组合。如果可能，加载适合的 cubin，否则，编译 PTX，只在 CUDA 运行时中有效

PTX 是一种中间语言，将作为驱动程序即时编译的输入文件。因为这项编译会消耗大量的时间，如果硬件和设备没有改变过，驱动程序对给定的 PTX 模块会保存编译过的模块并复用它们。如果驱动和硬件改变，所有的 PTX 模块也需要重新编译。

通过 fatbins 选项，CUDA 运行时会自动操作使用合适的 cubin 版本，前提是 cubin 可用，不然会编译 PTX。不同的版本以字符串的形式被嵌入在 nvcc 发出的主机 C++ 代码中。应用程序使用驱动程序 API 具有对模块的细粒度控制的优势。例如，它可以作为资源嵌入在可执

㊀ cuModuleLoadDataEx() 函数在 4.2 节中有详细描述。

行文件中，在运行时加密或生成，但是这一使用 cubins（如果不可用，则可以编译 PTX）的过程必须显式实现。

一旦模块被加载，应用程序就可以查询其中包含的资源：全局资源、函数（内核）和纹理引用。还是如前文所述：所有的这些资源在模块加载时就被创建，所以查询函数（表 3-4）不可能由于缺少资源而失败。

表 3-4　模块查询函数

函　　数	描　　述
cuModuleGetGlobal()	传回模块中的指针与符号大小
cuModuleGetTexRef()	传回模块中声明的纹理引用大小
cuModuleGetFunction()	传回模块中声明的一个内核函数

3.5　内核（函数）

在 .cu 文件中，用 __global__ 标识了内核。在 CUDA 运行时中，它们可以使用由三对尖括号（<<< >>>）构成的单行代码启动。在第 7 章给出了内核怎样被启动和怎样在 GPU 上执行的细节。

模块中的 GPU 可执行代码以内核形式呈现，或者使用 CUDA 运行时的语言集成特性启动（<<< >>> 符号）或者使用驱动程序 API 中的 cuLaunchKernel() 函数启动。在本书写作时，CUDA 还没有任何 CUDA 模块中可执行代码的动态驻留管理。当模块加载时，内核整个地加载进设备内存。

一旦模块被加载，内核可使用 cuModuleGetFunction() 函数查询，内核的属性使用 cuFuncGetAttribute() 函数查询，内核通过 cuLaunchKernel() 函数启动，cuLaunchKernel() 包含了一整套已被废弃的 API 入口点：像 cuFuncSetBlockShape() 指定下次内核启动使用的线程块大小；函数 cuParamSetv() 指定下次内核启动传入的参数；cuLaunch()、cuLaunchGrid() 和 cuLaunchGridAsync() 使用此前设置的状态启动内核。这些 API 很低效，因为这花费了太多的调用来启动一次内核，并且很多参数，像线程块大小，最好在每次请求内核启动时统一指定。

函数 cuFuncGetAttribute() 可以被用来查询指定的属性，例如：

- 每线程块最大线程数
- 静态分配共享内存的数量
- 用户分配的常量内存大小
- 每个函数使用的本地内存数量
- 函数中每个线程使用的寄存器数量
- 被编译函数的虚拟（PTX）与二进制架构版本

当使用驱动程序 API，最好使用 extern C 来禁止 C++ 中的默认名称改编行为。否则，你

必须对 cuModuleGetFunction() 使用改编的名称。

1. CUDA 运行时

在 CUDA 运行时创建的可执行文件被加载的同时，它会在主机内存创建一个全局数据结构体，来描述当 CUDA 设备创建后应分配的 CUDA 资源。一旦 CUDA 设备被初始化，这些全局数据结构体就被用来一次性创建 CUDA 资源。因为这些全局数据在 CUDA 运行时中属于进程级共享，所以使用 CUDA 运行时不能递增的加载或卸载 CUDA 模块。

因为 CUDA 运行时用 C++ 语言集成，在 API 函数（像 cudaFuncGetAttributes() 和 cudaMemcpyToSymbol()）内，内核和符号必须用名字指定（即，不是字符串字面值）。

2. 缓存设置

在费米架构中，流处理器簇有一级缓存，它可以被分割为 16KB 大小的共享内存 /48KB 一级缓存或 48KB 的共享内存 /16KB 的一级缓存[⊖]。最初，CUDA 允许以每个内核为基础配置缓存，在 CUDA 运行时中使用 cudaFuncSetCacheConfig() 或驱动程序 API 中使用 cuFuncSetCacheConfig() 设置。后来，这一状态变得全局化：使用 cuCtxSetCacheConfig()/cudaDeviceSetCacheConfig() 函数设定默认缓存配置。

3.6 设备内存

设备内存（或线性设备内存）驻留在 CUDA 地址空间上，可以被 CUDA 内核通过标准 C/C++ 指针和数组解引用操作访问。大多数的 GPU 拥有一个专用的设备内存池，直接附加在 GPU 上，通过集成内存控制器进行访问。

CUDA 硬件并不支持请求式换页，所以，所有的内存分配都被真实的物理内存支持着。不同于 CPU 应用程序，可以分配超过实际内存容量的虚拟内存。CUDA 内存分配在物理内存耗尽时会失败。关于内存怎样分配、释放和访问内存的细节会在 5.2 节给出。

1. CUDA 运行时

CUDA 运行时应用程序可以通过调用 cudaGetDeviceProperties() 函数得到设备信息并检查 cudaDeviceProp::totalGlobalMem 查询给定设备的可用内存总量。使用 cudaMalloc() 和 cudaFree() 分配和释放设备内存。cudaMallocPitch() 分配等宽内存，cudaFree() 可以用来释放等宽内存，cudaMalloc3D() 用来执行一次等宽内存的 3D 分配。

2. 驱动程序 API

驱动程序 API 应用程序可以通过调用 cuDeviceTotalMem() 函数查询给定设备的可用

⊖ SM 3.X 添加了均分缓存的能力（32KB/32KB）。

内存总量。作为选择，cuMemGetInfo() 函数除了可以查询总内存，也可以查询空闲的内存。cuMemGetInfo() 只有在 CUDA 上下文对给定 CPU 线程是当前状态时才能被调用。cuMemAlloc() 和 cuMemFree() 分配和释放设备内存。cuMemAllocPitch() 分配等宽内存，cuMemFree() 可以用来释放它。

3.7 流与事件

在 CUDA 中，流与事件使主机与设备之间的内存复制与内核操作可并发执行。后续的 CUDA 版本通过扩展流的能力来支持在一个 GPU 上的多内核并发执行，并支持多 GPU 并发执行。

CUDA 流被用来粗粒度管理多个处理单元的并发执行：

- GPU 与 CPU
- 在 SM 处理时可以执行 DMA 操作的复制引擎
- 流处理器簇（SM）
- 并发内核
- 并发执行的独立 GPU

对给定流中的请求操作是串行执行的。狭义上来讲，CUDA 流很像 CPU 的线程，因为一个 CUDA 流中的操作按顺序执行。

3.7.1 软件流水线

因为只有一个 DMA 引擎服务于 GPU 中各种各样的粗粒度硬件资源，应用软件必须对多流中执行的操作进行软件流水线操作。否则，DMA 引擎会因为每个流中的不同引擎间的同步而中断并发。6.5 节中给出怎样使用流水线技术发挥出 CUDA 流的全部优势，在 11 章中会给出更多的范例。

开普勒架构减少了软件流水线流操作的需求，使用英伟达的 Hyper-Q 技术（在 SM 3.5 中首次使用）实际上消除了软件流水线需求。

3.7.2 流回调

CUDA 5.0 引入了另一个 CPU/GPU 同步机制，这补充了现有的机制，这个新机制可使 CPU 线程等待，直到流空闲或事件被记录。流回调函数由应用程序提供，被 CUDA 注册，随后，当流到达 cuStreamAddCallback() 函数调用的节点时，被 CUDA 调用。

在流回调期间流执行被挂起，所以，为性能考虑，开发者应该小心确认，在回调时，其他流是否正处于忙碌状态。

3.7.3 NULL 流

任何一个异步的内存复制函数都可以使用 NULL 流作为参数，而且内存复制不会开始，

直到 GPU 之前的操作全部完成。实际上，NULL 流是 GPU 上所有引擎的集结地。除此之外，所有的流内存复制函数都是异步的。可能在内存复制完成之前把控制权返回给应用程序。NULL 流最有用的场合是在不需要使用多流来利用 GPU 内部的并发性时，它可以解决应用程序的 CPU/GPU 并发。一旦流操作使用 NULL 流初始化，应用程序必须使用同步函数（像 cuCtxSynchronize() 或 cudaThreadSynchronize()）来确保操作在执行下一步之前完成。但是应用程序可能在执行同步之前请求了许多此类操作。例如，应用程序可能执行：

- 一次异步主机到设备内存复制；
- 一次或多次内核启动；
- 一次异步设备到主机内存复制。

在上下文同步之前，cuCtxSynchronize() 或 cudaThreadSynchronize() 在 GPU 执行最后一个请求操作后返回。这一方法在小型的不会运行过长时间的内存复制或内核启动中非常有用。CUDA 驱动程序使用这段宝贵的时间来写入命令到 GPU，并且让 CPU 执行和 GPU 命令处理重叠进行，以提升性能。

即使在 CUDA 1.0 中，内核启动也是异步的。对任意的内核，在没有显式指定流的情况下，NULL 流被隐式指定。

3.7.4 事件

CUDA 事件表示了另一个同步机制。同 CUDA 流一同引入，记录 CUDA 事件是 CUDA 流中应用程序跟踪进度的一个方式。当之前的所有 CUDA 流的操作执行结束后，全部的 CUDA 事件通过写入一个共享同步内存位置而起作用[㊀]。查询 CUDA 事件会引发驱动程序窥探这一内存位置，报告事件是否被记录。同步 CUDA 事件会引发驱动程序等待，直到事件被记录。

可选择的是，CUDA 事件同样可以写入一个从硬件高分辨率计时器中派生出的时间戳。基于事件的计时可以比基于 CPU 的计时更加准确，尤其对小型的操作，因为这不会受限于伪非相关事件（例如页错误或网络流量），但这类事件会影响 CPU 系统时钟计时。系统时钟时间是无法更改的，因为它是最终用户看到的时间的更好近似，因此 CUDA 事件计时最好使用在生产环境中的性能调优中[㊁]。

CUDA 事件计时最好与 NULL 流一同使用。使用这一规则的原因类似于 CPU 上的**串行指令** RDTSC（读时间戳计数器）：正像 CPU 是一个超标量处理器，可以同时处理许多指令，

㊀ 指定 NULL 流给 cuEventRecord() 或 cudaEventRecord() 函数，意味着事件将会在 GPU 处理完所有待处理操作后被记录。

㊁ 此外，用于计时的 CUDA 事件不能在其他操作中使用，最近的 CUDA 版本允许“禁用计时”特性，便于在其他操作中使用 CUDA 事件，例如跨设备同步。

GPU 也可在多个流里同时运行。在没有显式序列化的情况下，一个计时操作可能无意的包含了本不想被计时的操作，也可能排除了一些应该被包含进的操作。正如 RDTSC 经常使用的技巧，我们为计时提供足够的工作，所以计时本身的执行时间可以忽略。

随机的 CUDA 事件可能会引发一次中断，硬件会为中断发出信号，使驱动程序执行一次所谓的阻塞等待。当驱动程序等待 GPU 的同时阻塞等待会挂起等待的 CPU 线程，节省 CPU 时钟周期和功率。在阻塞等待机制可用之前，CUDA 开发者普遍的抱怨 CUDA 驱动程序通过轮询内存位置让整个 CPU 核心空转来等待 GPU。但与此同时，阻塞等待可能由于处理中断的额外开销将消耗更多的时间，所以很多应用程序可能还是希望使用默认的轮询行为。

3.8　主机内存

主机内存就是 CPU 内存——那个在我们听说 CUDA 之前使用 malloc()/free() 和 new[]/delete[] 函数操作了好多年的东西。在所有运行 CUDA 的操作系统上，主机内存是虚拟化的。硬件实施的内存保护使进程在没有特殊规定的情况下无法互相读写对方的内存[㊀]。内存中的“页”，通常为 4KB 或 8KB 大小，可在不改变它们虚拟地址的情况下被重定位。特别的是，它们可被交换到磁盘上，有效地使电脑中有比物理内存更多的虚拟内存。当一个页被标记为“非驻留的”，一次访问这个页的操作会发送给操作系统“页错误”的信号，这会提示操作系统寻找一块可用的内存，复制磁盘上的数据到内存中，使虚拟的内存页指向新的物理位置从而恢复请求操作。

操作系统中管理虚拟内存的部件叫做“虚拟内存管理器”或 VMM。除了别的之外，VMM 监视内存活动并使用启发式方法决定什么时候从磁盘驱换出一个页，以及在换出页发生引用时解决“缺页故障”。

VMM 作为提供给硬件驱动的服务，使硬件直接访问主机内存变得容易。在现代计算机中，许多外设，包括控制器、网络控制器和 GPU，可以通过 DMA 读写主机内存。DMA 的前两个优点：它避免了数据复制[㊁]并且使硬件可以与 CPU 并发操作。第三个优点是硬件通过 DMA 可以得到更好的总线表现。

为了便利 DMA，操作系统 VMM 提供一个页面锁定（page-locking）功能。被 VMM 标记为锁页的内存就不能被换出了，所以物理内存地址不能被修改。一旦内存被锁页，驱动程序便可以使用 DMA 硬件引用内存的物理地址。这一硬件设置是与锁页操作本身独立且不同的操作。因为页面锁定使其低层物理地址不再为其他分配所用，太多的锁页反而会影响计算机性能。

没被锁页的内存称作可换页的（pageable）。锁页后的内存被称作锁页内存（pinned），由

㊀ 实现进程间共享的 API 包括 Windows 的 MapViewFile() 或 Linux 的 mmap()。

㊁ 这一额外复制对使用 GPU 的开发者来说很显眼，它会比磁盘或网络控制器这样的设备占用更多的带宽。无论设备是什么类型的，没有 DMA，驱动程序就必须使用 CPU 或特殊的硬件缓冲区来复制数据。

于其物理内存不能被操作系统改变（它被固定在那个位置）。

3.8.1 锁页主机内存

锁页主机内存由 CUDA 使用函数 cuMemHostAlloc() /cudaHostAlloc() 分配。该内存是页锁定的并且由当前 CUDA 上下文为 DMA 设置[㊀]。

CUDA 跟踪分配的内存范围并且自动加速引用锁页内存的内存复制操作。异步内存复制操作只在锁页内存上工作。应用程序可以通过 cuMemHostGetFlags() 函数判断一个给定的主机内存地址范围是否被锁定。

在操作系统文档中，页面锁定（page-locked）和锁页（pinned）是同义的。但是对 CUDA，锁页内存（pinned memory）是经过页锁定后且为硬件访问而映射后的主机内存，而页面锁定（page-locking）只是指操作系统的一种不允许换页的机制。

3.8.2 可分享的锁页内存

可分享锁页内存在页锁定后映射给所有的 CUDA 上下文。这一操作的底层机制十分复杂：当一个可分享锁页内存分配操作执行，它会在返回前映射到所有的 CUDA 上下文。此外，无论何时 CUDA 上下文被创建，所有的可分享锁页内存分配都会被映射到新的 CUDA 上下文中。无论是对可分享内存分配还是上下文创建，任何映射的失败都会引发内存分配错误或上下文创建错误。可喜的是，在 CUDA 4.0 中，如果 UVA 有效，所有的锁页内存分配全部可分享。

3.8.3 映射锁页内存

映射锁页内存映射到 CUDA 上下文中的地址空间中，所以内核可能读写这块内存。默认的，锁页内存不被映射到 CUDA 地址空间，所以它不能被内核的不合法写入毁坏。对集成 GPU，映射的锁页内存带来“零复制”：由于主机（CPU）和设备（GPU）共享一块内存池，它们可以不用显式复制来交换数据。

对于独立 GPU，映射锁页内存使主机内存可被内核直接读写。对于小型数据，这一特点可以消除显式内存复制命令的花销。映射的锁页内存在写入方面尤其有优势，因为这不会有延时存在。在 CUDA 4.0 中，如果 UVA（统一虚拟编址）有效，所有的锁页分配都会被映射。

3.8.4 主机内存注册

由于开发者（尤其是库开发者）有时无法分配他们想要访问的内存，CUDA 4.0 添加了注册已存在的虚拟内存地址范围的能力。cuMemHostRegister()/cudaHostRegister() 函数

㊀ CUDA 开发者经常会有疑问：页锁定内存与 CUDA 的锁页内存是否有区别？答案是：有！锁页内存由 CUDA 进行分配或注册，进行映射之后可为 GPU 直接访问，普通的页锁定内存则不是。

会接受虚拟内存范围，之后将页面锁定，并为当前 GPU（或为所有的 GPU，如果 CU_MEMHOSTREGISTER_PORTABLE 或 cudaHostRegisterPortable 被指定）映射它。主机内存注册与 UVA 有着反常关系，由于任何的适合注册的地址范围必须不包含在当 CUDA 驱动程序被初始化时为 UVA 保留的虚拟地址空间里。

3.9　CUDA 数组与纹理操作

CUDA 数组与设备内存从相同的物理内存池中分配，但是前者拥有一个细节不明的布局：为 2D 和 3D 局部性做了优化。图形驱动程序使用这些布局保存纹理，从地址中分离出数组索引，硬件可以在 2D 或 3D 元素块上操作，取代 1D 寻址。对展示出稀疏访问模式的应用程序，特别是有维度局部性的程序（例如，计算机视觉领域的应用程序），CUDA 数组无疑会胜出；对拥有常规访问模式的应用程序，尤其是那些没有重用或那些重用可以被应用程序在共享内存上显式管理的，设备内存指针是显而易见的选择。

有些应用程序，像图像处理程序，会处在选择设备指针还是 CUDA 数组的两难处境中。如果所有的因素都相同，设备内存会是更好的选择，但是如下的几点考虑可能会对选择产生影响：

- CUDA 数组不消耗 CUDA 地址空间。
- 在 WDDM 驱动程序中，系统可以自动管理 CUDA 数组的驻留。
- CUDA 可以只在设备内存上驻留，GPU 在总线上传输数据时在这两种形式之间转化。对于某些应用程序，在主机内存上保持一个等宽形式内存，在设备内存上建立 CUDA 数组是最好的处理办法。

3.9.1　纹理引用

纹理引用是 CUDA 用来设置纹理硬件解释实际内存内容的对象[㊀]。这个间接层存在的一部分原因是：这会使多个纹理引用使用不同的属性引用相同的内存变得有效。

纹理引用的属性可能是不可变的，也可能是可变的。前者在编译时间指定，不会导致应用程序表现的不正确或不定，后者中的应用程序以编译器不可见的方式改变纹理行为（见表 3-5）。例如，纹理维数（1D、2D 和 3D）是不可变的，这必须由编译器预先获取并使用正确数量的输入参数，发出正确的机器指令。相对之下，滤波与寻址模式是可变的，它们隐式改变应用程序的行为，而编译器毫不知情。

表 3-5　可变的与不可变纹理属性

不可变属性	可变属性	不可变属性	可变属性
维度	滤波模式	返回类型	标准化坐标
类型（格式化）	寻址模式		色彩空间（sRGB）转换

㊀　在 CUDA 3.2 前，纹理引用是除了显式的内存复制之外读取 CUDA 数组的唯一方式。

CUDA 运行时（语言集成特性）和 CUDA 驱动程序 API 处理纹理引用的行为十分不同。对双方来说，纹理引用都需要调用一个名为 texture 的模板来声明：

```
texture<Type, Dimension, ReadMode> Name;
```

Type 指的是纹理所要在内存中读取的对象类型，Dimension 是纹理的维度（1、2 或 3），ReadMode 指定了在纹理读取时整数纹理类型是否转化为归一化浮点数。

纹理引用必须在使用前绑定在实际内存上。硬件从 CUDA 数组绑定纹理会更有优势，但在下述情况下，应用程序可以从设备内存绑定纹理中获利：

- 作为带宽聚合器支持纹理缓存。
- 使应用程序满足合并访问限制。
- 对那些最好通过设备内存写入的应用，可以避免从内存读取时的多余副本。例如，一个视频解码器或许希望发送给设备内存一个帧，而通过纹理方式读取它们。

一旦纹理引用与实际内存绑定，CUDA 内核便可以调用 tex* 内建函数读取内存，例如表 3-6 中的 tex1D() 函数。

表 3-6 纹理内建函数

纹理类型	内建函数
设备内存	tex1Dfetch(int)
1D	tex1D(float x, float y)
2D	tex2D(float x, float y)
3D	tex3D(float x, float y, float z)
立方图纹理	texCubemap(float x, float y, float z)
分层纹理（1D）	tex1DLayered(float x, int layer)
分层纹理（2D）	tex2DLayered(float x, float y, int layer)
分层纹理（立方图）	texCubemapLayered(float x, float y, float z, int layer)

在这里对通过全局加载 / 存储或表面加载 / 存储执行的纹理读和写之间没有一致性保证。因此，CUDA 内核当心不要从那些可以使用其他方式访问的内存读取纹理。

1. CUDA 运行时

为了绑定内存到纹理上，应用程序必须调用表 3-7 中的一个函数。CUDA 运行时应用程序可以通过直接指定结构体成员修改可变的纹理引用属性。

```
texture<float, 1, cudaReadModeElementType> tex1;
...
tex1.filterMode = cudaFilterModeLinear; // enable linear filtering
tex1.normalized = true; // texture coordinates will be normalized
```

表 3-7　绑定设备内存到纹理的函数

内　　存	函　　数
1D 设备内存	cudaBindTexture()
2D 设备内存	cudaBindTexture2D()
CUDA 数组	cudaBindTextureToArray()

指定这些结构体成员会立即生效，且不用重新绑定。

2. 驱动程序 API

由于在使用驱动程序 API 时，CPU 代码和 GPU 代码之间存在严格的分区，任何 CUDA 模块中声明的纹理引用都必须通过 cuModuleGetTexRef() 查询，这个函数传回一个 CUtexref。不同于 CUDA 运行时，纹理引用必须使用所有正确的属性初始化——可变的和不变的——因为编译器不会编码纹理引用的不可变属性到 CUDA 模块中。表 3-8 总结了驱动程序 API 函数——在绑定纹理引用与内存中使用。

表 3-8　绑定内存到纹理的驱动程序 API 函数

内　　存	函　　数
1D 设备内存	cuTexRefSetAddress()
2D 设备内存	cuTexRefSetAddress2D()
CUDA 数组	cuTexRefSetArray()

3.9.2　表面引用

表面引用可以使 CUDA 内核使用表面加载 / 存储内建函数读写 CUDA 数组。它是最近添加到 CUDA 中的，在特斯拉架构 GPU 上不可用。设置表面引用的初衷是使 CUDA 内核可以直接写入数组。在表面加载 / 存储函数实现之前，内核必须先写入设备内存，再执行一次从设备到数组的内存复制并转化输出为 CUDA 数组。

纹理引用与表面引用对比明显。前者可以把输入的任何坐标转化到设置的输出格式，后者则暴露了一个常规的逐位操作接口来读写 CUDA 数组内容。

你可能会想，为什么 CUDA 没有实现表面加载 / 存储内建函数来直接在 CUDA 数组上操作（像 OpenGL 一样）。原因是想要表面操作与时俱进，能够以更加高级的方式使图形加载 / 存储操作转化为底层表达，例如将采样使用分数坐标放入 CUDA 数组，或与抗锯齿图形表面互操作。目前，CUDA 开发者将需要通过软件来完成这些操作。

3.10　图形互操作性

图形互操作性函数族使得 CUDA 可以读写属于 OpenGL 或 Direct3D API 的内存。如果

应用程序可以通过主机内存共享数据获取可以接受的性能，这些 API 就没有存在的必要了。但是本地内存的带宽可以高达 140G/s，而事实上 PCIe 总线带宽却很少能达到 6G/s，在可能的情况下让应用程序保持数据在 GPU 上是十分重要的。使用这些图形交互操作 API，CUDA 内核可以写入数据到图像和纹理中，随后合并到由 OpenGL 和 Direct 3D 执行的图形输出。

因为图形和 CUDA 应用程序必须在低层协调以开启互操作性。应用程序必须发出信号表明它们的意图来尽早执行图形交互操作。特别是，必须通过调用特殊的上下文创建 API，例如 cuD3D10CtxCreate() 或 cudaGLSetDevice()，通知 CUDA 上下文，让它知道会与给定的 API 交互。

在驱动程序之间的协调同样会促成在图形 API 和 CUDA 之间的资源共享，按以下的两个步骤进行：

1）注册：是一个成本高昂的底层操作，该操作把开发者需要共享资源的意图告诉给底层驱动程序，可能会引发所需要资源的移动或锁定。

2）映射：一个预期会频繁出现的轻量级操作。

在 CUDA 的早期版本中，图形互操作 API 按四种不同的图形 API（OpenGL、Direct3D 9、Direct3D 10 和 Direct3D 11）严格的分离开。例如，对 Direct3D 9 互操作中，下列的函数可以配合使用：

- `cuD3D9RegisterResource()/cudaD3D9RegisterResource()`
- `cuD3D9MapResources()/cudaD3D9MapResources()`
- `cuD3D9UnmapResources()/cudaD3D9UnmapResources()`
- `cuD3D9UnregisterResource()/cudaD3D9UnregisterResource()`

因为实际的硬件能力是相同的，忽略访问它们的 API，许多函数被在 CUDA 3.2 中予以合并。注册函数仍保持着 API 分离性，因为它们在绑定时需要区分 API。但映射、解映射和移除注册资源这些函数变为公用的了，对应上述函数的 CUDA 3.2 API 如下：

- `cuD3D9RegisterResource()/cudaD3D9RegisterResource()`
- `cuGraphicsMapResources()/cudaGraphicsMapResources()`
- `cuGraphicsUnmapResources()/cudaGraphicsUnmapResources()`
- `cuGraphicsUnregisterResource()/cudaGraphicsUnregisterResource()`

Direct3D 10 中的互操作 API 与上述函数全部类同，唯一不同的是开发者必须使用 cuD3D10RegisterResource()/ cudaD3D10RegisterResource() 代替 cuD3D9* 形式。

CUDA 3.2 同样增加了使用图形 API 访问 CUDA 数组形式的纹理功能。在 Direct 3D 中，纹理只是资源的一种，可以被 IDirect3DResource9 *（或 IDirect3DResource10 *，等）引用。而在 OpenGL 中，提供了一个独立的函数：cuGraphicsGLRegisterImage()。

3.11　CUDA 运行时与 CUDA 驱动程序 API

CUDA 运行时提供了方便的语言集成功能，可以使程序很轻松的从零开始。通过自动进行像初始化上下文或加载模块这样的任务，并且用其他的 C++ 代码以内联方式启动内核。CUDART 使开发者专注于怎样使代码工作的更快。少数的 CUDA 抽象，像 CUDA 模块，在 CUDART 中并不可用。

另一方面，驱动程序 API 暴露了所有的 CUDA 抽象，使开发者可以在应用程序中操作它们。驱动程序 API 没有提供任何性能优势。但它为应用程序提供了显式的资源管理，这在需要资源管理的应用程序中十分重要，例如拥有插件架构的大规模商业应用。

驱动程序 API 的运行速度并不比 CUDA 运行时显著提升，如果你正在寻找改进 CUDA 应用程序性能的方法，请到其他章节去找。

CUDA 的大部分特性对 CUDA 运行时和驱动程序 API 都可使用，只有几个例外。表 3-9 给出了详细信息：

表 3-9　CUDA 运行时 vs 驱动程序 API

特　　性	CUDART	驱动程序 API
设备内存分配	*	*
锁页主机内存分配	*	*
内存复制	*	*
CUDA 流	*	*
CUDA 事件	*	*
图形互用性	*	*
纹理支持	*	*
表面支持	*	*
cuMemGetAddressRange		*
语言集成	*	
胖二进制（Fat binary）	*	
显式 JIT 选项		*
简化内核调用	*	
显式上下文与模块管理		*
上下文迁移		*
16 位浮点纹理		*
16 位与 32 位内存赋值		*
独立编译		*

在两种 API 之间，像内存复制这样的操作趋向于功能性相同，但是接口却全然不同。但流 API 是几乎相同的。

CUDART	驱动程序 API
cudaStream_t stream;	CUstream stream;
cudaError_t status = cudaStreamCreate(&stream);	CUresult status = cuStreamCreate(&stream, 0);
...	...
status = cudaStreamSynchronize(stream);	status = cuStreamSynchronize(stream);

事件 API 有一些细微的区别，如果开发者想要指定一个标志词（创建一个阻塞事件时需要），CUDART 提供了一个独立的 cudaEventCreateWithFlags() 函数。

CUDART	驱动程序 API
cudaEvent_t eventPolling;	CUevent eventPolling;
cudaEvent_t eventBlocking;	CUevent eventBlocking;
cudaError_t status = cudaEventCreate(&eventPolling);	CUresult status = cuEventCreate(&eventPolling, 0);
cudaError_t status = cudaEventCreateWithFlags(&eventBlocking, cudaEventBlockingSync);	CUresult status = cuEventCreate(&eventBlocking, CU_EVENT_BLOCKING_SYNC);
...	...
status = cudaEventSynchronize(event);	status = cuEventSynchronize(event);

内存复制函数的接口大部分都是不同的，尽管它们的实际功能是相同的。CUDA 支持 3 种类型的内存——主机内存、设备内存、CUDA 数组——内存复制发生在它们的全排列间，并可以发生在 1D、2D 或 3D 三种结构上。所以内存复制函数要么包括大量的不同函数，要么包括小数量却能支持不同内存类型的函数。

CUDA 中的最简单的内存复制是主机与设备内存间的复制，但是这些函数接口仍然大不相同：CUDART 使用 void * 既代表主机指针又能表示设备指针，并且一个内存复制函数接受一个目标参数；而驱动程序 API 使用 void * 代表主机内存，CUdeviceptr 代表设备内存，3 个独立的函数（cuMemcpyHtoD()、cuMemcpyDtoH() 和 cuMemcpyDtoD()）对应不同的复制目标。下表给出 CUDART 和驱动程序 API 主机与内存间的相关参数列表：

CUDART	驱动程序 API
void *dptr; void *hptr; void *dptr2; status = cudaMemcpy(dptr, hptr, size, cudaMemcpyHostToDevice);	CUdeviceptr dptr; void *hptr; Cudeviceptr dptr2; status = cuMemcpyHtoD(dptr, hptr, size);
status = cudaMemcpy(hptr, dptr, size, cudaMemcpyDeviceToHost);	status = cuMemcpyDtoH(hptr, dptr, size);
status = cudaMemcpy(dptr, dptr2, size, cudaMemcpyDeviceToDevice);	status = cuMemcpyDtoD(dptr, dptr2, size);

对于 2D 和 3D 内存复制，驱动程序 API 实现了一批接受一个描述结构体并支持所有类型的内存复制的函数，这也支持低维内存复制。例如，如果需要，cuMemcpy3D() 可以代替

cuMemcpyHtoD() 执行 1D 主机设备间的内存复制：

```
CUDA_MEMCPY3D cp = {0};
cp.dstMemoryType = CU_MEMORYTYPE_DEVICE;
cp.dstDevice = dptr;
cp.srcMemoryType = CU_MEMORYTYPE_HOST;
cp.srcHost = host;
cp.WidthInBytes = bytes;
cp.Height = cp.Depth = 1;
status = cuMemcpy3D( &cp );
```

CUDART 使用描述结构体的组合来描述复杂的内存复制函数（例如，cudaMemcpy3D()），同时使用不同的函数实现不同内存类型的复制。如同 cuMemcpy3D() 函数，CUDART 中的 cudaMemcpy3D() 函数可以接受一个描述结构体来描述内存复制的双方，也包括跨维度内存复制（执行 1D 与 2D CUDA 数组中的一列之间的复制，或执行 2D CUDA 数组与 3D CUDA 数组一个切片之间的复制）。它的描述结构仅有微小改变，这个改变在于另外嵌入了其他结构体。表 3-10 给出了两个 API 的 3D 复制结构体的对照表。

表 3-10 3D 内存复制结构体

```
struct cudaMemcpy3DParms
{
    struct cudaArray *srcArray;
    struct cudaPos srcPos;
    struct cudaPitchedPtr
srcPtr;

    struct cudaArray *dstArray;
    struct cudaPos dstPos;
    struct cudaPitchedPtr
dstPtr;

    struct cudaExtent extent;
    enum cudaMemcpyKind kind;
};

struct cudaPos
{
    size_t x;
    size_t y;
    size_t z;
};

struct cudaPitchedPtr
{
    void *ptr;
    size_t pitch;
    size_t xsize;
    size_t ysize;
};

struct cudaExtent
{
    size_t width;
    size_t height;
    size_t depth;
};
```

```
typedef struct CUDA_MEMCPY3D_st {
    size_t srcXInBytes;
    size_t srcY;
    size_t srcZ;
    size_t srcLOD;
    CUmemorytype srcMemoryType;
    const void *srcHost;
    CUdeviceptr srcDevice;
    CUarray srcArray;
    void *reserved0;
    size_t srcPitch;
    size_t srcHeight;

    size_t dstXInBytes;
    size_t dstY;
    size_t dstZ;
    size_t dstLOD;
    CUmemorytype dstMemoryType;
    void *dstHost;
    CUdeviceptr dstDevice;
    CUarray dstArray;
    void *reserved1;
    size_t dstPitch;
    size_t dstHeight;

    size_t WidthInBytes;
    size_t Height;
    size_t Depth;
} CUDA_MEMCPY3D;
```

两者 3D 内存复制的用法是相似的。它们均被设计为零初始化，并且开发者需要对给定的操作设置需要的成员。例如，执行主机到 3D 数组间的复制可以按下述步骤执行：

```
struct cudaMemcpy3DParms cp = {0};
cp.srcPtr.ptr = host;
cp.srcPtr.pitch = pitch;
cp.dstArray = hArray;
cp.extent.width = Width;
cp.extent.height = Height;
cp.extent.depth = Depth;
cp.kind = cudaMemcpyHostToDevice;

status = cudaMemcpy3D( &cp );
```

```
CUDA_MEMCPY3D cp = {0};
cp.srcMemoryType = CU_MEMORYTYPE_HOST;
cp.srcHost = host;
cp.srcPitch = pitch;
cp.srcHeight = Height;
cp.dstMemoryType = CU_MEMORYTYPE_ARRAY;
cp.dstArray = hArray;
cp.WidthInBytes = Width;
cp.Height = Height;
cp.Depth = Depth;
status = cuMemcpy3D( &cp );
```

对于一个覆盖整个 CUDA 数组的 3D 复制，源和目标地址偏移量在第一行被设置为 0，且只被引用一次。不同于函数的参数，代码只需要引用复制操作需要的参数，而且如果程序执行超过一个相同的操作（例如填充不止一个设备内存区或 CUDA 数组），这个描述结构体可被复用。

HZ BOOKS
华章图书

推荐阅读

大规模分布式系统
架构与设计实战

大规模分布式存储系统
原理解析与架构实战

Hadoop应用开发
技术详解

Hadoop实战

Hadoop技术内幕
深入解析MapReduce架构设计与实现原理

Hadoop技术内幕
深入解析YARN架构设计与实现原理

Hadoop技术内幕
深入解析Hadoop Common和
HDFS架构设计与实现原理

数据挖掘与数据化运营实战
思路、方法、技巧与应用

大数据管理
数据集成的技术、方法与
最佳实践

机器学习实践指南
案例应用解析

网站数据分析
数据驱动的网站管理、优化和运营

数据挖掘
实用案例分析